forecasting-service

Python Documentation

Files: 45 | Extensions: .py

Project Structure

```
forecasting-service/
├── src/
│   └── forecasting_service/
│       ├── api/
│       │   ├── v1/
│       │   │   ├── __init__.py
│       │   │   └── predictions.py
│       │   ├── __init__.py
│       │   ├── dependencies.py
│       │   ├── main.py
│       │   └── schemas.py
│       ├── data/
│       │   ├── datasets/
│       │   │   ├── ml_dataset_final_v2.csv
│       │   │   ├── ml_dataset_final_v4.csv
│       │   │   ├── ml_dataset_v2.csv
│       │   │   ├── ml_dataset_v3.csv
│       │   │   └── ml_dataset_v4.csv
│       │   ├── models/
│       │   │   ├── feature_importance_rf_total_price.csv
│       │   │   ├── feature_importance_xgb_total_price.csv
│       │   │   ├── feature_names.json
│       │   │   ├── random_forest_total_price_final.pkl
│       │   │   ├── scaler_total_price_final.pkl
│       │   │   └── xgboost_total_price_final.pkl
│       │   ├── collector.py
│       │   ├── dataset.csv
│       │   ├── importance_comparison.png
│       │   ├── ml_dataset_v4.csv
│       │   ├── normalize_dataset.py
│       │   ├── shap_summary.png
│       │   ├── storage.py
│       │   └── train_final.py
│       ├── datasets/
│       │   ├── flats.db
│       │   ├── flats.db-shm
│       │   ├── flats.db-wal
│       │   └── flats.png
│       ├── ml/
│       │   ├── __init__.py
│       │   └── predictor.py
│       ├── models/
│       ├── parsers/
│       │   ├── avito/
│       │   │   └── __init__.py
│       │   ├── cian/
```

```
│       │   │   ├── __init__.py
│       │   │   ├── constants.py
│       │   │   ├── detail_parser.py
│       │   │   ├── models.py
│       │   │   ├── page_parser.py
│       │   │   └── parser.py
│       │   ├── common/
│       │   │   ├── __init__.py
│       │   │   ├── browser.py
│       │   │   ├── rate_limiter.py
│       │   │   └── warmup.py
│       │   ├── domclick/
│       │   │   ├── __init__.py
│       │   │   ├── constants.py
│       │   │   ├── detail_parser.py
│       │   │   ├── http_client.py
│       │   │   ├── page_parser.py
│       │   │   ├── parser.py
│       │   │   ├── ssr_state.py
│       │   │   └── state_parser.py
│       │   ├── __init__.py
│       │   └── base.py
│       ├── scripts/
│       │   ├── domclick/
│       │   │   └── collect_details.py
│       │   ├── collect_details.py
│       │   ├── collect_listings.py
│       │   ├── coverage_report.py
│       │   ├── debug_domclick.py
│       │   └── debug_html.py
│       ├── utils/
│       │   ├── __init__.py
│       │   ├── formatting.py
│       │   └── logging_setup.py
│       ├── __init__.py
│       └── config.py
├── .python-version
├── debug.py
├── pyproject.toml
└── README.md
```

Source Code (Python)

debug.py

```python
import json

import sqlite3

from pathlib import Path

from loguru import logger

from forecasting_service.parsers.domclick.http_client import DomclickClient

from forecasting_service.parsers.domclick.ssr_state import extract_ssr_state

from forecasting_service.data.storage import FlatStorage

from forecasting_service.config import DEFAULT_DB_NAME

QRATOR_JSID = "v2.0.1775797162.167.57e14b78fcCOW2mA|lM7O4rTHxVK9cCDB|7BCJdHavBGYsyrNYLYo6PkPtSCc4+6G ...

OUTPUT_DIR = "debug_ssr_states"

def run_debug_dump():

    output_path = Path(OUTPUT_DIR)

    output_path.mkdir(exist_ok=True)

    storage = FlatStorage(db_name=DEFAULT_DB_NAME)

    client = DomclickClient(QRATOR_JSID)

    conn = storage._get_conn()

    query = "SELECT id, url FROM flats WHERE url LIKE '%domclick.ru%'"

    rows = conn.execute(query).fetchall()

    logger.info(f"Начинаю обработку {len(rows)} объектов...")

    try:

        for row in rows:

            flat_id, url = row['id'], row['url']

            logger.info(f"[{flat_id}] Загрузка: {url}")

            try:

                html = client.get_page(url)

                raw_ssr = extract_ssr_state(html)

                if not raw_ssr:
```

debug.py (continued)

```
 54 |
 55 |                     logger.warning(f"  [!] SSR не найден для ID {flat_id}. Возможно, Qrator заблокир ...
 56 |
 57 |                     (output_path / f"error_{flat_id}.html").write_text(html, encoding="utf-8")
 58 |
 59 |                     continue
 60 |
 61 |                 if isinstance(raw_ssr, dict):
 62 |
 63 |                     ssr_str = json.dumps(raw_ssr)
 64 |
 65 |                 else:
 66 |
 67 |                     ssr_str = str(raw_ssr)
 68 |
 69 |                 clean_json_str = ssr_str.replace(": undefined", ": null")
 70 |
 71 |                 try:
 72 |
 73 |                     final_data = json.loads(clean_json_str)
 74 |
 75 |                     file_path = output_path / f"state_{flat_id}.json"
 76 |
 77 |                     with open(file_path, "w", encoding="utf-8") as f:
 78 |
 79 |                         json.dump(final_data, f, ensure_ascii=False, indent=4)
 80 |
 81 |                     logger.success(f"  [+] Сохранено: {file_path.name}")
 82 |
 83 |                 except json.JSONDecodeError as je:
 84 |
 85 |                     logger.error(f"  [!] Ошибка декодирования JSON для ID {flat_id}: {je}")
 86 |
 87 |                     (output_path / f"raw_bad_{flat_id}.txt").write_text(clean_json_str, encoding="ut ...
 88 |
 89 |             except Exception as e:
 90 |
 91 |                 logger.error(f"  [!] Ошибка при запросе {flat_id}: {e}")
 92 |
 93 |     finally:
 94 |
 95 |         storage.close()
 96 |
 97 |         logger.info("Скрипт завершил работу.")
 98 |
 99 | if __name__ == "__main__":
100 |
101 |     if QRATOR_JSID == "":
102 |
103 |         logger.critical("НЕОБХОДИМО ВСТАВИТЬ QRATOR_JSID В СКРИПТ!")
104 |
105 |     else:
106 |
107 |         run_debug_dump()
108 |
```

src\forecasting_service__init__.py

```
__version__ = "0.1.0"

```

src\forecasting_service\api__init__.py

```

```

src\forecasting_service\api\dependencies.py

```
from functools import lru_cache

from pathlib import Path

from ..ml.predictor import PricePredictor

MODELS_DIR = Path(__file__).parent.parent.parent.parent / "src" / "forecasting_service" / "data" / " ...

@lru_cache

def get_predictor() -> PricePredictor:

    model_path = MODELS_DIR / "xgboost_total_price_final.pkl"

    scaler_path = MODELS_DIR / "scaler_total_price_final.pkl"

    feature_names_path = MODELS_DIR / "feature_names.json"

    return PricePredictor(

        model_path=model_path,

        scaler_path=scaler_path,

        feature_names_path=feature_names_path,

    )

```

src\forecasting_service\api\main.py

```
from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from .dependencies import get_predictor

from .schemas import HealthResponse

from .v1.predictions import router as predictions_router

def create_app() -> FastAPI:

    app = FastAPI(

        title="Forecasting Service",
```

src\forecasting_service\api\main.py (continued)

```
16 |
17 |         version="0.1.0",
18 |
19 |         description="Real Estate Price Forecasting API",
20 |
21 |     )
22 |
23 |     app.add_middleware(
24 |
25 |         CORSMiddleware,
26 |
27 |         allow_origins=["*"],
28 |
29 |         allow_credentials=True,
30 |
31 |         allow_methods=["*"],
32 |
33 |         allow_headers=["*"],
34 |
35 |     )
36 |
37 |     @app.get("/health", response_model=HealthResponse, tags=["Health"])
38 |
39 |     async def health_check():
40 |
41 |         try:
42 |
43 |             predictor = get_predictor()
44 |
45 |             return HealthResponse(
46 |
47 |                 status="ok",
48 |
49 |                 service="forecasting-service",
50 |
51 |                 model_loaded=True,
52 |
53 |                 model_version=predictor._model_version,
54 |
55 |             )
56 |
57 |         except Exception as e:
58 |
59 |             return HealthResponse(
60 |
61 |                 status="error",
62 |
63 |                 service="forecasting-service",
64 |
65 |                 model_loaded=False,
66 |
67 |                 model_version="unknown",
68 |
69 |             )
70 |
71 |     app.include_router(predictions_router)
72 |
73 |     return app
74 |
75 | app = create_app()
```

src\forecasting_service\api\main.py (continued)

```
 76 |
```

src\forecasting_service\api\schemas.py

```
  1 | from pydantic import BaseModel, Field
  2 |
  3 | class PredictionRequest(BaseModel):
  4 |
  5 |     features: dict = Field(
  6 |
  7 |         ...,
  8 |
  9 |         description="Признаки квартиры (см. FlatFeatures)",
 10 |
 11 |     )
 12 |
 13 |     horizon: str = Field(
 14 |
 15 |         default="now",
 16 |
 17 |         pattern="^(now|6_months|1_year)$",
 18 |
 19 |         description="Горизонт прогноза",
 20 |
 21 |     )
 22 |
 23 | class PredictionResponse(BaseModel):
 24 |
 25 |     status: str = "ok"
 26 |
 27 |     predicted_price: float
 28 |
 29 |     predicted_price_per_sqm: float
 30 |
 31 |     confidence: float
 32 |
 33 |     model_version: str
 34 |
 35 |     horizon: str
 36 |
 37 | class HealthResponse(BaseModel):
 38 |
 39 |     status: str = "ok"
 40 |
 41 |     service: str = "forecasting-service"
 42 |
 43 |     model_loaded: bool
 44 |
 45 |     model_version: str
 46 |
```

src\forecasting_service\api\v1__init__.py

```

```

src\forecasting_service\api\v1\predictions.py

```
from __future__ import annotations

import logging

from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, status

from ...ml.predictor import FlatFeatures, PredictionResult, PricePredictor

from ..dependencies import get_predictor

from ..schemas import PredictionRequest, PredictionResponse

logger = logging.getLogger(__name__)

router = APIRouter(prefix="/api/v1/predictions", tags=["Predictions"])

@router.post("/", response_model=PredictionResponse)

async def predict_price(

    request: PredictionRequest,

    predictor: Annotated[PricePredictor, Depends(get_predictor)],

):

    try:

        features = FlatFeatures(**request.features)

        result: PredictionResult = predictor.predict(features)

        multiplier = 1.0

        if request.horizon == "6_months":

            multiplier = 1.032

        elif request.horizon == "1_year":

            multiplier = 1.068

        predicted_price = result.predicted_price * multiplier

        predicted_price_per_sqm = result.predicted_price_per_sqm * multiplier

        logger.info(

            f"Prediction completed: "

            f"price={predicted_price:.0f}, "
```

src\forecasting_service\api\v1\predictions.py (continued)

```
54 |
55 |             f"horizon={request.horizon}, "
56 |
57 |             f"multiplier={multiplier}"
58 |
59 |         )
60 |
61 |         return PredictionResponse(
62 |
63 |             status="ok",
64 |
65 |             predicted_price=predicted_price,
66 |
67 |             predicted_price_per_sqm=predicted_price_per_sqm,
68 |
69 |             confidence=result.confidence,
70 |
71 |             model_version=result.model_version,
72 |
73 |             horizon=request.horizon,
74 |
75 |         )
76 |
77 |     except Exception as e:
78 |
79 |         logger.error(f"Prediction failed: {e}", exc_info=True)
80 |
81 |         raise HTTPException(
82 |
83 |             status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
84 |
85 |             detail=f"Prediction failed: {str(e)}",
86 |
87 |         )
88 |
```

src\forecasting_service\config.py

```
1 | from pathlib import Path
2 |
3 | BASE_DIR = Path(__file__).resolve().parent
4 |
5 | DATASETS_DIR = BASE_DIR / "datasets"
6 |
7 | ARTIFACTS_DIR = BASE_DIR / "artifacts"
8 |
9 | DEBUG_DIR = BASE_DIR / "debug"
10 |
11 | PROJECT_ROOT = BASE_DIR.parent.parent.parent
12 |
13 | LOGS_DIR = PROJECT_ROOT / "logs"
14 |
15 | DEFAULT_DB_NAME = "flats.db"
16 |
17 | LOCATION = "Владивосток"
18 |
19 | DEAL_TYPE = "sale"
20 |
21 | ROOMS_TO_PARSE = ("studio", 1, 2, 3, 4, 5, 6)
```

src\forecasting_service\config.py (continued)

```
 22 |
 23 | DEFAULT_START_PAGE = 1
 24 |
 25 | DEFAULT_END_PAGE = 54
 26 |
 27 | DEFAULT_PAGE_DELAY = (5.0, 15.0)
 28 |
 29 | DEFAULT_DETAIL_DELAY = (20.0, 40.0)
 30 |
 31 | DEFAULT_BATCH_SIZE = 30
 32 |
 33 | DEFAULT_RESTART_EVERY = 7
 34 |
 35 | CRITICAL_ML_FIELDS = (
 36 |
 37 |     "price",
 38 |
 39 |     "total_meters",
 40 |
 41 |     "rooms_count",
 42 |
 43 |     "district",
 44 |
 45 |     "floor",
 46 |
 47 |     "floors_count",
 48 |
 49 |     "year_of_construction",
 50 |
 51 |     "house_material_type",
 52 |
 53 |     "finish_type",
 54 |
 55 | )
 56 |
 57 | COVERAGE_FIELDS = (
 58 |
 59 |     "price", "total_meters", "rooms_count",
 60 |
 61 |     "floor", "floors_count", "district",
 62 |
 63 |     "microdistrict", "street", "house_number",
 64 |
 65 |     "living_meters", "kitchen_meters",
 66 |
 67 |     "ceiling_height", "object_type",
 68 |
 69 |     "year_of_construction", "house_material_type",
 70 |
 71 |     "finish_type", "bathroom_type", "bathroom_count",
 72 |
 73 |     "elevator_passenger", "elevator_cargo",
 74 |
 75 |     "parking_type", "floor_type",
 76 |
 77 |     "heating_type", "balcony_count",
 78 |
 79 |     "loggia_count", "window_view",
 80 |
 81 |     "layout_type", "has_furniture",
```

src\forecasting_service\config.py (continued)

```
 82 |
 83 |     "latitude", "longitude",
 84 |
 85 |     "address_guid", "district_guid", "district_slug", "street_guid",
 86 |
 87 |     "locality", "locality_guid",
 88 |
 89 |     "is_apartment", "has_gas", "has_redevelopment",
 90 |
 91 |     "security_features", "yard_features", "infrastructure_features",
 92 |
 93 |     "views_count", "calls_count", "favorites_count",
 94 |
 95 |     "online_show", "chat_available",
 96 |
 97 |     "price_history_json", "market_price",
 98 |
 99 |     "hot_water_type",
100 |
101 |     "offer_photos_count",
102 |
103 |     "house_photos_count",
104 |
105 |     "has_plan_photo",
106 |
107 |     "has_window_photo",
108 |
109 |     "approve_for_mortgage",
110 |
111 |     "without_evaluation",
112 |
113 |     "is_individual_seller",
114 |
115 |     "tariff_name",
116 |
117 |     "area_common_property",
118 |
119 |     "area_residential_total",
120 |
121 |     "has_family_mortgage",
122 |
123 |     "has_domclick_plan",
124 |
125 | )
126 |
127 | DETAIL_FIELDS = (
128 |
129 |     "living_meters", "kitchen_meters", "ceiling_height",
130 |
131 |     "object_type", "layout_type",
132 |
133 |     "bathroom_type", "bathroom_count",
134 |
135 |     "window_view", "finish_type",
136 |
137 |     "balcony_count", "loggia_count", "has_furniture",
138 |
139 |     "year_of_construction", "house_material_type",
140 |
141 |     "floor_type",
```

src\forecasting_service\config.py (continued)

```
142 |
143 |     "elevator_passenger", "elevator_cargo",
144 |
145 |     "entrances_count",
146 |
147 |     "has_garbage_chute", "has_ramp", "has_concierge",
148 |
149 |     "parking_type", "heating_type", "is_emergency",
150 |
151 |     "jk_name", "jk_class", "jk_deadline", "developer",
152 |
153 |     "cadastral_number", "encumbrances", "owners_count",
154 |
155 |     "latitude", "longitude",
156 |
157 |     "address_guid",
158 |
159 |     "district_guid", "district_slug",
160 |
161 |     "street_guid",
162 |
163 |     "locality", "locality_guid", "province_guid",
164 |
165 |     "timezone", "timezone_offset",
166 |
167 |     "is_apartment",
168 |
169 |     "window_view_json",
170 |
171 |     "has_gas",
172 |
173 |     "has_redevelopment",
174 |
175 |     "quarters_count",
176 |
177 |     "living_quarters_count",
178 |
179 |     "hot_water_type",
180 |
181 |     "cold_water_type",
182 |
183 |     "foundation_type",
184 |
185 |     "ventilation_type",
186 |
187 |     "energy_efficiency",
188 |
189 |     "has_intercom",
190 |
191 |     "has_closed_territory",
192 |
193 |     "has_code_door",
194 |
195 |     "has_garage",
196 |
197 |     "security_features",
198 |
199 |     "yard_features",
200 |
201 |     "infrastructure_features",
```

src\forecasting_service\config.py (continued)

```
202 |
203 |     "parking_keys_json",
204 |
205 |     "security_keys_json",
206 |
207 |     "yard_keys_json",
208 |
209 |     "infrastructure_keys_json",
210 |
211 |     "jk_id",
212 |
213 |     "jk_slug",
214 |
215 |     "is_owner",
216 |
217 |     "owner_minors",
218 |
219 |     "residence_minors",
220 |
221 |     "years_ownership",
222 |
223 |     "sale_type",
224 |
225 |     "seller_name",
226 |
227 |     "seller_phone_masked",
228 |
229 |     "seller_is_agent",
230 |
231 |     "seller_is_sbol_verified",
232 |
233 |     "seller_is_esia_verified",
234 |
235 |     "seller_company_name",
236 |
237 |     "seller_company_id",
238 |
239 |     "source_type",
240 |
241 |     "views_count",
242 |
243 |     "calls_count",
244 |
245 |     "favorites_count",
246 |
247 |     "online_show",
248 |
249 |     "chat_available",
250 |
251 |     "is_auction",
252 |
253 |     "is_exclusive",
254 |
255 |     "is_placement_paid",
256 |
257 |     "duplicates_offer_count",
258 |
259 |     "published_at_source",
260 |
261 |     "updated_at_source",
```

src\forecasting_service\config.py (continued)

```
262 |
263 |     "description",
264 |
265 |     "egrn_area_status",
266 |
267 |     "egrn_floor_status",
268 |
269 |     "egrn_owners_status",
270 |
271 |     "collateral",
272 |
273 |     "collateral_sber",
274 |
275 |     "market_price_min",
276 |
277 |     "market_price_max",
278 |
279 |     "market_price",
280 |
281 |     "rent_long_predicted",
282 |
283 |     "rent_short_predicted",
284 |
285 |     "repair_quality_predicted",
286 |
287 |     "price_history_json",
288 |
289 |     "offer_photos_json",
290 |
291 |     "house_photos_json",
292 |
293 |     "address_parents_json",
294 |
295 |     "discounts_json",
296 |
297 |     "offer_photos_count",
298 |
299 |     "house_photos_count",
300 |
301 |     "has_plan_photo",
302 |
303 |     "has_window_photo",
304 |
305 |     "house_photo_facade_count",
306 |
307 |     "house_photo_lift_count",
308 |
309 |     "house_photo_entrance_inside_count",
310 |
311 |     "house_photo_entrance_outside_count",
312 |
313 |     "house_photo_territory_count",
314 |
315 |     "approve_for_mortgage",
316 |
317 |     "without_evaluation",
318 |
319 |     "is_individual_seller",
320 |
321 |     "seller_cas_id",
```

src\forecasting_service\config.py (continued)

```
322 |
323 |     "seller_phone_visible",
324 |
325 |     "tariff_name",
326 |
327 |     "tariff_display_name",
328 |
329 |     "area_common_property",
330 |
331 |     "area_residential_total",
332 |
333 |     "area_non_residential",
334 |
335 |     "parking_square",
336 |
337 |     "gas_type",
338 |
339 |     "sewerage_type",
340 |
341 |     "electrical_type",
342 |
343 |     "has_family_mortgage",
344 |
345 |     "has_domclick_plan",
346 |
347 |     "mortgage_badge",
348 |
349 |     "base_credit_rate",
350 |
351 | )
352 |
353 | JSON_DETAIL_FIELDS = (
354 |
355 |     "window_view_json",
356 |
357 |     "parking_keys_json",
358 |
359 |     "security_keys_json",
360 |
361 |     "yard_keys_json",
362 |
363 |     "infrastructure_keys_json",
364 |
365 |     "price_history_json",
366 |
367 |     "offer_photos_json",
368 |
369 |     "house_photos_json",
370 |
371 |     "address_parents_json",
372 |
373 |     "discounts_json",
374 |
375 | )
376 |
```

src\forecasting_service\data\collector.py

```
import time

import random

from datetime import datetime

from pathlib import Path

from typing import Optional

from loguru import logger

from forecasting_service.config import (

    DATASETS_DIR,

    CRITICAL_ML_FIELDS,

    DEFAULT_BATCH_SIZE,

    DEFAULT_DETAIL_DELAY,

    DEFAULT_RESTART_EVERY,

)

from forecasting_service.data.storage import FlatStorage

from forecasting_service.utils.formatting import (

    format_stats_block,

    format_coverage_block,

    format_critical_fields,

    format_session_report,

)

class DataCollector:

    def __init__(

        self,

        location: str = "Владивосток",

        db_name: str = "flats.db",

    ):

        self.location = location

        self.storage = FlatStorage(db_name=db_name)

    def collect_listings(

        self,
```

src\forecasting_service\data\collector.py (continued)

```
 60 |
 61 |         rooms: tuple = ("studio", 1, 2, 3, 4, 5, 6),
 62 |
 63 |         start_page: int = 1,
 64 |
 65 |         end_page: int = 54,
 66 |
 67 |         headless: bool = False,
 68 |
 69 |         page_delay: tuple[float, float] = (5.0, 15.0),
 70 |
 71 |     ) -> None:
 72 |
 73 |         from forecasting_service.parsers.cian.parser import (
 74 |
 75 |             CianListingParser,
 76 |
 77 |         )
 78 |
 79 |         self._log_phase_header(
 80 |
 81 |             phase=1,
 82 |
 83 |             title="СБОР ЛИСТИНГА",
 84 |
 85 |             extra={
 86 |
 87 |                 "Город": self.location,
 88 |
 89 |                 "Стр.": f"{start_page}–{end_page}",
 90 |
 91 |                 "Комнаты": str(rooms),
 92 |
 93 |             },
 94 |
 95 |         )
 96 |
 97 |         parser = CianListingParser(
 98 |
 99 |             location=self.location,
100 |
101 |             headless=headless,
102 |
103 |             page_delay=page_delay,
104 |
105 |             storage=self.storage,
106 |
107 |         )
108 |
109 |         parser.collect(
110 |
111 |             rooms=rooms,
112 |
113 |             start_page=start_page,
114 |
115 |             end_page=end_page,
116 |
117 |         )
118 |
119 |         logger.info(format_stats_block(self.storage.get_stats()))
```

src\forecasting_service\data\collector.py (continued)

```
120 |
121 |     def collect_details(
122 |
123 |         self,
124 |
125 |         batch_size: int = DEFAULT_BATCH_SIZE,
126 |
127 |         detail_delay: tuple[float, float] = DEFAULT_DETAIL_DELAY,
128 |
129 |         restart_every: int = DEFAULT_RESTART_EVERY,
130 |
131 |         headless: bool = False,
132 |
133 |         reset_blocked: bool = False,
134 |
135 |         max_captcha: int = 10,
136 |
137 |     ) -> None:
138 |
139 |         from forecasting_service.parsers.common.browser import (
140 |
141 |             BrowserManager,
142 |
143 |             CaptchaDetectedError,
144 |
145 |         )
146 |
147 |         from forecasting_service.parsers.common.rate_limiter import (
148 |
149 |             AdaptiveRateLimiter,
150 |
151 |         )
152 |
153 |         from forecasting_service.parsers.cian.detail_parser import (
154 |
155 |             parse_detail_page,
156 |
157 |         )
158 |
159 |         from forecasting_service.parsers.common.warmup import (
160 |
161 |             perform_warmup,
162 |
163 |             perform_mini_warmup,
164 |
165 |         )
166 |
167 |         if reset_blocked:
168 |
169 |             self.storage.reset_blocked()
170 |
171 |         self.storage.reset_stale_in_progress()
172 |
173 |         stats = self.storage.get_stats()
174 |
175 |         self._log_phase_header(
176 |
177 |             phase=2,
178 |
179 |             title="СБОР ДЕТАЛЕЙ",
```

src\forecasting_service\data\collector.py (continued)

```
180 |
181 |             extra={
182 |
183 |                 "Pending": stats["pending"],
184 |
185 |                 "Done": stats["done"],
186 |
187 |                 "Failed": stats["failed"],
188 |
189 |                 "Blocked": stats["blocked"],
190 |
191 |                 "Батч": batch_size,
192 |
193 |                 "Задержка": f"{detail_delay[0]}–{detail_delay[1]}c",
194 |
195 |             },
196 |
197 |         )
198 |
199 |         if stats["pending"] == 0 and stats["failed"] == 0:
200 |
201 |             logger.info("  Все объявления обработаны!")
202 |
203 |             return
204 |
205 |         browser = BrowserManager(
206 |
207 |             headless=headless,
208 |
209 |             manual_captcha=not headless,
210 |
211 |             captcha_wait_timeout=300,
212 |
213 |         )
214 |
215 |         rate_limiter = AdaptiveRateLimiter(
216 |
217 |             base_detail_delay=detail_delay,
218 |
219 |         )
220 |
221 |         processed = 0
222 |
223 |         success = 0
224 |
225 |         captcha_count = 0
226 |
227 |         try:
228 |
229 |             browser.start()
230 |
231 |             perform_warmup(browser, self.location)
232 |
233 |             while processed < batch_size:
234 |
235 |                 flat = self.storage.get_next_for_detail()
236 |
237 |                 if not flat:
238 |
239 |                     logger.info("  Нет объявлений для обработки")
```

src\forecasting_service\data\collector.py (continued)

```
240 |
241 |                     break
242 |
243 |                 flat_id = flat["id"]
244 |
245 |                 url = flat["url"]
246 |
247 |                 processed += 1
248 |
249 |                 logger.info(
250 |
251 |                     f"  [{processed}/{batch_size}] "
252 |
253 |                     f"id={flat_id} {url[:50]}..."
254 |
255 |                 )
256 |
257 |                 if processed > 1 and processed % restart_every == 0:
258 |
259 |                     logger.info("  Рестарт браузера...")
260 |
261 |                     browser.restart_with_new_ua()
262 |
263 |                     time.sleep(random.uniform(3, 5))
264 |
265 |                     perform_mini_warmup(browser, self.location)
266 |
267 |                 rate_limiter.wait_between_details()
268 |
269 |                 try:
270 |
271 |                     html = browser.get_page(url, scroll=True)
272 |
273 |                     details = parse_detail_page(html)
274 |
275 |                     self.storage.update_detail(flat_id, details)
276 |
277 |                     rate_limiter.record_success()
278 |
279 |                     success += 1
280 |
281 |                     filled = sum(
282 |
283 |                         1 for v in details.values()
284 |
285 |                         if v is not None and v != ""
286 |
287 |                     )
288 |
289 |                     logger.info(f"    {filled} полей")
290 |
291 |                 except CaptchaDetectedError:
292 |
293 |                     self.storage.mark_blocked(flat_id)
294 |
295 |                     rate_limiter.record_captcha()
296 |
297 |                     captcha_count += 1
298 |
299 |                     logger.warning(
```

src\forecasting_service\data\collector.py (continued)

```
300 |
301 |                         f"  CAPTCHA #{captcha_count} "
302 |
303 |                         f"(×{rate_limiter.multiplier:.1f})"
304 |
305 |                     )
306 |
307 |                     if captcha_count >= max_captcha:
308 |
309 |                         logger.error(
310 |
311 |                             f"  Достигнут лимит CAPTCHA ({max_captcha})"
312 |
313 |                         )
314 |
315 |                         break
316 |
317 |                 except Exception as e:
318 |
319 |                     self.storage.mark_failed(flat_id)
320 |
321 |                     rate_limiter.record_error()
322 |
323 |                     logger.warning(f"  Ошибка: {e}")
324 |
325 |         except KeyboardInterrupt:
326 |
327 |             logger.warning("  Прервано пользователем (Ctrl+C)")
328 |
329 |         finally:
330 |
331 |             browser.stop()
332 |
333 |         report = format_session_report(
334 |
335 |             processed=processed,
336 |
337 |             success=success,
338 |
339 |             captcha_count=captcha_count,
340 |
341 |             multiplier=rate_limiter.multiplier,
342 |
343 |             stats=self.storage.get_stats(),
344 |
345 |             coverage=self.storage.get_coverage(),
346 |
347 |         )
348 |
349 |         logger.info(report)
350 |
351 |     def export_csv(
352 |
353 |         self,
354 |
355 |         filename: Optional[str] = None,
356 |
357 |         only_done: bool = False,
358 |
359 |     ) -> Path:
```

src\forecasting_service\data\collector.py (continued)

```
360 |
361 |         DATASETS_DIR.mkdir(parents=True, exist_ok=True)
362 |
363 |         if filename is None:
364 |
365 |             timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
366 |
367 |             suffix = "_done" if only_done else "_all"
368 |
369 |             filename = f"vladivostok{suffix}_{timestamp}.csv"
370 |
371 |         filepath = DATASETS_DIR / filename
372 |
373 |         if only_done:
374 |
375 |             self.storage.export_done_to_csv(str(filepath))
376 |
377 |         else:
378 |
379 |             self.storage.export_to_csv(str(filepath))
380 |
381 |         return filepath
382 |
383 |     def print_coverage(self) -> None:
384 |
385 |         stats = self.storage.get_stats()
386 |
387 |         coverage = self.storage.get_coverage()
388 |
389 |         total = stats["total"]
390 |
391 |         print("\n" + "=" * 60)
392 |
393 |         print("  ОТЧЁТ О ПОКРЫТИИ ДАТАСЕТА")
394 |
395 |         print("=" * 60)
396 |
397 |         print(f"\n  Всего: {total}")
398 |
399 |         print(f"  Done:    {stats['done']}")
400 |
401 |         print(f"  Pending: {stats['pending']}")
402 |
403 |         print(f"  Failed:  {stats['failed']}")
404 |
405 |         print(f"  Blocked: {stats['blocked']}")
406 |
407 |         if not coverage:
408 |
409 |             print("\n  Нет данных")
410 |
411 |             return
412 |
413 |         done_pct = stats["done"] / total * 100 if total else 0
414 |
415 |         print(f"\n  Детали собраны: {done_pct:.1f}%")
416 |
417 |         print(format_coverage_block(coverage, total))
418 |
419 |         print(format_critical_fields(coverage, CRITICAL_ML_FIELDS))
```

src\forecasting_service\data\collector.py (continued)

```
420 |
421 |     @staticmethod
422 |
423 |     def _log_phase_header(
424 |
425 |         phase: int,
426 |
427 |         title: str,
428 |
429 |         extra: dict[str, object],
430 |
431 |     ) -> None:
432 |
433 |         logger.info("=" * 60)
434 |
435 |         logger.info(f"  ФАЗА {phase}: {title}")
436 |
437 |         for key, value in extra.items():
438 |
439 |             logger.info(f"  {key:<12} {value}")
440 |
441 |         logger.info("=" * 60)
442 |
443 |     def close(self) -> None:
444 |
445 |         self.storage.close()
446 |
447 |     def __enter__(self) -> "DataCollector":
448 |
449 |         return self
450 |
451 |     def __exit__(self, exc_type, exc_val, exc_tb) -> None:
452 |
453 |         self.close()
454 |
```

src\forecasting_service\data\normalize_dataset.py

```
  1 | import pandas as pd
  2 |
  3 | import numpy as np
  4 |
  5 | import json
  6 |
  7 | from pathlib import Path
  8 |
  9 | print("="*70)
 10 |
 11 | print(" FEATURE ENGINEERING FINAL v4 (без GUID-конфликтов)")
 12 |
 13 | print("="*70)
 14 |
 15 | df = pd.read_csv('dataset.csv', sep=',', encoding='utf-8')
 16 |
 17 | print(f"\n Загружено: {len(df)} записей")
 18 |
 19 | df = df[df['detail_status'] == 'done'].copy()
 20 |
 21 | df = df[(df['total_meters'] >= 10) & (df['total_meters'] <= 300)]
```

src\forecasting_service\data\normalize_dataset.py (continued)

```
22 |
23 | df['price_per_sqm'] = df['price'] / df['total_meters']
24 |
25 | df = df[(df['price_per_sqm'] >= 30000) & (df['price_per_sqm'] <= 500000)]
26 |
27 | print(f"✓ После фильтрации и очистки: {len(df)} записей")
28 |
29 | cols_to_drop = [
30 |
31 |     'id', 'url', 'source', 'source_id', 'cian_id', 'address_raw',
32 |
33 |     'detail_status', 'detail_attempts', 'last_attempt_at', 'created_at', 'updated_at',
34 |
35 |     'address_parents_json', 'window_view_json', 'infrastructure_keys_json',
36 |
37 |     'house_photos_json', 'parking_keys_json', 'security_keys_json', 'yard_keys_json',
38 |
39 |     'discounts_json', 'price_history_json', 'offer_photos_json',
40 |
41 |     'district_guid', 'address_guid', 'locality_guid', 'street_guid', 'province_guid',
42 |
43 |     'district_slug', 'microdistrict', 'street', 'house_number', 'underground',
44 |
45 |     'residential_complex', 'jk_slug', 'jk_id', 'title_raw', 'author', 'author_type',
46 |
47 |     'seller_name', 'seller_phone_masked', 'seller_is_agent', 'seller_company_name',
48 |
49 |     'seller_company_id', 'source_type', 'tariff_name', 'tariff_display_name',
50 |
51 |     'market_price', 'market_price_min', 'market_price_max', 'rent_long_predicted',
52 |
53 |     'rent_short_predicted', 'repair_quality_predicted', 'base_credit_rate',
54 |
55 |     'has_family_mortgage', 'mortgage_badge', 'approve_for_mortgage', 'without_evaluation',
56 |
57 |     'is_auction', 'online_show', 'chat_available', 'is_exclusive', 'is_placement_paid',
58 |
59 |     'duplicates_offer_count', 'published_at_source', 'updated_at_source'
60 |
61 | ]
62 |
63 | existing_drop = [c for c in cols_to_drop if c in df.columns]
64 |
65 | df = df.drop(columns=existing_drop)
66 |
67 | print(f" Удалено {len(existing_drop)} служебных колонок")
68 |
69 | def extract_district(address_json, slug):
70 |
71 |     try:
72 |
73 |         if pd.notna(address_json) and address_json != '[]':
74 |
75 |             data = json.loads(address_json)
76 |
77 |             for item in data:
78 |
79 |                 if 'район' in item.get('name', '').lower() or item.get('kind') == 'district':
80 |
81 |                     return item.get('name')
```

src\forecasting_service\data\normalize_dataset.py (continued)

```
 82 |
 83 |     except: pass
 84 |
 85 |     if pd.notna(slug):
 86 |
 87 |         slug_map = {'pervomajskij':'Первомайский', 'sovetskij':'Советский',
 88 |
 89 |                     'leninskij':'Ленинский', 'frunzenskij':'Фрунзенский', 'pervorechenskij':'Перворе ...
 90 |
 91 |         for k, v in slug_map.items():
 92 |
 93 |             if k in slug.lower(): return v
 94 |
 95 |     return 'unknown'
 96 |
 97 | df['district'] = df.apply(lambda r: extract_district(r.get('address_parents_json'), r.get('district_ ...
 98 |
 99 | df['year_of_construction'] = df.groupby('jk_name')['year_of_construction'].transform('median').filln ...
100 |
101 | df['latitude'] = df['latitude'].fillna(df.groupby('district')['latitude'].transform('mean')).fillna( ...
102 |
103 | df['longitude'] = df['longitude'].fillna(df.groupby('district')['longitude'].transform('mean')).fill ...
104 |
105 | CENTER_LAT, CENTER_LON = 43.1155, 131.8855
106 |
107 | SEA_LAT, SEA_LON = 43.1000, 131.9200
108 |
109 | df['dist_to_center_km'] = np.sqrt((df['latitude']-CENTER_LAT)**2 + (df['longitude']-CENTER_LON)**2)  ...
110 |
111 | df['dist_to_sea_km'] = np.sqrt((df['latitude']-SEA_LAT)**2 + (df['longitude']-SEA_LON)**2) * 111
112 |
113 | df['building_age'] = 2026 - df['year_of_construction']
114 |
115 | df['living_meters'] = df['living_meters'].fillna(df['total_meters'] * 0.70)
116 |
117 | df['kitchen_meters'] = df['kitchen_meters'].fillna(df['total_meters'] * 0.15)
118 |
119 | bool_cols = ['has_intercom','has_closed_territory','has_code_door','has_garage','has_concierge']
120 |
121 | for c in bool_cols:
122 |
123 |     if c in df.columns: df[c] = df[c].fillna(0).astype(int)
124 |
125 | for c in ['offer_photos_count','house_photos_count','has_plan_photo']:
126 |
127 |     if c in df.columns: df[c] = df[c].fillna(0).astype(int)
128 |
129 | df['house_material_type'] = df.get('house_material_type', pd.Series()).fillna('unknown')
130 |
131 | df['finish_type'] = df.get('finish_type', pd.Series()).fillna('unknown')
132 |
133 | df['jk_class'] = df.get('jk_class', pd.Series()).fillna('unknown')
134 |
135 | df['infrastructure_keys_json'] = df.get('infrastructure_keys_json', pd.Series()).fillna('[]')
136 |
137 | df['living_ratio'] = df['living_meters'] / df['total_meters']
138 |
139 | df['kitchen_ratio'] = df['kitchen_meters'] / df['total_meters']
140 |
141 | df['floor_ratio'] = df['floor'] / df['floors_count']
```

src\forecasting_service\data\normalize_dataset.py (continued)

```
142 |
143 | def cat_floor(r):
144 |
145 |     if r['floor']==1: return 'ground'
146 |
147 |     if r['floor']==r['floors_count']: return 'top'
148 |
149 |     if r['floor_ratio']<0.33: return 'low'
150 |
151 |     if r['floor_ratio']<0.66: return 'mid'
152 |
153 |     return 'high'
154 |
155 | df['floor_category'] = df.apply(cat_floor, axis=1)
156 |
157 | df['rooms_category'] = df['rooms_count'].map({0:'studio',1:'1room',2:'2room',3:'3room'}).fillna('4pl ...
158 |
159 | df['house_type'] = df['house_material_type'].str.lower().apply(lambda x: 'monolith' if 'монолит' in  ...
160 |
161 | df['renovation_category'] = df['finish_type'].str.lower().apply(lambda x: 'premium' if any(w in x fo ...
162 |
163 | df['jk_class_category'] = df['jk_class'].str.lower().apply(lambda x: 'economy' if 'эконом' in x else ...
164 |
165 | def cat_window(j):
166 |
167 |     try:
168 |
169 |         if pd.isna(j) or j=='[]': return 'unknown'
170 |
171 |         v = json.loads(j)[0].lower()
172 |
173 |         return 'yard' if 'двор' in v else 'street' if 'улиц' in v else 'park' if 'парк' in v else 'w ...
174 |
175 |     except: return 'unknown'
176 |
177 | df['window_view_category'] = df.get('window_view_json', pd.Series()).apply(cat_window)
178 |
179 | def count_infra(j):
180 |
181 |     try: return len(json.loads(j)) if pd.notna(j) and j!='[]' else 0
182 |
183 |     except: return 0
184 |
185 | df['infrastructure_count'] = df.get('infrastructure_keys_json', pd.Series()).apply(count_infra)
186 |
187 | df['security_score'] = df['has_intercom'] + df['has_closed_territory'] + df['has_code_door']
188 |
189 | df['building_age_category'] = df['building_age'].apply(lambda a: 'new' if a<=5 else 'modern' if a<=1 ...
190 |
191 | categorical_cols = ['rooms_category','floor_category','house_type','renovation_category',
192 |
193 |                     'jk_class_category','building_age_category','window_view_category','district']
194 |
195 | for c in categorical_cols:
196 |
197 |     df[c] = df[c].fillna('unknown').astype(str)
198 |
199 | df_encoded = pd.get_dummies(df, columns=categorical_cols, prefix=categorical_cols, dummy_na=False)
200 |
201 | prefixes = [f"{cat}_" for cat in categorical_cols]
```

src\forecasting_service\data\normalize_dataset.py (continued)

```
202 |
203 | one_hot_cols = [col for col in df_encoded.columns if any(col.startswith(p) for p in prefixes)]
204 |
205 | for col in one_hot_cols:
206 |
207 |     df_encoded[col] = df_encoded[col].fillna(0).astype(int)
208 |
209 | print(f"✓ One-hot колонок создано: {len(one_hot_cols)}")
210 |
211 | ml_features = [
212 |
213 |     'price_per_sqm', 'total_meters', 'living_meters', 'kitchen_meters',
214 |
215 |     'rooms_count', 'floor', 'floors_count', 'floor_ratio', 'living_ratio', 'kitchen_ratio',
216 |
217 |     'building_age', 'year_of_construction', 'latitude', 'longitude',
218 |
219 |     'dist_to_center_km', 'dist_to_sea_km', 'infrastructure_count', 'security_score',
220 |
221 |     'has_intercom', 'has_closed_territory', 'has_code_door', 'has_garage', 'has_concierge',
222 |
223 |     'offer_photos_count', 'house_photos_count', 'has_plan_photo'
224 |
225 | ] + one_hot_cols
226 |
227 | final_df = df_encoded[[c for c in ml_features if c in df_encoded.columns]].copy()
228 |
229 | final_df = final_df.fillna(0)
230 |
231 | Path('datasets').mkdir(exist_ok=True)
232 |
233 | final_df.to_csv('datasets/ml_dataset_final_v4.csv', index=False)
234 |
235 | print(f"\n Сохранено: {len(final_df)} записей, {len(final_df.columns)} признаков")
236 |
237 | print(f"\n price_per_sqm: min={final_df['price_per_sqm'].min():.0f}, max={final_df['price_per_sqm'] ...
238 |
239 | print(" Готово к обучению!")
240 |
```

src\forecasting_service\data\storage.py

```
  1 | import sqlite3
  2 |
  3 | from datetime import datetime, timedelta
  4 |
  5 | from enum import Enum
  6 |
  7 | from pathlib import Path
  8 |
  9 | from typing import Optional
 10 |
 11 | import pandas as pd
 12 |
 13 | from loguru import logger
 14 |
 15 | from forecasting_service.config import (
 16 |
 17 |     DATASETS_DIR,
```

src\forecasting_service\data\storage.py (continued)

```
18 |
19 |     DETAIL_FIELDS,
20 |
21 |     COVERAGE_FIELDS,
22 |
23 | )
24 |
25 | import json
26 |
27 | class DetailStatus(str, Enum):
28 |
29 |     PENDING = "pending"
30 |
31 |     IN_PROGRESS = "in_progress"
32 |
33 |     DONE = "done"
34 |
35 |     FAILED = "failed"
36 |
37 |     BLOCKED = "blocked"
38 |
39 | _LISTING_NUMERIC_FIELDS = (
40 |
41 |     "price", "total_meters", "rooms_count", "floor", "floors_count",
42 |
43 | )
44 |
45 | _LISTING_TEXT_FIELDS = (
46 |
47 |     "district", "microdistrict", "street", "house_number",
48 |
49 |     "residential_complex", "underground", "address_raw", "title_raw",
50 |
51 | )
52 |
53 | class FlatStorage:
54 |
55 |     def __init__(self, db_name: str = "flats.db"):
56 |
57 |         DATASETS_DIR.mkdir(parents=True, exist_ok=True)
58 |
59 |         self.db_path = DATASETS_DIR / db_name
60 |
61 |         self._conn: Optional[sqlite3.Connection] = None
62 |
63 |         self._init_db()
64 |
65 |     def _get_conn(self) -> sqlite3.Connection:
66 |
67 |         if self._conn is None:
68 |
69 |             self._conn = sqlite3.connect(str(self.db_path), timeout=30)
70 |
71 |             self._conn.row_factory = sqlite3.Row
72 |
73 |             self._conn.execute("PRAGMA journal_mode=WAL")
74 |
75 |             self._conn.execute("PRAGMA foreign_keys=ON")
76 |
77 |         return self._conn
```

src\forecasting_service\data\storage.py (continued)

```
 78 |
 79 |     def _init_db(self) -> None:
 80 |
 81 |         conn = self._get_conn()
 82 |
 83 |         conn.executescript("""
 84 |             CREATE TABLE IF NOT EXISTS flats (
 85 |                 id INTEGER PRIMARY KEY AUTOINCREMENT,
 86 |                 url TEXT UNIQUE NOT NULL,
 87 |                 source TEXT DEFAULT '',
 88 |                 source_id TEXT,
 89 |                 cian_id INTEGER,
 90 |
 91 |                 price INTEGER,
 92 |
 93 |                 total_meters REAL,
 94 |                 rooms_count INTEGER,
 95 |                 floor INTEGER,
 96 |                 floors_count INTEGER,
 97 |
 98 |                 district TEXT DEFAULT '',
 99 |                 microdistrict TEXT DEFAULT '',
100 |                 street TEXT DEFAULT '',
101 |                 house_number TEXT DEFAULT '',
102 |                 underground TEXT DEFAULT '',
103 |                 residential_complex TEXT DEFAULT '',
104 |                 address_raw TEXT DEFAULT '',
105 |
106 |                 living_meters REAL,
107 |                 kitchen_meters REAL,
108 |                 ceiling_height REAL,
109 |                 object_type TEXT DEFAULT '',
110 |                 layout_type TEXT DEFAULT '',
111 |                 bathroom_type TEXT DEFAULT '',
112 |                 bathroom_count INTEGER,
113 |                 window_view TEXT DEFAULT '',
114 |                 finish_type TEXT DEFAULT '',
115 |                 balcony_count INTEGER,
116 |                 loggia_count INTEGER,
117 |                 has_furniture INTEGER,
118 |
119 |                 year_of_construction INTEGER,
120 |                 house_material_type TEXT DEFAULT '',
121 |                 floor_type TEXT DEFAULT '',
122 |                 elevator_passenger INTEGER,
123 |                 elevator_cargo INTEGER,
124 |                 entrances_count INTEGER,
125 |                 has_garbage_chute INTEGER,
126 |                 has_ramp INTEGER,
127 |                 has_concierge INTEGER,
128 |                 parking_type TEXT DEFAULT '',
129 |                 heating_type TEXT DEFAULT '',
130 |                 is_emergency INTEGER,
131 |
132 |                 jk_name TEXT DEFAULT '',
133 |                 jk_class TEXT DEFAULT '',
134 |                 jk_deadline TEXT DEFAULT '',
135 |                 developer TEXT DEFAULT '',
136 |
137 |                 cadastral_number TEXT DEFAULT '',
```

src\forecasting_service\data\storage.py (continued)

```
                encumbrances TEXT DEFAULT '',
                owners_count INTEGER,

                title_raw TEXT DEFAULT '',
                author TEXT DEFAULT '',
                author_type TEXT DEFAULT '',

                latitude REAL,
                longitude REAL,

                address_guid TEXT DEFAULT '',
                district_guid TEXT DEFAULT '',
                district_slug TEXT DEFAULT '',
                street_guid TEXT DEFAULT '',
                locality TEXT DEFAULT '',
                locality_guid TEXT DEFAULT '',
                province_guid TEXT DEFAULT '',
                timezone TEXT DEFAULT '',
                timezone_offset INTEGER,

                is_apartment INTEGER,
                window_view_json TEXT DEFAULT '',
                has_gas INTEGER,
                has_redevelopment INTEGER,

                quarters_count INTEGER,
                living_quarters_count INTEGER,
                hot_water_type TEXT DEFAULT '',
                cold_water_type TEXT DEFAULT '',
                foundation_type TEXT DEFAULT '',
                ventilation_type TEXT DEFAULT '',
                energy_efficiency TEXT DEFAULT '',

                has_intercom INTEGER,
                has_closed_territory INTEGER,
                has_code_door INTEGER,
                has_garage INTEGER,
                security_features TEXT DEFAULT '',
                yard_features TEXT DEFAULT '',
                infrastructure_features TEXT DEFAULT '',

                parking_keys_json TEXT DEFAULT '',
                security_keys_json TEXT DEFAULT '',
                yard_keys_json TEXT DEFAULT '',
                infrastructure_keys_json TEXT DEFAULT '',

                jk_id INTEGER,
                jk_slug TEXT DEFAULT '',

                is_owner INTEGER,
                owner_minors INTEGER,
                residence_minors INTEGER,
                years_ownership TEXT DEFAULT '',
                sale_type TEXT DEFAULT '',

                seller_name TEXT DEFAULT '',
                seller_phone_masked TEXT DEFAULT '',
                seller_is_agent INTEGER,
                seller_is_sbol_verified INTEGER,
                seller_is_esia_verified INTEGER,
```

src\forecasting_service\data\storage.py (continued)

```
198 |                 seller_company_name TEXT DEFAULT '',
199 |                 seller_company_id INTEGER,
200 |                 source_type TEXT DEFAULT '',
201 |
202 |                 views_count INTEGER,
203 |                 calls_count INTEGER,
204 |                 favorites_count INTEGER,
205 |                 online_show INTEGER,
206 |                 chat_available INTEGER,
207 |                 is_auction INTEGER,
208 |                 is_exclusive INTEGER,
209 |                 is_placement_paid INTEGER,
210 |                 duplicates_offer_count INTEGER,
211 |                 published_at_source TEXT,
212 |                 updated_at_source TEXT,
213 |                 description TEXT DEFAULT '',
214 |
215 |                 egrn_area_status TEXT DEFAULT '',
216 |                 egrn_floor_status TEXT DEFAULT '',
217 |                 egrn_owners_status TEXT DEFAULT '',
218 |                 collateral INTEGER,
219 |                 collateral_sber INTEGER,
220 |
221 |                 market_price_min INTEGER,
222 |                 market_price_max INTEGER,
223 |                 market_price INTEGER,
224 |                 rent_long_predicted INTEGER,
225 |                 rent_short_predicted INTEGER,
226 |                 repair_quality_predicted INTEGER,
227 |
228 |                 price_history_json TEXT DEFAULT '',
229 |                 offer_photos_json TEXT DEFAULT '',
230 |                 house_photos_json TEXT DEFAULT '',
231 |                 address_parents_json TEXT DEFAULT '',
232 |                 discounts_json TEXT DEFAULT '',
233 |
234 |                 offer_photos_count INTEGER,
235 |                 house_photos_count INTEGER,
236 |                 has_plan_photo INTEGER,
237 |                 has_window_photo INTEGER,
238 |                 house_photo_facade_count INTEGER,
239 |                 house_photo_lift_count INTEGER,
240 |                 house_photo_entrance_inside_count INTEGER,
241 |                 house_photo_entrance_outside_count INTEGER,
242 |                 house_photo_territory_count INTEGER,
243 |
244 |                 approve_for_mortgage INTEGER,
245 |                 without_evaluation INTEGER,
246 |                 is_individual_seller INTEGER,
247 |
248 |                 seller_cas_id INTEGER,
249 |                 seller_phone_visible INTEGER,
250 |
251 |                 tariff_name TEXT DEFAULT '',
252 |                 tariff_display_name TEXT DEFAULT '',
253 |
254 |                 area_common_property REAL,
255 |                 area_residential_total REAL,
256 |                 area_non_residential REAL,
257 |                 parking_square REAL,
```

src\forecasting_service\data\storage.py (continued)

```
258 |
259 |                 gas_type INTEGER,
260 |                 sewerage_type TEXT DEFAULT '',
261 |                 electrical_type TEXT DEFAULT '',
262 |
263 |                 has_family_mortgage INTEGER,
264 |                 has_domclick_plan INTEGER,
265 |                 mortgage_badge INTEGER,
266 |                 base_credit_rate REAL,
267 |
268 |                 detail_status TEXT DEFAULT 'pending',
269 |                 detail_attempts INTEGER DEFAULT 0,
270 |                 last_attempt_at TEXT,
271 |                 created_at TEXT DEFAULT (datetime('now')),
272 |                 updated_at TEXT DEFAULT (datetime('now'))
273 |             );
274 |
275 |             CREATE INDEX IF NOT EXISTS idx_detail_status
276 |                 ON flats(detail_status);
277 |             CREATE INDEX IF NOT EXISTS idx_cian_id
278 |                 ON flats(cian_id);
279 |             CREATE INDEX IF NOT EXISTS idx_district
280 |                 ON flats(district);
281 |             CREATE INDEX IF NOT EXISTS idx_source
282 |                 ON flats(source);
283 |             CREATE INDEX IF NOT EXISTS idx_source_id
284 |                 ON flats(source, source_id);
285 |         """)
286 |
287 |         conn.commit()
288 |
289 |         logger.info(f"  БД инициализирована: {self.db_path}")
290 |
291 |     def upsert_from_listing(
292 |
293 |         self,
294 |
295 |         flat_dict: dict,
296 |
297 |         *,
298 |
299 |         _commit: bool = True,
300 |
301 |     ) -> bool:
302 |
303 |         conn = self._get_conn()
304 |
305 |         url = flat_dict.get("url", "")
306 |
307 |         if not url:
308 |
309 |             return False
310 |
311 |         existing = conn.execute(
312 |
313 |             "SELECT id FROM flats WHERE url = ?", (url,)
314 |
315 |         ).fetchone()
316 |
317 |         if existing:
```

src\forecasting_service\data\storage.py (continued)

```
318 |
319 |             self._update_from_listing(conn, flat_dict, url)
320 |
321 |             if _commit:
322 |
323 |                 conn.commit()
324 |
325 |             return False
326 |
327 |         self._insert_from_listing(conn, flat_dict, url)
328 |
329 |         if _commit:
330 |
331 |             conn.commit()
332 |
333 |         return True
334 |
335 |     def _update_from_listing(
336 |
337 |         self,
338 |
339 |         conn: sqlite3.Connection,
340 |
341 |         flat_dict: dict,
342 |
343 |         url: str,
344 |
345 |     ) -> None:
346 |
347 |         set_parts = []
348 |
349 |         values = []
350 |
351 |         for field in _LISTING_NUMERIC_FIELDS:
352 |
353 |             set_parts.append(f"{field} = COALESCE(?, {field})")
354 |
355 |             values.append(flat_dict.get(field))
356 |
357 |         for field in _LISTING_TEXT_FIELDS:
358 |
359 |             set_parts.append(f"{field} = CASE WHEN ? != '' THEN ? ELSE {field} END")
360 |
361 |             val = flat_dict.get(field, "")
362 |
363 |             values.extend([val, val])
364 |
365 |         set_parts.append("updated_at = datetime('now')")
366 |
367 |         values.append(url)
368 |
369 |         sql = f"UPDATE flats SET {', '.join(set_parts)} WHERE url = ?"
370 |
371 |         conn.execute(sql, values)
372 |
373 |     def _insert_from_listing(
374 |
375 |         self,
376 |
377 |         conn: sqlite3.Connection,
```

src\forecasting_service\data\storage.py (continued)

```
378 |
379 |         flat_dict: dict,
380 |
381 |         url: str,
382 |
383 |     ) -> None:
384 |
385 |         all_fields = ("url", "source", "source_id", "cian_id") + _LISTING_NUMERIC_FIELDS + _LISTING_ ...
386 |
387 |         field_names = ", ".join(all_fields) + ", detail_status"
388 |
389 |         placeholders = ", ".join(["?"] * len(all_fields)) + ", 'pending'"
390 |
391 |         values = [
392 |
393 |             url,
394 |
395 |             flat_dict.get("source", ""),
396 |
397 |             flat_dict.get("source_id"),
398 |
399 |             flat_dict.get("cian_id"),
400 |
401 |         ]
402 |
403 |         for field in _LISTING_NUMERIC_FIELDS:
404 |
405 |             values.append(flat_dict.get(field))
406 |
407 |         for field in _LISTING_TEXT_FIELDS:
408 |
409 |             values.append(flat_dict.get(field, ""))
410 |
411 |         sql = f"INSERT INTO flats ({field_names}) VALUES ({placeholders})"
412 |
413 |         conn.execute(sql, values)
414 |
415 |     def bulk_upsert_from_listing(
416 |
417 |         self,
418 |
419 |         flats: list[dict],
420 |
421 |     ) -> tuple[int, int]:
422 |
423 |         conn = self._get_conn()
424 |
425 |         new_count = 0
426 |
427 |         updated_count = 0
428 |
429 |         try:
430 |
431 |             for flat in flats:
432 |
433 |                 is_new = self.upsert_from_listing(flat, _commit=False)
434 |
435 |                 if is_new:
436 |
437 |                     new_count += 1
```

src\forecasting_service\data\storage.py (continued)

```
438 |
439 |                 else:
440 |
441 |                     updated_count += 1
442 |
443 |             conn.commit()
444 |
445 |         except Exception:
446 |
447 |             conn.rollback()
448 |
449 |             raise
450 |
451 |         return new_count, updated_count
452 |
453 |     def get_next_for_detail(
454 |
455 |         self,
456 |
457 |         max_attempts: int = 3,
458 |
459 |         cooldown_minutes: int = 30,
460 |
461 |         source: Optional[str] = None,
462 |
463 |     ) -> Optional[dict]:
464 |
465 |         conn = self._get_conn()
466 |
467 |         cooldown_time = (
468 |
469 |             datetime.now() - timedelta(minutes=cooldown_minutes)
470 |
471 |         ).isoformat()
472 |
473 |         source_clause = ""
474 |
475 |         params = [max_attempts, cooldown_time]
476 |
477 |         if source:
478 |
479 |             source_clause = " AND source = ? "
480 |
481 |             params.append(source)
482 |
483 |         row = conn.execute(f"""
484 |         SELECT id, url FROM flats
485 |         WHERE (
486 |             detail_status = 'pending'
487 |             OR (
488 |                 detail_status = 'failed'
489 |                 AND detail_attempts < ?
490 |                 AND (last_attempt_at IS NULL OR last_attempt_at < ?)
491 |             )
492 |         )
493 |         AND url != ''
494 |         {source_clause}
495 |         ORDER BY
496 |             CASE detail_status
497 |                 WHEN 'pending' THEN 0
```

src\forecasting_service\data\storage.py (continued)

```
498 |                 WHEN 'failed' THEN 1
499 |             END,
500 |             detail_attempts ASC
501 |         LIMIT 1
502 |         """, tuple(params)).fetchone()
503 |
504 |         if not row:
505 |
506 |             return None
507 |
508 |         conn.execute("""
509 |             UPDATE flats
510 |             SET detail_status = 'in_progress',
511 |                 last_attempt_at = datetime('now')
512 |             WHERE id = ?
513 |         """, (row["id"],))
514 |
515 |         conn.commit()
516 |
517 |         return {"id": row["id"], "url": row["url"]}
518 |
519 |     def update_detail(self, flat_id: int, details: dict) -> None:
520 |
521 |         conn = self._get_conn()
522 |
523 |         unknown_fields = [k for k in details.keys() if k not in DETAIL_FIELDS]
524 |
525 |         if unknown_fields:
526 |
527 |             logger.debug(
528 |
529 |                 f"Поля, отсутствующие в DETAIL_FIELDS и не сохранённые в БД: {unknown_fields}"
530 |
531 |             )
532 |
533 |         set_clauses = []
534 |
535 |         values = []
536 |
537 |         for field in DETAIL_FIELDS:
538 |
539 |             if field not in details or details[field] is None:
540 |
541 |                 continue
542 |
543 |             set_clauses.append(f"{field} = COALESCE(?, {field})")
544 |
545 |             val = details[field]
546 |
547 |             if isinstance(val, bool):
548 |
549 |                 val = int(val)
550 |
551 |             elif isinstance(val, (list, dict)):
552 |
553 |                 val = json.dumps(val, ensure_ascii=False)
554 |
555 |             values.append(val)
556 |
557 |         set_clauses.extend([
```

src\forecasting_service\data\storage.py (continued)

```
558 |
559 |             "detail_status = 'done'",
560 |
561 |             "detail_attempts = detail_attempts + 1",
562 |
563 |             "updated_at = datetime('now')",
564 |
565 |         ])
566 |
567 |         values.append(flat_id)
568 |
569 |         sql = f"UPDATE flats SET {', '.join(set_clauses)} WHERE id = ?"
570 |
571 |         conn.execute(sql, values)
572 |
573 |         conn.commit()
574 |
575 |     def mark_failed(self, flat_id: int) -> None:
576 |
577 |         self._update_status(flat_id, DetailStatus.FAILED)
578 |
579 |     def mark_blocked(self, flat_id: int) -> None:
580 |
581 |         self._update_status(flat_id, DetailStatus.BLOCKED)
582 |
583 |     def _update_status(self, flat_id: int, status: DetailStatus) -> None:
584 |
585 |         conn = self._get_conn()
586 |
587 |         conn.execute("""
588 |             UPDATE flats
589 |             SET detail_status = ?,
590 |                 detail_attempts = detail_attempts + 1,
591 |                 last_attempt_at = datetime('now'),
592 |                 updated_at = datetime('now')
593 |             WHERE id = ?
594 |         """, (status.value, flat_id))
595 |
596 |         conn.commit()
597 |
598 |     def reset_blocked(self) -> int:
599 |
600 |         conn = self._get_conn()
601 |
602 |         cursor = conn.execute("""
603 |             UPDATE flats
604 |             SET detail_status = 'pending'
605 |             WHERE detail_status = 'blocked'
606 |         """)
607 |
608 |         conn.commit()
609 |
610 |         count = cursor.rowcount
611 |
612 |         if count:
613 |
614 |             logger.info(f"  Сброшено {count} blocked → pending")
615 |
616 |         return count
617 |
```

src\forecasting_service\data\storage.py (continued)

```
618 |     def reset_stale_in_progress(
619 |
620 |         self,
621 |
622 |         timeout_minutes: int = 60,
623 |
624 |     ) -> int:
625 |
626 |         conn = self._get_conn()
627 |
628 |         cutoff = (
629 |
630 |             datetime.now() - timedelta(minutes=timeout_minutes)
631 |
632 |         ).isoformat()
633 |
634 |         cursor = conn.execute("""
635 |             UPDATE flats
636 |             SET detail_status = 'pending'
637 |             WHERE detail_status = 'in_progress'
638 |               AND (last_attempt_at IS NULL OR last_attempt_at < ?)
639 |         """, (cutoff,))
640 |
641 |         conn.commit()
642 |
643 |         count = cursor.rowcount
644 |
645 |         if count:
646 |
647 |             logger.info(f"  Сброшено {count} in_progress → pending")
648 |
649 |         return count
650 |
651 |     def get_stats(self) -> dict:
652 |
653 |         conn = self._get_conn()
654 |
655 |         rows = conn.execute("""
656 |             SELECT detail_status, COUNT(*) as cnt
657 |             FROM flats
658 |             GROUP BY detail_status
659 |         """).fetchall()
660 |
661 |         stats = {row["detail_status"]: row["cnt"] for row in rows}
662 |
663 |         stats["total"] = sum(stats.values())
664 |
665 |         for status in DetailStatus:
666 |
667 |             stats.setdefault(status.value, 0)
668 |
669 |         return stats
670 |
671 |     def get_coverage(self) -> dict:
672 |
673 |         conn = self._get_conn()
674 |
675 |         total = conn.execute("SELECT COUNT(*) FROM flats").fetchone()[0]
676 |
677 |         if total == 0:
```

src\forecasting_service\data\storage.py (continued)

```
678 |
679 |             return {}
680 |
681 |         parts = []
682 |
683 |         for field in COVERAGE_FIELDS:
684 |
685 |             parts.append(
686 |
687 |                 f"SUM(CASE WHEN {field} IS NOT NULL "
688 |
689 |                 f"AND {field} != '' "
690 |
691 |                 f"AND {field} != 0 "
692 |
693 |                 f"THEN 1 ELSE 0 END) AS {field}"
694 |
695 |             )
696 |
697 |         sql = f"SELECT {', '.join(parts)} FROM flats"
698 |
699 |         row = conn.execute(sql).fetchone()
700 |
701 |         coverage = {}
702 |
703 |         for i, field in enumerate(COVERAGE_FIELDS):
704 |
705 |             filled = row[i] or 0
706 |
707 |             coverage[field] = round(filled / total * 100, 1)
708 |
709 |         return coverage
710 |
711 |     def export_to_csv(self, filepath: str) -> int:
712 |
713 |         return self._export(
714 |
715 |             filepath,
716 |
717 |             "SELECT * FROM flats ORDER BY district, street",
718 |
719 |         )
720 |
721 |     def export_done_to_csv(self, filepath: str) -> int:
722 |
723 |         return self._export(
724 |
725 |             filepath,
726 |
727 |             "SELECT * FROM flats WHERE detail_status = 'done' "
728 |
729 |             "ORDER BY district",
730 |
731 |         )
732 |
733 |     def _export(self, filepath: str, query: str) -> int:
734 |
735 |         conn = self._get_conn()
736 |
737 |         df = pd.read_sql_query(query, conn)
```

src\forecasting_service\data\storage.py (continued)

```
738 |
739 |         df.to_csv(filepath, index=False, sep=";", encoding="utf-8")
740 |
741 |         logger.info(f"  Экспорт: {filepath} ({len(df)} записей)")
742 |
743 |         return len(df)
744 |
745 |     def close(self) -> None:
746 |
747 |         if self._conn:
748 |
749 |             self._conn.close()
750 |
751 |             self._conn = None
752 |
753 |     def __enter__(self) -> "FlatStorage":
754 |
755 |         return self
756 |
757 |     def __exit__(self, exc_type, exc_val, exc_tb) -> None:
758 |
759 |         self.close()
760 |
761 |     def __del__(self) -> None:
762 |
763 |         try:
764 |
765 |             self.close()
766 |
767 |         except Exception:
768 |
769 |             pass
770 |
```

src\forecasting_service\data\train_final.py

```
  1 | import pandas as pd
  2 |
  3 | import numpy as np
  4 |
  5 | from sklearn.model_selection import train_test_split, GridSearchCV
  6 |
  7 | from sklearn.preprocessing import StandardScaler
  8 |
  9 | from sklearn.ensemble import RandomForestRegressor
 10 |
 11 | from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
 12 |
 13 | import xgboost as xgb
 14 |
 15 | import joblib
 16 |
 17 | from pathlib import Path
 18 |
 19 | print("="*70)
 20 |
 21 | print(" СРАВНЕНИЕ МОДЕЛЕЙ: XGBoost vs Random Forest (total_price)")
 22 |
 23 | print("="*70)
```

src\forecasting_service\data\train_final.py (continued)

```
24 |
25 | df = pd.read_csv('datasets/ml_dataset_final_v4.csv')
26 |
27 | print(f"\n Загружено: {len(df)} записей")
28 |
29 | initial_count = len(df)
30 |
31 | df = df[df['rooms_count'] <= 3].copy()
32 |
33 | print(f" Удалено 4+ комнатных: {initial_count - len(df)} записей")
34 |
35 | print(f"✓ Осталось: {len(df)} записей")
36 |
37 | print("\n Распределение комнат:")
38 |
39 | for rooms in [0, 1, 2, 3]:
40 |
41 |     count = (df['rooms_count'] == rooms).sum()
42 |
43 |     label = 'studio' if rooms == 0 else f'{rooms}room'
44 |
45 |     print(f"  {label}: {count} ({count*100/len(df):.1f}%)")
46 |
47 | meta_cols = ['id', 'url', 'price', 'price_per_sqm']
48 |
49 | X = df.drop([col for col in meta_cols if col in df.columns] + ['total_price'], axis=1, errors='ignor ...
50 |
51 | if 'total_price' not in df.columns:
52 |
53 |     df['total_price'] = df['price_per_sqm'] * df['total_meters']
54 |
55 |     print(f"\n⚠ Восстановлена total_price из price_per_sqm * total_meters")
56 |
57 | y = df['total_price']
58 |
59 | print(f"\n✓ Признаков: {X.shape[1]}")
60 |
61 | print(f"✓ Целевая переменная: total_price")
62 |
63 | print(f"  Min: {y.min():,.0f} руб")
64 |
65 | print(f"  Max: {y.max():,.0f} руб")
66 |
67 | print(f"  Mean: {y.mean():,.0f} руб")
68 |
69 | print(f"  Median: {y.median():,.0f} руб")
70 |
71 | y_log = np.log1p(y)
72 |
73 | print(f"\n✓ Применено логарифмирование")
74 |
75 | X_train, X_test, y_train, y_test = train_test_split(
76 |
77 |     X, y_log, test_size=0.2, random_state=42
78 |
79 | )
80 |
81 | print(f"✓ Train: {len(X_train)}, Test: {len(X_test)}")
82 |
83 | scaler = StandardScaler()
```

src\forecasting_service\data\train_final.py (continued)

```
 84 |
 85 | X_train_scaled = scaler.fit_transform(X_train)
 86 |
 87 | X_test_scaled = scaler.transform(X_test)
 88 |
 89 | print("\n" + "="*70)
 90 |
 91 | print(" ОБУЧЕНИЕ XGBoost")
 92 |
 93 | print("="*70)
 94 |
 95 | param_grid_xgb = {
 96 |
 97 |     'n_estimators': [200, 300, 500],
 98 |
 99 |     'max_depth': [6, 8, 10],
100 |
101 |     'learning_rate': [0.01, 0.05, 0.1],
102 |
103 |     'subsample': [0.8, 1.0],
104 |
105 |     'colsample_bytree': [0.8, 1.0]
106 |
107 | }
108 |
109 | xgb_model = xgb.XGBRegressor(
110 |
111 |     objective='reg:squarederror',
112 |
113 |     random_state=42,
114 |
115 |     n_jobs=-1
116 |
117 | )
118 |
119 | grid_search_xgb = GridSearchCV(
120 |
121 |     xgb_model, param_grid_xgb, cv=3,
122 |
123 |     scoring='neg_mean_absolute_error',
124 |
125 |     n_jobs=-1, verbose=1
126 |
127 | )
128 |
129 | grid_search_xgb.fit(X_train_scaled, y_train)
130 |
131 | best_xgb = grid_search_xgb.best_estimator_
132 |
133 | print(f"\n✓ Лучшие параметры XGBoost: {grid_search_xgb.best_params_}")
134 |
135 | y_pred_xgb_log = best_xgb.predict(X_test_scaled)
136 |
137 | y_pred_xgb = np.expm1(y_pred_xgb_log)
138 |
139 | y_test_original = np.expm1(y_test)
140 |
141 | r2_xgb = r2_score(y_test_original, y_pred_xgb)
142 |
143 | mae_xgb = mean_absolute_error(y_test_original, y_pred_xgb)
```

src\forecasting_service\data\train_final.py (continued)

```
144 |
145 | rmse_xgb = np.sqrt(mean_squared_error(y_test_original, y_pred_xgb))
146 |
147 | mape_xgb = np.mean(np.abs((y_test_original - y_pred_xgb) / y_test_original)) * 100
148 |
149 | importance_xgb = pd.DataFrame({
150 |
151 |     'feature': X.columns,
152 |
153 |     'importance': best_xgb.feature_importances_
154 |
155 | }).sort_values('importance', ascending=False)
156 |
157 | print("\n" + "="*70)
158 |
159 | print(" ОБУЧЕНИЕ Random Forest")
160 |
161 | print("="*70)
162 |
163 | param_grid_rf = {
164 |
165 |     'n_estimators': [100, 200, 300],
166 |
167 |     'max_depth': [10, 15, 20, None],
168 |
169 |     'min_samples_split': [2, 5, 10],
170 |
171 |     'min_samples_leaf': [1, 2, 4]
172 |
173 | }
174 |
175 | rf_model = RandomForestRegressor(
176 |
177 |     random_state=42,
178 |
179 |     n_jobs=-1
180 |
181 | )
182 |
183 | grid_search_rf = GridSearchCV(
184 |
185 |     rf_model, param_grid_rf, cv=3,
186 |
187 |     scoring='neg_mean_absolute_error',
188 |
189 |     n_jobs=-1, verbose=1
190 |
191 | )
192 |
193 | grid_search_rf.fit(X_train_scaled, y_train)
194 |
195 | best_rf = grid_search_rf.best_estimator_
196 |
197 | print(f"\n✓ Лучшие параметры Random Forest: {grid_search_rf.best_params_}")
198 |
199 | y_pred_rf_log = best_rf.predict(X_test_scaled)
200 |
201 | y_pred_rf = np.expm1(y_pred_rf_log)
202 |
203 | r2_rf = r2_score(y_test_original, y_pred_rf)
```

src\forecasting_service\data\train_final.py (continued)

```
204 |
205 | mae_rf = mean_absolute_error(y_test_original, y_pred_rf)
206 |
207 | rmse_rf = np.sqrt(mean_squared_error(y_test_original, y_pred_rf))
208 |
209 | mape_rf = np.mean(np.abs((y_test_original - y_pred_rf) / y_test_original)) * 100
210 |
211 | importance_rf = pd.DataFrame({
212 |
213 |     'feature': X.columns,
214 |
215 |     'importance': best_rf.feature_importances_
216 |
217 | }).sort_values('importance', ascending=False)
218 |
219 | print("\n" + "="*70)
220 |
221 | print(" СРАВНЕНИЕ МОДЕЛЕЙ")
222 |
223 | print("="*70)
224 |
225 | print(f"\n{'Метрика':<20} {'XGBoost':<25} {'Random Forest':<25} {'Разница':<15}")
226 |
227 | print("-" * 85)
228 |
229 | print(f"{'R²':<20} {r2_xgb:<25.4f} {r2_rf:<25.4f} {r2_xgb - r2_rf:+.4f}")
230 |
231 | print(f"{'MAE':<20} {mae_xgb:<25,.0f} {mae_rf:<25,.0f} {mae_xgb - mae_rf:+,.0f}")
232 |
233 | print(f"{'RMSE':<20} {rmse_xgb:<25,.0f} {rmse_rf:<25,.0f} {rmse_xgb - rmse_rf:+,.0f}")
234 |
235 | print(f"{'MAPE':<20} {mape_xgb:<25.2f}% {mape_rf:<25.2f}% {mape_xgb - mape_rf:+.2f}%")
236 |
237 | print("\n" + "="*70)
238 |
239 | print(" ТОП-15 ВАЖНЫХ ПРИЗНАКОВ")
240 |
241 | print("="*70)
242 |
243 | print("\n XGBoost:")
244 |
245 | print(importance_xgb.head(15).to_string(index=False))
246 |
247 | print("\n Random Forest:")
248 |
249 | print(importance_rf.head(15).to_string(index=False))
250 |
251 | print("\n" + "="*70)
252 |
253 | print(" СРАВНЕНИЕ ВАЖНОСТИ ПРИЗНАКОВ")
254 |
255 | print("="*70)
256 |
257 | comparison = pd.DataFrame({
258 |
259 |     'feature': importance_xgb['feature'],
260 |
261 |     'xgb_importance': importance_xgb['importance'],
262 |
263 |     'rf_importance': importance_rf['importance']
```

src\forecasting_service\data\train_final.py (continued)

```
264 |
265 | }).head(20)
266 |
267 | comparison['rank_diff'] = (
268 |
269 |     comparison['rf_importance'].rank(ascending=False) -
270 |
271 |     comparison['xgb_importance'].rank(ascending=False)
272 |
273 | )
274 |
275 | print(comparison.to_string(index=False))
276 |
277 | Path('models').mkdir(exist_ok=True)
278 |
279 | joblib.dump(best_xgb, 'models/xgboost_total_price_final.pkl')
280 |
281 | joblib.dump(best_rf, 'models/random_forest_total_price_final.pkl')
282 |
283 | joblib.dump(scaler, 'models/scaler_total_price_final.pkl')
284 |
285 | importance_xgb.to_csv('models/feature_importance_xgb_total_price.csv', index=False)
286 |
287 | importance_rf.to_csv('models/feature_importance_rf_total_price.csv', index=False)
288 |
289 | print("\n Модели сохранены:")
290 |
291 | print("  - models/xgboost_total_price_final.pkl")
292 |
293 | print("  - models/random_forest_total_price_final.pkl")
294 |
295 | print("  - models/scaler_total_price_final.pkl")
296 |
297 | print("\n" + "="*70)
298 |
299 | print(" ИТОГОВОЕ СРАВНЕНИЕ")
300 |
301 | print("="*70)
302 |
303 | if r2_xgb > r2_rf:
304 |
305 |     print(f" Лучшая модель: XGBoost (R²={r2_xgb:.4f})")
306 |
307 |     print(f"   Превосходство над RF: +{r2_xgb - r2_rf:.4f} по R²")
308 |
309 |     print(f"   MAE лучше на: {mae_rf - mae_xgb:,.0f} руб")
310 |
311 |     print(f"   MAPE лучше на: {mape_rf - mape_xgb:.2f}%")
312 |
313 | else:
314 |
315 |     print(f" Лучшая модель: Random Forest (R²={r2_rf:.4f})")
316 |
317 |     print(f"   Превосходство над XGBoost: +{r2_rf - r2_xgb:.4f} по R²")
318 |
319 | print("\n Готово!")
320 |
321 | import json
322 |
323 | feature_names = list(X.columns)
```

src\forecasting_service\data\train_final.py (continued)

```
324 |
325 | with open('models/feature_names.json', 'w') as f:
326 |
327 |     json.dump(feature_names, f)
328 |
329 | print(f" Имена признаков сохранены: models/feature_names.json")
330 |
331 | print(f"   Всего признаков: {len(feature_names)}")
332 |
```

src\forecasting_service\ml__init__.py

```
1 |
2 |
```

src\forecasting_service\ml\predictor.py

```
 1 | from __future__ import annotations
 2 |
 3 | import json
 4 |
 5 | import logging
 6 |
 7 | from pathlib import Path
 8 |
 9 | from typing import Optional
10 |
11 | import joblib
12 |
13 | import numpy as np
14 |
15 | import pandas as pd
16 |
17 | from pydantic import BaseModel
18 |
19 | logger = logging.getLogger(__name__)
20 |
21 | class FlatFeatures(BaseModel):
22 |
23 |     total_meters: float
24 |
25 |     living_meters: float
26 |
27 |     kitchen_meters: float
28 |
29 |     rooms_count: int
30 |
31 |     floor: int
32 |
33 |     floors_count: int
34 |
35 |     floor_ratio: float
36 |
37 |     living_ratio: float
38 |
39 |     kitchen_ratio: float
40 |
41 |     building_age: float
```

src\forecasting_service\ml\predictor.py (continued)

```
 42 |
 43 |     year_of_construction: float
 44 |
 45 |     latitude: float
 46 |
 47 |     longitude: float
 48 |
 49 |     dist_to_center_km: float
 50 |
 51 |     dist_to_sea_km: float
 52 |
 53 |     infrastructure_count: int
 54 |
 55 |     security_score: int
 56 |
 57 |     has_intercom: int
 58 |
 59 |     has_closed_territory: int
 60 |
 61 |     has_code_door: int
 62 |
 63 |     has_garage: int
 64 |
 65 |     has_concierge: int
 66 |
 67 |     offer_photos_count: int
 68 |
 69 |     house_photos_count: int
 70 |
 71 |     has_plan_photo: int
 72 |
 73 | class PredictionResult(BaseModel):
 74 |
 75 |     predicted_price: float
 76 |
 77 |     predicted_price_per_sqm: float
 78 |
 79 |     confidence: float
 80 |
 81 |     model_version: str
 82 |
 83 | class PricePredictor:
 84 |
 85 |     def __init__(self, model_path: str | Path, scaler_path: str | Path, feature_names_path: str | Pa ...
 86 |
 87 |         self._model_path = Path(model_path)
 88 |
 89 |         self._scaler_path = Path(scaler_path)
 90 |
 91 |         self._feature_names_path = Path(feature_names_path)
 92 |
 93 |         self._model = None
 94 |
 95 |         self._scaler = None
 96 |
 97 |         self._feature_names: list[str] = []
 98 |
 99 |         self._model_version: str = "xgboost_total_price_v1"
100 |
101 |         self._load_model()
```

src\forecasting_service\ml\predictor.py (continued)

```
102 |
103 |     def _load_model(self) -> None:
104 |
105 |         try:
106 |
107 |             logger.info(f"Loading model from {self._model_path}")
108 |
109 |             self._model = joblib.load(self._model_path)
110 |
111 |             logger.info(f"Loading scaler from {self._scaler_path}")
112 |
113 |             self._scaler = joblib.load(self._scaler_path)
114 |
115 |             logger.info(f"Loading feature names from {self._feature_names_path}")
116 |
117 |             if self._feature_names_path.exists():
118 |
119 |                 with open(self._feature_names_path, 'r') as f:
120 |
121 |                     self._feature_names = json.load(f)
122 |
123 |                 logger.info(f" Loaded {len(self._feature_names)} feature names from JSON")
124 |
125 |             else:
126 |
127 |                 logger.warning("Feature names JSON not found, trying to extract from model...")
128 |
129 |                 self._feature_names = self._extract_feature_names_from_model()
130 |
131 |             if not self._feature_names:
132 |
133 |                 raise ValueError("Could not determine feature names")
134 |
135 |             logger.info(
136 |
137 |                 f" Model loaded successfully. "
138 |
139 |                 f"Features: {len(self._feature_names)}"
140 |
141 |             )
142 |
143 |         except FileNotFoundError as e:
144 |
145 |             logger.error(f"Model file not found: {e}")
146 |
147 |             raise
148 |
149 |         except Exception as e:
150 |
151 |             logger.error(f"Failed to load model: {e}", exc_info=True)
152 |
153 |             raise
154 |
155 |     def _extract_feature_names_from_model(self) -> list[str]:
156 |
157 |         if hasattr(self._model, 'feature_names_in_'):
158 |
159 |             names = self._model.feature_names_in_
160 |
161 |             if names is not None:
```

src\forecasting_service\ml\predictor.py (continued)

```
162 |
163 |                 return list(names)
164 |
165 |         if hasattr(self._model, 'get_booster'):
166 |
167 |             try:
168 |
169 |                 booster = self._model.get_booster()
170 |
171 |                 if booster.feature_names:
172 |
173 |                     return booster.feature_names
174 |
175 |             except Exception:
176 |
177 |                 pass
178 |
179 |         if hasattr(self._scaler, 'feature_names_in_'):
180 |
181 |             names = self._scaler.feature_names_in_
182 |
183 |             if names is not None:
184 |
185 |                 return list(names)
186 |
187 |         logger.warning("Could not extract feature names from model/scaler")
188 |
189 |         return []
190 |
191 |     def predict(self, features: FlatFeatures) -> PredictionResult:
192 |
193 |         if self._model is None or self._scaler is None:
194 |
195 |             raise RuntimeError("Model not loaded")
196 |
197 |         features_dict = features.model_dump()
198 |
199 |         df = pd.DataFrame([features_dict])
200 |
201 |         for feature_name in self._feature_names:
202 |
203 |             if feature_name not in df.columns:
204 |
205 |                 df[feature_name] = 0
206 |
207 |         df = df[self._feature_names]
208 |
209 |         X_scaled = self._scaler.transform(df)
210 |
211 |         y_pred_log = self._model.predict(X_scaled)
212 |
213 |         y_pred = np.expm1(y_pred_log)
214 |
215 |         predicted_price = float(y_pred[0])
216 |
217 |         predicted_price_per_sqm = predicted_price / features.total_meters
218 |
219 |         confidence = 0.85
220 |
221 |         return PredictionResult(
```

src\forecasting_service\ml\predictor.py (continued)

```
222 |
223 |             predicted_price=predicted_price,
224 |
225 |             predicted_price_per_sqm=predicted_price_per_sqm,
226 |
227 |             confidence=confidence,
228 |
229 |             model_version=self._model_version,
230 |
231 |         )
232 |
```

src\forecasting_service\parsers__init__.py

```
  1 | from forecasting_service.parsers.base import (
  2 |
  3 |     ListingParser,
  4 |
  5 |     DetailParser,
  6 |
  7 |     StorageProtocol,
  8 |
  9 | )
 10 |
 11 | __all__ = ["ListingParser", "DetailParser", "StorageProtocol"]
 12 |
```

src\forecasting_service\parsers\avito__init__.py

```
  1 |
  2 |
```

src\forecasting_service\parsers\base.py

```
  1 | from typing import Protocol, Optional, runtime_checkable
  2 |
  3 | @runtime_checkable
  4 |
  5 | class ListingParser(Protocol):
  6 |
  7 |     def collect(
  8 |
  9 |         self,
 10 |
 11 |         rooms: int | str | tuple,
 12 |
 13 |         start_page: int,
 14 |
 15 |         end_page: int,
 16 |
 17 |     ) -> None: ...
 18 |
 19 | @runtime_checkable
 20 |
 21 | class DetailParser(Protocol):
 22 |
 23 |     def parse(self, html: str) -> dict: ...
```

src\forecasting_service\parsers\base.py (continued)

```
 24 |
 25 | @runtime_checkable
 26 |
 27 | class StorageProtocol(Protocol):
 28 |
 29 |     def upsert_from_listing(self, flat_dict: dict) -> bool: ...
 30 |
 31 |     def bulk_upsert_from_listing(
 32 |
 33 |         self, flats: list[dict]
 34 |
 35 |     ) -> tuple[int, int]: ...
 36 |
 37 |     def get_next_for_detail(
 38 |
 39 |         self,
 40 |
 41 |         max_attempts: int = 3,
 42 |
 43 |         cooldown_minutes: int = 30,
 44 |
 45 |         source: Optional[str] = None,
 46 |
 47 |     ) -> Optional[dict]: ...
 48 |
 49 |     def update_detail(self, flat_id: int, details: dict) -> None: ...
 50 |
 51 |     def mark_failed(self, flat_id: int) -> None: ...
 52 |
 53 |     def mark_blocked(self, flat_id: int) -> None: ...
 54 |
 55 |     def get_stats(self) -> dict: ...
 56 |
 57 |     def get_coverage(self) -> dict: ...
 58 |
 59 |     def close(self) -> None: ...
 60 |
```

src\forecasting_service\parsers\cian__init__.py

```
  1 | from forecasting_service.parsers.cian.parser import CianListingParser
  2 |
  3 | from forecasting_service.parsers.cian.detail_parser import parse_detail_page
  4 |
  5 | from forecasting_service.parsers.cian.models import CianFlat
  6 |
  7 | __all__ = ["CianListingParser", "parse_detail_page", "CianFlat"]
  8 |
```

src\forecasting_service\parsers\cian\constants.py

```
REGIONS = {

    "Владивосток": 4701,

    "Москва": 1,

    "Санкт-Петербург": 2,

    "Новосибирск": 4897,

    "Екатеринбург": 4743,

    "Казань": 4777,

    "Хабаровск": 4680,

}

SUBDOMAINS = {

    "Владивосток": "vladivostok",

    "Москва": "www",

    "Санкт-Петербург": "spb",

    "Новосибирск": "novosibirsk",

    "Екатеринбург": "ekaterinburg",

    "Казань": "kazan",

    "Хабаровск": "khabarovsk",

}

ROOM_PARAMS = {

    "studio": "room9=1",

    0: "room9=1",

    1: "room1=1",

    2: "room2=1",

    3: "room3=1",

    4: "room4=1",

    5: "room5=1",

    6: "room6=1",

}

DEAL_TYPES = {

    "sale": "sale",
```

src\forecasting_service\parsers\cian\constants.py (continued)

```
60 |
61 |     "rent_long": "rent",
62 |
63 |     "rent_short": "rent",
64 |
65 | }
66 |
67 | BASE_LISTING_URL = "https://cian.ru/cat.php"
68 |
69 | LISTING_CARD_SELECTOR = '[data-name="CardComponent"]'
70 |
71 | MAX_PAGES = 54
72 |
73 | OFFERS_PER_PAGE = 28
74 |
```

src\forecasting_service\parsers\cian\detail_parser.py

```
 1 | import re
 2 |
 3 | from typing import Optional
 4 |
 5 | from bs4 import BeautifulSoup, Tag
 6 |
 7 | _FLAT_MAPPING = {
 8 |
 9 |     "тип жилья": ("object_type", "_str"),
10 |
11 |     "общая площадь": ("total_meters", "_parse_area"),
12 |
13 |     "жилая площадь": ("living_meters", "_parse_area"),
14 |
15 |     "площадь кухни": ("kitchen_meters", "_parse_area"),
16 |
17 |     "высота потолков": ("ceiling_height", "_parse_area"),
18 |
19 |     "санузел": ("_bathroom_raw", "_str"),
20 |
21 |     "вид из окон": ("window_view", "_str"),
22 |
23 |     "вид из окна": ("window_view", "_str"),
24 |
25 |     "отделка": ("finish_type", "_str"),
26 |
27 |     "ремонт": ("finish_type", "_str"),
28 |
29 |     "балкон/лоджия": ("_balcony_raw", "_str"),
30 |
31 |     "балкон": ("_balcony_raw", "_str"),
32 |
33 |     "планировка": ("layout_type", "_str"),
34 |
35 |     "мебель": ("has_furniture", "_parse_furniture"),
36 |
37 | }
38 |
39 | _BUILDING_MAPPING = {
40 |
41 |     "год постройки": ("year_of_construction", "_parse_year"),
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
 42 |
 43 |     "количество лифтов": ("_elevator_raw", "_str"),
 44 |
 45 |     "лифт": ("_elevator_raw", "_str"),
 46 |
 47 |     "тип дома": ("house_material_type", "_str"),
 48 |
 49 |     "тип перекрытий": ("floor_type", "_str"),
 50 |
 51 |     "подъезды": ("entrances_count", "_parse_int"),
 52 |
 53 |     "о подъезде": ("_entrance_info_raw", "_str"),
 54 |
 55 |     "о\xa0подъезде": ("_entrance_info_raw", "_str"),
 56 |
 57 |     "парковка": ("parking_type", "_str"),
 58 |
 59 |     "отопление": ("heating_type", "_str"),
 60 |
 61 |     "аварийность": ("is_emergency", "_parse_emergency"),
 62 |
 63 | }
 64 |
 65 | _PARSERS: dict[str, callable] = {}
 66 |
 67 | def parse_detail_page(html: str) -> dict:
 68 |
 69 |     soup = BeautifulSoup(html, "lxml")
 70 |
 71 |     details = {}
 72 |
 73 |     _parse_about_flat(soup, details)
 74 |
 75 |     _parse_about_building(soup, details)
 76 |
 77 |     _parse_about_jk(soup, details)
 78 |
 79 |     _parse_rosreestr(soup, details)
 80 |
 81 |     return details
 82 |
 83 | def _parse_about_flat(soup: BeautifulSoup, details: dict) -> None:
 84 |
 85 |     group = _find_info_group(soup, "квартир")
 86 |
 87 |     if not group:
 88 |
 89 |         return
 90 |
 91 |     items = _extract_info_items(group)
 92 |
 93 |     _apply_mapping(items, _FLAT_MAPPING, details)
 94 |
 95 |     if "_bathroom_raw" in details:
 96 |
 97 |         _parse_bathroom(details.pop("_bathroom_raw"), details)
 98 |
 99 |     if "_balcony_raw" in details:
100 |
101 |         _parse_balcony(details.pop("_balcony_raw"), details)
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
102 |
103 | def _parse_about_building(soup: BeautifulSoup, details: dict) -> None:
104 |
105 |     group = _find_info_group(soup, "доме")
106 |
107 |     if not group:
108 |
109 |         return
110 |
111 |     items = _extract_info_items(group)
112 |
113 |     _apply_mapping(items, _BUILDING_MAPPING, details)
114 |
115 |     if "_elevator_raw" in details:
116 |
117 |         _parse_elevators(details.pop("_elevator_raw"), details)
118 |
119 |     if "_entrance_info_raw" in details:
120 |
121 |         _parse_entrance_info(details.pop("_entrance_info_raw"), details)
122 |
123 | def _parse_about_jk(soup: BeautifulSoup, details: dict) -> None:
124 |
125 |     jk_section = None
126 |
127 |     for h2 in soup.select("h2"):
128 |
129 |         text = h2.get_text(strip=True)
130 |
131 |         if text.startswith("О ЖК") or text.startswith("О\xa0ЖК"):
132 |
133 |             jk_section = h2.find_parent("div", class_=re.compile("inner"))
134 |
135 |             break
136 |
137 |     if not jk_section:
138 |
139 |         return
140 |
141 |     for h2 in jk_section.select("h2"):
142 |
143 |         text = h2.get_text(strip=True)
144 |
145 |         jk_match = re.search(r'[«""](.+?)[»""]', text)
146 |
147 |         if jk_match:
148 |
149 |             details["jk_name"] = jk_match.group(1)
150 |
151 |     for li in jk_section.select("li"):
152 |
153 |         spans = li.select("span")
154 |
155 |         if len(spans) >= 2:
156 |
157 |             name = spans[0].get_text(strip=True).lower()
158 |
159 |             value = spans[-1].get_text(strip=True)
160 |
161 |             if "сдача" in name:
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
162 |
163 |                 details["jk_deadline"] = value
164 |
165 |             elif "класс" in name:
166 |
167 |                 details["jk_class"] = value
168 |
169 |             elif "тип дома" in name:
170 |
171 |                 details.setdefault("house_material_type", value)
172 |
173 |             elif "парковка" in name:
174 |
175 |                 details.setdefault("parking_type", value)
176 |
177 |             elif "отделка" in name:
178 |
179 |                 details.setdefault("finish_type", value)
180 |
181 |         link = li.select_one('a[data-mark="Link"]')
182 |
183 |         if link:
184 |
185 |             label = li.select_one("span")
186 |
187 |             if label and "застройщик" in label.get_text(strip=True).lower():
188 |
189 |                 details["developer"] = link.get_text(strip=True)
190 |
191 | def _parse_rosreestr(soup: BeautifulSoup, details: dict) -> None:
192 |
193 |     rosreestr = soup.select_one('[data-name="RosreestrSection"]')
194 |
195 |     if not rosreestr:
196 |
197 |         return
198 |
199 |     items = rosreestr.select('[data-name="NameValueListItem"]')
200 |
201 |     for item in items:
202 |
203 |         name_el = item.select_one("dt")
204 |
205 |         value_el = item.select_one("dd")
206 |
207 |         if not name_el or not value_el:
208 |
209 |             continue
210 |
211 |         name = name_el.get_text(strip=True).lower()
212 |
213 |         value = value_el.get_text(strip=True)
214 |
215 |         if "обременен" in name:
216 |
217 |             details["encumbrances"] = value
218 |
219 |         elif "собственник" in name:
220 |
221 |             details["owners_count"] = _parse_int(value)
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
222 |
223 |         elif "кадастровый" in name:
224 |
225 |             details["cadastral_number"] = value
226 |
227 | def _find_info_group(soup: BeautifulSoup, keyword: str) -> Optional[Tag]:
228 |
229 |     groups = soup.select('[data-name="OfferSummaryInfoGroup"]')
230 |
231 |     for group in groups:
232 |
233 |         header = group.select_one("h2")
234 |
235 |         if header and keyword in header.get_text(strip=True).lower():
236 |
237 |             return group
238 |
239 |     return None
240 |
241 | def _extract_info_items(container: Tag) -> list[tuple[str, str]]:
242 |
243 |     items = []
244 |
245 |     info_divs = container.select('[data-name="OfferSummaryInfoItem"]')
246 |
247 |     for div in info_divs:
248 |
249 |         paragraphs = div.select("p")
250 |
251 |         if len(paragraphs) >= 2:
252 |
253 |             name = paragraphs[0].get_text(strip=True)
254 |
255 |             value = paragraphs[1].get_text(strip=True)
256 |
257 |             items.append((name, value))
258 |
259 |     return items
260 |
261 | def _apply_mapping(
262 |
263 |     items: list[tuple[str, str]],
264 |
265 |     mapping: dict[str, tuple[str, str]],
266 |
267 |     details: dict,
268 |
269 | ) -> None:
270 |
271 |     for name, value in items:
272 |
273 |         name_lower = name.lower()
274 |
275 |         for key, (field, parser_name) in mapping.items():
276 |
277 |             if key in name_lower:
278 |
279 |                 parser_func = _PARSERS[parser_name]
280 |
281 |                 details[field] = parser_func(value)
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
282 |
283 |                 break
284 |
285 | def _str(value: str) -> str:
286 |
287 |     return value.strip()
288 |
289 | def _parse_area(value: str) -> Optional[float]:
290 |
291 |     match = re.search(r'([\d]+[,.]?\d*)', value)
292 |
293 |     if match:
294 |
295 |         return float(match.group(1).replace(",", "."))
296 |
297 |     return None
298 |
299 | def _parse_year(value: str) -> Optional[int]:
300 |
301 |     match = re.search(r'((?:19|20)\d{2})', value)
302 |
303 |     if match:
304 |
305 |         return int(match.group(1))
306 |
307 |     return None
308 |
309 | def _parse_int(value: str) -> Optional[int]:
310 |
311 |     match = re.search(r'(\d+)', value)
312 |
313 |     if match:
314 |
315 |         return int(match.group(1))
316 |
317 |     return None
318 |
319 | def _parse_emergency(value: str) -> Optional[bool]:
320 |
321 |     lower = value.lower()
322 |
323 |     if "нет" in lower:
324 |
325 |         return False
326 |
327 |     if "да" in lower or "есть" in lower:
328 |
329 |         return True
330 |
331 |     return None
332 |
333 | def _parse_furniture(value: str) -> Optional[bool]:
334 |
335 |     lower = value.lower()
336 |
337 |     if "с мебелью" in lower or "есть" in lower:
338 |
339 |         return True
340 |
341 |     if "без мебели" in lower or "нет" in lower:
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
342 |
343 |         return False
344 |
345 |     return None
346 |
347 | def _parse_bathroom(raw: str, details: dict) -> None:
348 |
349 |     lower = raw.lower()
350 |
351 |     details["bathroom_count"] = _parse_int(raw) or 1
352 |
353 |     if "раздельн" in lower:
354 |
355 |         details["bathroom_type"] = "раздельный"
356 |
357 |     elif "совмещ" in lower:
358 |
359 |         details["bathroom_type"] = "совмещённый"
360 |
361 |     else:
362 |
363 |         details["bathroom_type"] = raw
364 |
365 | def _parse_balcony(raw: str, details: dict) -> None:
366 |
367 |     lower = raw.lower()
368 |
369 |     balcony_match = re.search(r'(\d+)\s*балкон', lower)
370 |
371 |     if balcony_match:
372 |
373 |         details["balcony_count"] = int(balcony_match.group(1))
374 |
375 |     elif "балкон" in lower:
376 |
377 |         details["balcony_count"] = 1
378 |
379 |     loggia_match = re.search(r'(\d+)\s*лоджи', lower)
380 |
381 |     if loggia_match:
382 |
383 |         details["loggia_count"] = int(loggia_match.group(1))
384 |
385 |     elif "лоджи" in lower:
386 |
387 |         details["loggia_count"] = 1
388 |
389 | def _parse_elevators(raw: str, details: dict) -> None:
390 |
391 |     lower = raw.lower()
392 |
393 |     pass_match = re.search(r'(\d+)\s*пассажирск', lower)
394 |
395 |     if pass_match:
396 |
397 |         details["elevator_passenger"] = int(pass_match.group(1))
398 |
399 |     cargo_match = re.search(r'(\d+)\s*грузов', lower)
400 |
401 |     if cargo_match:
```

src\forecasting_service\parsers\cian\detail_parser.py (continued)

```
402 |
403 |         details["elevator_cargo"] = int(cargo_match.group(1))
404 |
405 |     if not pass_match and not cargo_match and "есть" in lower:
406 |
407 |         details["elevator_passenger"] = 1
408 |
409 | def _parse_entrance_info(raw: str, details: dict) -> None:
410 |
411 |     lower = raw.lower()
412 |
413 |     if "мусоропровод" in lower:
414 |
415 |         details["has_garbage_chute"] = True
416 |
417 |     if "консьерж" in lower:
418 |
419 |         details["has_concierge"] = True
420 |
421 |     if "пандус" in lower:
422 |
423 |         details["has_ramp"] = True
424 |
425 | _PARSERS = {
426 |
427 |     "_str": _str,
428 |
429 |     "_parse_area": _parse_area,
430 |
431 |     "_parse_year": _parse_year,
432 |
433 |     "_parse_int": _parse_int,
434 |
435 |     "_parse_emergency": _parse_emergency,
436 |
437 |     "_parse_furniture": _parse_furniture,
438 |
439 | }
440 |
```

src\forecasting_service\parsers\cian\models.py

```
  1 | from typing import Optional
  2 |
  3 | from pydantic import BaseModel, computed_field
  4 |
  5 | class CianFlat(BaseModel):
  6 |
  7 |     url: str = ""
  8 |
  9 |     cian_id: Optional[int] = None
 10 |
 11 |     price: Optional[int] = None
 12 |
 13 |     total_meters: Optional[float] = None
 14 |
 15 |     rooms_count: Optional[int] = None
 16 |
 17 |     floor: Optional[int] = None
```

src\forecasting_service\parsers\cian\models.py (continued)

```
18 |
19 |     floors_count: Optional[int] = None
20 |
21 |     district: str = ""
22 |
23 |     microdistrict: str = ""
24 |
25 |     street: str = ""
26 |
27 |     house_number: str = ""
28 |
29 |     underground: str = ""
30 |
31 |     residential_complex: str = ""
32 |
33 |     address_raw: str = ""
34 |
35 |     living_meters: Optional[float] = None
36 |
37 |     kitchen_meters: Optional[float] = None
38 |
39 |     ceiling_height: Optional[float] = None
40 |
41 |     object_type: str = ""
42 |
43 |     layout_type: str = ""
44 |
45 |     bathroom_type: str = ""
46 |
47 |     bathroom_count: Optional[int] = None
48 |
49 |     window_view: str = ""
50 |
51 |     finish_type: str = ""                                                    ...
52 |
53 |     balcony_count: Optional[int] = None
54 |
55 |     loggia_count: Optional[int] = None
56 |
57 |     has_furniture: Optional[bool] = None
58 |
59 |     year_of_construction: Optional[int] = None
60 |
61 |     house_material_type: str = ""                                            ...
62 |
63 |     floor_type: str = ""
64 |
65 |     elevator_passenger: Optional[int] = None
66 |
67 |     elevator_cargo: Optional[int] = None
68 |
69 |     entrances_count: Optional[int] = None
70 |
71 |     has_garbage_chute: Optional[bool] = None
72 |
73 |     has_ramp: Optional[bool] = None
74 |
75 |     has_concierge: Optional[bool] = None
76 |
77 |     parking_type: str = ""
```

src\forecasting_service\parsers\cian\models.py (continued)

```
 78 |
 79 |     heating_type: str = ""
 80 |
 81 |     is_emergency: Optional[bool] = None
 82 |
 83 |     jk_name: str = ""
 84 |
 85 |     jk_class: str = ""
 86 |
 87 |     jk_deadline: str = ""
 88 |
 89 |     developer: str = ""
 90 |
 91 |     cadastral_number: str = ""
 92 |
 93 |     encumbrances: str = ""
 94 |
 95 |     owners_count: Optional[int] = None
 96 |
 97 |     title_raw: str = ""
 98 |
 99 |     author: str = ""
100 |
101 |     author_type: str = ""
102 |
103 |     is_detail_parsed: bool = False
104 |
105 |     @computed_field
106 |
107 |     @property
108 |
109 |     def price_per_sqm(self) -> Optional[float]:
110 |
111 |         if self.price and self.total_meters and self.total_meters > 0:
112 |
113 |             return round(self.price / self.total_meters, 2)
114 |
115 |         return None
116 |
117 |     @computed_field
118 |
119 |     @property
120 |
121 |     def has_balcony(self) -> Optional[bool]:
122 |
123 |         if self.balcony_count is not None or self.loggia_count is not None:
124 |
125 |             return (self.balcony_count or 0) + (self.loggia_count or 0) > 0
126 |
127 |         return None
128 |
129 |     @computed_field
130 |
131 |     @property
132 |
133 |     def has_elevator(self) -> Optional[bool]:
134 |
135 |         if self.elevator_passenger is not None or self.elevator_cargo is not None:
136 |
137 |             return (self.elevator_passenger or 0) + (self.elevator_cargo or 0) > 0
```

src\forecasting_service\parsers\cian\models.py (continued)

```
138 |
139 |         return None
140 |
```

src\forecasting_service\parsers\cian\page_parser.py

```
  1 | import re
  2 |
  3 | from typing import Optional
  4 |
  5 | from bs4 import BeautifulSoup, Tag
  6 |
  7 | from loguru import logger
  8 |
  9 | from forecasting_service.parsers.cian.models import CianFlat
 10 |
 11 | _REGION_MARKERS = ("край", "область", "республика", "округ", "автономн")
 12 |
 13 | _STREET_MARKERS = (
 14 |
 15 |     "улица", "проспект", "бульвар", "переулок", "шоссе",
 16 |
 17 |     "набережная", "проезд", "аллея", "тупик", "площадь", "дорога",
 18 |
 19 | )
 20 |
 21 | def parse_listing_page(html: str) -> list[CianFlat]:
 22 |
 23 |     soup = BeautifulSoup(html, "lxml")
 24 |
 25 |     cards = soup.select('[data-name="CardComponent"]')
 26 |
 27 |     if not cards:
 28 |
 29 |         logger.warning("Карточки CardComponent не найдены")
 30 |
 31 |         return []
 32 |
 33 |     flats = []
 34 |
 35 |     for card in cards:
 36 |
 37 |         try:
 38 |
 39 |             flat = _parse_card(card)
 40 |
 41 |             if flat and flat.url:
 42 |
 43 |                 flats.append(flat)
 44 |
 45 |         except Exception as e:
 46 |
 47 |             logger.debug(f"Ошибка парсинга карточки: {e}")
 48 |
 49 |     return flats
 50 |
 51 | def is_empty_listing(html: str) -> bool:
 52 |
 53 |     html_lower = html[:10000].lower()
```

src\forecasting_service\parsers\cian\page_parser.py (continued)

```
 54 |
 55 |     empty_indicators = [
 56 |
 57 |         "по вашему запросу ничего не найдено",
 58 |
 59 |         "нет объявлений",
 60 |
 61 |         "объявления не найдены",
 62 |
 63 |         "попробуйте изменить параметры",
 64 |
 65 |         "ничего не нашлось",
 66 |
 67 |     ]
 68 |
 69 |     return any(ind in html_lower for ind in empty_indicators)
 70 |
 71 | def _parse_card(card: Tag) -> Optional[CianFlat]:
 72 |
 73 |     url = _extract_url(card)
 74 |
 75 |     cian_id = _extract_cian_id(url)
 76 |
 77 |     title = _extract_title(card)
 78 |
 79 |     rooms, total_meters, floor, floors_count = _parse_title(title)
 80 |
 81 |     price = _extract_price(card)
 82 |
 83 |     geo = _extract_geo_labels(card)
 84 |
 85 |     residential_complex = _extract_jk(card)
 86 |
 87 |     underground = _extract_underground(card)
 88 |
 89 |     return CianFlat(
 90 |
 91 |         url=url,
 92 |
 93 |         cian_id=cian_id,
 94 |
 95 |         price=price,
 96 |
 97 |         total_meters=total_meters,
 98 |
 99 |         rooms_count=rooms,
100 |
101 |         floor=floor,
102 |
103 |         floors_count=floors_count,
104 |
105 |         district=geo.get("district", ""),
106 |
107 |         microdistrict=geo.get("microdistrict", ""),
108 |
109 |         street=geo.get("street", ""),
110 |
111 |         house_number=geo.get("house", ""),
112 |
113 |         underground=underground,
```

src\forecasting_service\parsers\cian\page_parser.py (continued)

```
114 |
115 |         residential_complex=residential_complex,
116 |
117 |         title_raw=title,
118 |
119 |         address_raw=geo.get("full_address", ""),
120 |
121 |     )
122 |
123 | def _extract_url(card: Tag) -> str:
124 |
125 |     link = (
126 |
127 |         card.select_one('[data-name="LinkArea"] a[href*="/flat/"]')
128 |
129 |         or card.select_one('a[href*="/flat/"]')
130 |
131 |         or card.select_one('a[href*="cian.ru"]')
132 |
133 |     )
134 |
135 |     if not link:
136 |
137 |         return ""
138 |
139 |     href = link.get("href", "")
140 |
141 |     return re.sub(r'\?mlSearchSessionGuid=[^&]*', '', href)
142 |
143 | def _extract_cian_id(url: str) -> Optional[int]:
144 |
145 |     match = re.search(r'/flat/(\d+)', url)
146 |
147 |     return int(match.group(1)) if match else None
148 |
149 | def _extract_title(card: Tag) -> str:
150 |
151 |     title_el = card.select_one('[data-mark="OfferTitle"]')
152 |
153 |     return title_el.get_text(strip=True) if title_el else ""
154 |
155 | def _extract_price(card: Tag) -> Optional[int]:
156 |
157 |     price_el = card.select_one('[data-mark="MainPrice"]')
158 |
159 |     if not price_el:
160 |
161 |         return None
162 |
163 |     digits = re.sub(r'[^\d]', '', price_el.get_text(strip=True))
164 |
165 |     return int(digits) if digits else None
166 |
167 | def _extract_jk(card: Tag) -> str:
168 |
169 |     jk_link = card.select_one('a[class*="jk"]')
170 |
171 |     if not jk_link:
172 |
173 |         return ""
```

src\forecasting_service\parsers\cian\page_parser.py (continued)

```
174 |
175 |     text = jk_link.get_text(strip=True)
176 |
177 |     text = re.sub(r'^ЖК\s*[«""]?\s*', '', text)
178 |
179 |     text = re.sub(r'[»""]$', '', text)
180 |
181 |     return text.strip()
182 |
183 | def _extract_underground(card: Tag) -> str:
184 |
185 |     el = card.select_one(
186 |
187 |         '[data-name*="nderground"], [class*="underground"]'
188 |
189 |     )
190 |
191 |     if not el:
192 |
193 |         return ""
194 |
195 |     text = el.get_text(strip=True)
196 |
197 |     return re.sub(r'\d+\s*мин.*$', '', text).strip()
198 |
199 | def _parse_title(
200 |
201 |     title: str,
202 |
203 | ) -> tuple[Optional[int], Optional[float], Optional[int], Optional[int]]:
204 |
205 |     rooms = None
206 |
207 |     total_meters = None
208 |
209 |     floor = None
210 |
211 |     floors_count = None
212 |
213 |     rooms_match = re.search(r'(\d+)-комн', title)
214 |
215 |     if rooms_match:
216 |
217 |         rooms = int(rooms_match.group(1))
218 |
219 |     elif "студия" in title.lower():
220 |
221 |         rooms = 0
222 |
223 |     meters_match = re.search(r'([\d]+[,.]?\d*)\s*м²', title)
224 |
225 |     if meters_match:
226 |
227 |         total_meters = float(meters_match.group(1).replace(',', '.'))
228 |
229 |     floor_match = re.search(r'(\d+)\s*/\s*(\d+)\s*эт', title)
230 |
231 |     if floor_match:
232 |
233 |         floor = int(floor_match.group(1))
```

src\forecasting_service\parsers\cian\page_parser.py (continued)

```
234 |
235 |         floors_count = int(floor_match.group(2))
236 |
237 |     return rooms, total_meters, floor, floors_count
238 |
239 | def _extract_geo_labels(card: Tag) -> dict:
240 |
241 |     geo_links = card.select('[data-name="GeoLabel"]')
242 |
243 |     result = {
244 |
245 |         "region": "",
246 |
247 |         "city": "",
248 |
249 |         "district": "",
250 |
251 |         "microdistrict": "",
252 |
253 |         "street": "",
254 |
255 |         "house": "",
256 |
257 |         "full_address": "",
258 |
259 |     }
260 |
261 |     if not geo_links:
262 |
263 |         return result
264 |
265 |     labels = [link.get_text(strip=True) for link in geo_links]
266 |
267 |     result["full_address"] = ", ".join(labels)
268 |
269 |     region_items = []
270 |
271 |     district_items = []
272 |
273 |     street_item = ""
274 |
275 |     house_item = ""
276 |
277 |     for link in geo_links:
278 |
279 |         text = link.get_text(strip=True)
280 |
281 |         href = link.get("href", "")
282 |
283 |         if "house%5B" in href or "house[" in href:
284 |
285 |             house_item = text
286 |
287 |         elif "street%5B" in href or "street[" in href:
288 |
289 |             street_item = text
290 |
291 |         elif "district%5B" in href or "district[" in href:
292 |
293 |             district_items.append(text)
```

src\forecasting_service\parsers\cian\page_parser.py (continued)

```
294 |
295 |         elif "region=" in href:
296 |
297 |             region_items.append(text)
298 |
299 |     if len(region_items) >= 2:
300 |
301 |         result["region"] = region_items[0]
302 |
303 |         result["city"] = region_items[1]
304 |
305 |     elif len(region_items) == 1:
306 |
307 |         text = region_items[0]
308 |
309 |         if _is_region_name(text):
310 |
311 |             result["region"] = text
312 |
313 |         else:
314 |
315 |             result["city"] = text
316 |
317 |     if len(district_items) >= 2:
318 |
319 |         result["district"] = district_items[0]
320 |
321 |         result["microdistrict"] = district_items[1]
322 |
323 |     elif len(district_items) == 1:
324 |
325 |         result["district"] = district_items[0]
326 |
327 |     result["street"] = street_item
328 |
329 |     result["house"] = house_item
330 |
331 |     if not result["district"] and len(labels) >= 3:
332 |
333 |         _parse_geo_by_position(labels, result)
334 |
335 |     return result
336 |
337 | def _is_region_name(text: str) -> bool:
338 |
339 |     lower = text.lower()
340 |
341 |     return any(marker in lower for marker in _REGION_MARKERS)
342 |
343 | def _is_street_name(text_lower: str) -> bool:
344 |
345 |     return any(marker in text_lower for marker in _STREET_MARKERS)
346 |
347 | def _looks_like_house_number(text: str) -> bool:
348 |
349 |     return bool(
350 |
351 |         re.match(r'^[\d]+[а-яА-Яa-zA-Z/к\s]*\d*$', text.strip())
352 |
353 |     )
```

src\forecasting_service\parsers\cian\page_parser.py (continued)

```
354 |
355 | def _parse_geo_by_position(labels: list[str], result: dict) -> None:
356 |
357 |     for i, label in enumerate(labels):
358 |
359 |         lower = label.lower()
360 |
361 |         if i == 0 and _is_region_name(label):
362 |
363 |             if not result["region"]:
364 |
365 |                 result["region"] = label
366 |
367 |             continue
368 |
369 |         if i == 1 and not result["city"] and result["region"]:
370 |
371 |             result["city"] = label
372 |
373 |             continue
374 |
375 |         if ("р-н " in lower or "район" in lower) and not result["district"]:
376 |
377 |             result["district"] = label
378 |
379 |             continue
380 |
381 |         if ("мкр." in lower or "микрорайон" in lower) and not result["microdistrict"]:
382 |
383 |             result["microdistrict"] = label
384 |
385 |             continue
386 |
387 |         if _is_street_name(lower) and not result["street"]:
388 |
389 |             result["street"] = label
390 |
391 |             continue
392 |
393 |         if (
394 |
395 |             i == len(labels) - 1
396 |
397 |             and not result["house"]
398 |
399 |             and _looks_like_house_number(label)
400 |
401 |         ):
402 |
403 |             result["house"] = label
404 |
```

src\forecasting_service\parsers\cian\parser.py

```python
import random

from typing import Optional

from loguru import logger

from forecasting_service.parsers.common.browser import (

    BrowserManager,

    CaptchaDetectedError,

)

from forecasting_service.parsers.common.rate_limiter import (

    AdaptiveRateLimiter,

)

from forecasting_service.parsers.cian.constants import (

    BASE_LISTING_URL,

    REGIONS,

    ROOM_PARAMS,

    LISTING_CARD_SELECTOR,

)

from forecasting_service.parsers.cian.page_parser import (

    parse_listing_page,

    is_empty_listing,

)

from forecasting_service.parsers.cian.models import CianFlat

from forecasting_service.data.storage import FlatStorage

from forecasting_service.utils.formatting import format_stats_block

class CianListingParser:

    def __init__(

        self,

        location: str = "Владивосток",

        headless: bool = False,

        page_delay: tuple[float, float] = (1.0, 3.0),

        max_retries: int = 5,
```

src\forecasting_service\parsers\cian\parser.py (continued)

```
 60 |
 61 |         storage: Optional[FlatStorage] = None,
 62 |
 63 |     ):
 64 |
 65 |         if location not in REGIONS:
 66 |
 67 |             raise ValueError(
 68 |
 69 |                 f"Неизвестный город: {location}. "
 70 |
 71 |                 f"Доступные: {list(REGIONS.keys())}"
 72 |
 73 |             )
 74 |
 75 |         self.location = location
 76 |
 77 |         self.region_id = REGIONS[location]
 78 |
 79 |         self.max_retries = max_retries
 80 |
 81 |         self.browser = BrowserManager(
 82 |
 83 |             headless=headless,
 84 |
 85 |             manual_captcha=True,
 86 |
 87 |             captcha_wait_timeout=300,
 88 |
 89 |         )
 90 |
 91 |         self.rate_limiter = AdaptiveRateLimiter(
 92 |
 93 |             base_page_delay=page_delay,
 94 |
 95 |         )
 96 |
 97 |         self.storage = storage or FlatStorage()
 98 |
 99 |     def _build_listing_url(self, rooms: int | str, page: int = 1) -> str:
100 |
101 |         room_param = ROOM_PARAMS.get(rooms, "")
102 |
103 |         params = [
104 |
105 |             "engine_version=2",
106 |
107 |             f"p={page}",
108 |
109 |             "with_neighbors=0",
110 |
111 |             f"region={self.region_id}",
112 |
113 |             "deal_type=sale",
114 |
115 |             "offer_type=flat",
116 |
117 |             room_param,
118 |
119 |             "only_flat=1",
```

src\forecasting_service\parsers\cian\parser.py (continued)

```
120 |
121 |         ]
122 |
123 |         return f"{BASE_LISTING_URL}?{'&'.join(p for p in params if p)}"
124 |
125 |     def collect_page(self, rooms: int | str, page: int) -> list[CianFlat]:
126 |
127 |         url = self._build_listing_url(rooms=rooms, page=page)
128 |
129 |         for attempt in range(1, self.max_retries + 1):
130 |
131 |             try:
132 |
133 |                 logger.info(
134 |
135 |                     f"  Стр. {page} | rooms={rooms} | "
136 |
137 |                     f"попытка {attempt}/{self.max_retries}"
138 |
139 |                 )
140 |
141 |                 html = self.browser.get_page(
142 |
143 |                     url,
144 |
145 |                     wait_selector=LISTING_CARD_SELECTOR,
146 |
147 |                     wait_timeout=15,
148 |
149 |                     scroll=True,
150 |
151 |                 )
152 |
153 |                 if is_empty_listing(html):
154 |
155 |                     logger.info(f"  Стр. {page} пуста")
156 |
157 |                     return []
158 |
159 |                 flats = parse_listing_page(html)
160 |
161 |                 if not flats:
162 |
163 |                     logger.warning(f"  Стр. {page}: 0 объявлений")
164 |
165 |                     if attempt < self.max_retries:
166 |
167 |                         self.rate_limiter.wait_between_pages()
168 |
169 |                         continue
170 |
171 |                     return []
172 |
173 |                 self.rate_limiter.record_success()
174 |
175 |                 logger.info(f"  Стр. {page}: {len(flats)} объявлений")
176 |
177 |                 return flats
178 |
179 |             except CaptchaDetectedError:
```

src\forecasting_service\parsers\cian\parser.py (continued)

```
180 |
181 |                 self.rate_limiter.record_captcha()
182 |
183 |                 if attempt < self.max_retries:
184 |
185 |                     self.rate_limiter.wait_on_captcha()
186 |
187 |                     self.browser.restart_with_new_ua()
188 |
189 |                 else:
190 |
191 |                     raise
192 |
193 |             except Exception as e:
194 |
195 |                 logger.error(f"  Стр. {page}: {e}")
196 |
197 |                 self.rate_limiter.record_error()
198 |
199 |                 if attempt < self.max_retries:
200 |
201 |                     self.rate_limiter.wait_between_pages()
202 |
203 |         return []
204 |
205 |     def collect(
206 |
207 |         self,
208 |
209 |         rooms: int | str | tuple = (1, 2, 3),
210 |
211 |         start_page: int = 1,
212 |
213 |         end_page: int = 54,
214 |
215 |     ) -> None:
216 |
217 |         if isinstance(rooms, (int, str)):
218 |
219 |             rooms = (rooms,)
220 |
221 |         try:
222 |
223 |             self.browser.start()
224 |
225 |             for room_idx, room_type in enumerate(rooms):
226 |
227 |                 logger.info(f"\n{'=' * 50}")
228 |
229 |                 logger.info(f"  Комнатность: {room_type}")
230 |
231 |                 logger.info(f"{'=' * 50}")
232 |
233 |                 consecutive_empty = 0
234 |
235 |                 for page in range(start_page, end_page + 1):
236 |
237 |                     if page > start_page or room_idx > 0:
238 |
239 |                         self.rate_limiter.wait_between_pages()
```

src\forecasting_service\parsers\cian\parser.py (continued)

```
240 |
241 |                     page_flats = self.collect_page(
242 |
243 |                         rooms=room_type, page=page
244 |
245 |                     )
246 |
247 |                     if not page_flats:
248 |
249 |                         consecutive_empty += 1
250 |
251 |                         if consecutive_empty >= 2:
252 |
253 |                             logger.info(
254 |
255 |                                 "  2 пустые подряд → следующая комнатность"
256 |
257 |                             )
258 |
259 |                             break
260 |
261 |                     else:
262 |
263 |                         consecutive_empty = 0
264 |
265 |                         flat_dicts = [f.model_dump() for f in page_flats]
266 |
267 |                         new, updated = self.storage.bulk_upsert_from_listing(
268 |
269 |                             flat_dicts
270 |
271 |                         )
272 |
273 |                         logger.info(
274 |
275 |                             f"  БД: +{new} новых, ~{updated} обновлений"
276 |
277 |                         )
278 |
279 |                     stats = self.storage.get_stats()
280 |
281 |                     logger.info(
282 |
283 |                         f"  В БД: {stats['total']} "
284 |
285 |                         f"(pending: {stats['pending']})"
286 |
287 |                     )
288 |
289 |                 if room_type != rooms[-1]:
290 |
291 |                     self.rate_limiter.wait_between_sections()
292 |
293 |         except CaptchaDetectedError:
294 |
295 |             logger.error("  Остановлено (CAPTCHA). Данные сохранены.")
296 |
297 |         finally:
298 |
299 |             self.browser.stop()
```

src\forecasting_service\parsers\cian\parser.py (continued)

```
300 |
301 |         logger.info(format_stats_block(
302 |
303 |             self.storage.get_stats(), title="ИТОГО В БД"
304 |
305 |         ))
306 |
```

src\forecasting_service\parsers\common__init__.py

```
  1 | from forecasting_service.parsers.common.browser import (
  2 |
  3 |     BrowserManager,
  4 |
  5 |     CaptchaDetectedError,
  6 |
  7 | )
  8 |
  9 | from forecasting_service.parsers.common.rate_limiter import (
 10 |
 11 |     AdaptiveRateLimiter,
 12 |
 13 | )
 14 |
 15 | from forecasting_service.parsers.common.warmup import (
 16 |
 17 |     perform_warmup,
 18 |
 19 |     perform_mini_warmup,
 20 |
 21 | )
 22 |
 23 | __all__ = [
 24 |
 25 |     "BrowserManager",
 26 |
 27 |     "CaptchaDetectedError",
 28 |
 29 |     "AdaptiveRateLimiter",
 30 |
 31 |     "perform_warmup",
 32 |
 33 |     "perform_mini_warmup",
 34 |
 35 | ]
 36 |
```

src\forecasting_service\parsers\common\browser.py

```
  1 | import time
  2 |
  3 | import random
  4 |
  5 | from typing import Optional
  6 |
  7 | from selenium import webdriver
  8 |
  9 | from selenium.webdriver.chrome.options import Options
```

src\forecasting_service\parsers\common\browser.py (continued)

```
 10 |
 11 | from selenium.webdriver.support.ui import WebDriverWait
 12 |
 13 | from selenium.webdriver.support import expected_conditions as EC
 14 |
 15 | from selenium.webdriver.common.by import By
 16 |
 17 | from selenium.webdriver.common.action_chains import ActionChains
 18 |
 19 | from selenium.common.exceptions import TimeoutException, WebDriverException
 20 |
 21 | from loguru import logger
 22 |
 23 | try:
 24 |
 25 |     from selenium_stealth import stealth
 26 |
 27 |     HAS_STEALTH = True
 28 |
 29 | except ImportError:
 30 |
 31 |     HAS_STEALTH = False
 32 |
 33 | USER_AGENTS = [
 34 |
 35 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0 ...
 36 |
 37 |     "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1 ...
 38 |
 39 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0",
 40 |
 41 |     "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Versio ...
 42 |
 43 |     "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/ ...
 44 |
 45 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0 ...
 46 |
 47 |     "Mozilla/5.0 (X11; Linux x86_64; rv:133.0) Gecko/20100101 Firefox/133.0",
 48 |
 49 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0 ...
 50 |
 51 |     "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:133.0) Gecko/20100101 Firefox/133.0",
 52 |
 53 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0 ...
 54 |
 55 |     "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:133.0) Gecko/20100101 Firefox/133.0",
 56 |
 57 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0 ...
 58 |
 59 |     "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/1 ...
 60 |
 61 |     "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:132.0) Gecko/20100101 Firefox/132.0",
 62 |
 63 | ]
 64 |
 65 | _BLOCK_INDICATORS = (
 66 |
 67 |     "я не робот",
 68 |
 69 |     "i'm not a robot",
```

src\forecasting_service\parsers\common\browser.py (continued)

```
 70 |
 71 |     "проверка безопасности",
 72 |
 73 |     "access denied",
 74 |
 75 |     "заблокирован",
 76 |
 77 |     "подозрительная активность",
 78 |
 79 |     "security check",
 80 |
 81 |     "доступ ограничен",
 82 |
 83 |     "доступ запрещён",
 84 |
 85 |     "доступ запрещен",
 86 |
 87 |     "blocked",
 88 |
 89 |     "forbidden",
 90 |
 91 | )
 92 |
 93 | _MIN_PAGE_SIZE = 5000
 94 |
 95 | class CaptchaDetectedError(Exception):
 96 |
 97 |     def __init__(self, url: str):
 98 |
 99 |         self.url = url
100 |
101 |         super().__init__(f"CAPTCHA detected: {url}")
102 |
103 | class BrowserManager:
104 |
105 |     def __init__(
106 |
107 |         self,
108 |
109 |         headless: bool = True,
110 |
111 |         window_size: tuple[int, int] = (1920, 1080),
112 |
113 |         page_load_timeout: int = 60,
114 |
115 |         implicit_wait: int = 10,
116 |
117 |         rotate_ua_every: int = 10,
118 |
119 |         manual_captcha: bool = True,
120 |
121 |         captcha_wait_timeout: int = 300,
122 |
123 |         user_data_dir: Optional[str] = None,
124 |
125 |     ):
126 |
127 |         self.headless = headless
128 |
129 |         self.window_size = window_size
```

src\forecasting_service\parsers\common\browser.py (continued)

```
130 |
131 |         self.page_load_timeout = page_load_timeout
132 |
133 |         self.implicit_wait = implicit_wait
134 |
135 |         self.rotate_ua_every = rotate_ua_every
136 |
137 |         self.manual_captcha = manual_captcha
138 |
139 |         self.captcha_wait_timeout = captcha_wait_timeout
140 |
141 |         self.user_data_dir = user_data_dir
142 |
143 |         self._driver = None
144 |
145 |         self._current_ua: str = random.choice(USER_AGENTS)
146 |
147 |         self._page_count = 0
148 |
149 |     def start(self):
150 |
151 |         if self._driver:
152 |
153 |             logger.debug("Браузер уже запущен")
154 |
155 |             return self._driver
156 |
157 |         logger.info(f"  Запуск Chrome (headless={self.headless})")
158 |
159 |         self._driver = self._create_driver()
160 |
161 |         try:
162 |
163 |             self._driver.set_page_load_timeout(self.page_load_timeout)
164 |
165 |             self._driver.implicitly_wait(self.implicit_wait)
166 |
167 |         except Exception as e:
168 |
169 |             self.stop()
170 |
171 |             raise e
172 |
173 |         logger.info("  Chrome запущен")
174 |
175 |         return self._driver
176 |
177 |     def stop(self) -> None:
178 |
179 |         if self._driver:
180 |
181 |             try:
182 |
183 |                 self._driver.quit()
184 |
185 |                 logger.info("Chrome остановлен")
186 |
187 |             except Exception as e:
188 |
189 |                 logger.warning(f"Ошибка при остановке: {e}")
```

src\forecasting_service\parsers\common\browser.py (continued)

```
190 |
191 |             finally:
192 |
193 |                 self._driver = None
194 |
195 |     def restart_with_new_ua(self) -> None:
196 |
197 |         old_ua = self._current_ua
198 |
199 |         available = [ua for ua in USER_AGENTS if ua != old_ua]
200 |
201 |         self._current_ua = random.choice(available)
202 |
203 |         logger.info(f"  Ротация UA: ...{self._current_ua[-30:]}")
204 |
205 |         self.stop()
206 |
207 |         time.sleep(random.uniform(2, 5))
208 |
209 |         self.start()
210 |
211 |     @property
212 |
213 |     def driver(self):
214 |
215 |         if not self._driver:
216 |
217 |             return self.start()
218 |
219 |         return self._driver
220 |
221 |     def __enter__(self):
222 |
223 |         self.start()
224 |
225 |         return self
226 |
227 |     def __exit__(self, exc_type, exc_val, exc_tb):
228 |
229 |         self.stop()
230 |
231 |         return False
232 |
233 |     def __del__(self):
234 |
235 |         try:
236 |
237 |             self.stop()
238 |
239 |         except Exception:
240 |
241 |             pass
242 |
243 |     def get_page(
244 |
245 |         self,
246 |
247 |         url: str,
248 |
249 |         wait_selector: Optional[str] = None,
```

src\forecasting_service\parsers\common\browser.py (continued)

```
250 |
251 |         wait_timeout: int = 15,
252 |
253 |         scroll: bool = True,
254 |
255 |     ) -> str:
256 |
257 |         self._page_count += 1
258 |
259 |         if (
260 |
261 |             self._page_count > 1
262 |
263 |             and self._page_count % self.rotate_ua_every == 0
264 |
265 |         ):
266 |
267 |             self.restart_with_new_ua()
268 |
269 |         driver = self.driver
270 |
271 |         logger.debug(f"  Загрузка: {url[:80]}...")
272 |
273 |         try:
274 |
275 |             driver.get(url)
276 |
277 |         except TimeoutException:
278 |
279 |             logger.warning("  Таймаут загрузки, пробуем продолжить...")
280 |
281 |         if wait_selector:
282 |
283 |             self._wait_for_selector(driver, wait_selector, wait_timeout)
284 |
285 |         if scroll:
286 |
287 |             self._simulate_reading(driver)
288 |
289 |             if random.random() < 0.3:
290 |
291 |                 self._simulate_mouse_movement(driver)
292 |
293 |         time.sleep(random.uniform(1.0, 2.5))
294 |
295 |         html = driver.page_source
296 |
297 |         if self._is_captcha(html):
298 |
299 |             logger.warning("  CAPTCHA обнаружена!")
300 |
301 |             if self.manual_captcha and not self.headless:
302 |
303 |                 html = self._wait_for_manual_captcha(driver, url)
304 |
305 |             else:
306 |
307 |                 raise CaptchaDetectedError(url)
308 |
309 |         return html
```

src\forecasting_service\parsers\common\browser.py (continued)

```
310 |
311 |     @staticmethod
312 |
313 |     def _is_captcha(html: str) -> bool:
314 |
315 |         html_lower = html[:15000].lower()
316 |
317 |         has_block = any(ind in html_lower for ind in _BLOCK_INDICATORS)
318 |
319 |         if has_block:
320 |
321 |             return True
322 |
323 |         if len(html) < _MIN_PAGE_SIZE and "cian" in html_lower:
324 |
325 |             logger.debug(
326 |
327 |                 f"Подозрительно маленький HTML: {len(html)} символов"
328 |
329 |             )
330 |
331 |             return True
332 |
333 |         return False
334 |
335 |     def _wait_for_manual_captcha(self, driver, url: str) -> str:
336 |
337 |         logger.warning("=" * 60)
338 |
339 |         logger.warning("  CAPTCHA! Пройдите её в окне браузера")
340 |
341 |         logger.warning(f"  URL: {url[:60]}...")
342 |
343 |         logger.warning(f"  Ожидание: {self.captcha_wait_timeout} сек")
344 |
345 |         logger.warning("=" * 60)
346 |
347 |         start_time = time.time()
348 |
349 |         check_interval = 5
350 |
351 |         while time.time() - start_time < self.captcha_wait_timeout:
352 |
353 |             time.sleep(check_interval)
354 |
355 |             try:
356 |
357 |                 html = driver.page_source
358 |
359 |                 if not self._is_captcha(html):
360 |
361 |                     logger.info("  CAPTCHA пройдена! Продолжаем...")
362 |
363 |                     time.sleep(random.uniform(3, 6))
364 |
365 |                     return html
366 |
367 |             except Exception:
368 |
369 |                 pass
```

src\forecasting_service\parsers\common\browser.py (continued)

```
370 |
371 |             elapsed = int(time.time() - start_time)
372 |
373 |             remaining = self.captcha_wait_timeout - elapsed
374 |
375 |             if elapsed % 15 < check_interval:
376 |
377 |                 logger.info(f"  Жду... ({remaining} сек осталось)")
378 |
379 |         logger.error(
380 |
381 |             f"  CAPTCHA не пройдена за {self.captcha_wait_timeout} сек"
382 |
383 |         )
384 |
385 |         raise CaptchaDetectedError(url)
386 |
387 |     def _create_driver(self):
388 |
389 |         options = Options()
390 |
391 |         if self.headless:
392 |
393 |             options.add_argument("--headless=new")
394 |
395 |         w, h = self.window_size
396 |
397 |         options.add_argument(f"--window-size={w},{h}")
398 |
399 |         options.add_argument(
400 |
401 |             "--disable-blink-features=AutomationControlled"
402 |
403 |         )
404 |
405 |         options.add_argument("--disable-infobars")
406 |
407 |         options.add_argument("--disable-dev-shm-usage")
408 |
409 |         options.add_argument("--no-sandbox")
410 |
411 |         options.add_argument("--disable-gpu")
412 |
413 |         options.add_argument("--lang=ru-RU")
414 |
415 |         options.add_argument(f"user-agent={self._current_ua}")
416 |
417 |         if self.user_data_dir:
418 |
419 |             options.add_argument(f"--user-data-dir={self.user_data_dir}")
420 |
421 |         options.add_experimental_option(
422 |
423 |             "excludeSwitches", ["enable-automation"]
424 |
425 |         )
426 |
427 |         options.add_experimental_option("useAutomationExtension", False)
428 |
429 |         driver = webdriver.Chrome(options=options)
```

src\forecasting_service\parsers\common\browser.py (continued)

```
        self._apply_stealth(driver)

        return driver

    @staticmethod

    def _apply_stealth(driver) -> None:

        if HAS_STEALTH:

            stealth(

                driver,

                languages=["ru-RU", "ru", "en-US", "en"],

                vendor="Google Inc.",

                platform="Win32",

                webgl_vendor="Intel Inc.",

                renderer="Intel Iris OpenGL Engine",

                fix_hairline=True,

            )

        else:

            driver.execute_cdp_cmd(

                "Page.addScriptToEvaluateOnNewDocument",

                {

                    "source":

                },

            )

    @staticmethod

    def _wait_for_selector(

        driver, selector: str, timeout: int

    ) -> None:

        try:

            WebDriverWait(driver, timeout).until(

                EC.presence_of_element_located(

                    (By.CSS_SELECTOR, selector)

                )
```

src\forecasting_service\parsers\common\browser.py (continued)

```
490 |
491 |             )
492 |
493 |         except TimeoutException:
494 |
495 |             logger.warning(f"  Элемент '{selector}' не найден")
496 |
497 |     @staticmethod
498 |
499 |     def _simulate_reading(driver) -> None:
500 |
501 |         try:
502 |
503 |             viewport_h = driver.execute_script(
504 |
505 |                 "return window.innerHeight"
506 |
507 |             )
508 |
509 |             page_h = driver.execute_script(
510 |
511 |                 "return document.body.scrollHeight"
512 |
513 |             )
514 |
515 |             if page_h <= viewport_h:
516 |
517 |                 time.sleep(random.uniform(1.0, 2.0))
518 |
519 |                 return
520 |
521 |             num_scrolls = random.randint(2, 4)
522 |
523 |             step = page_h / (num_scrolls + 1)
524 |
525 |             pos = 0
526 |
527 |             for _ in range(num_scrolls):
528 |
529 |                 amount = step * random.uniform(0.7, 1.3)
530 |
531 |                 pos = min(pos + amount, page_h)
532 |
533 |                 driver.execute_script(
534 |
535 |                     f"window.scrollTo({ top: {int(pos)}, "
536 |
537 |                     f"behavior: 'smooth'} );"
538 |
539 |                 )
540 |
541 |                 time.sleep(random.uniform(1.0, 3.0))
542 |
543 |                 if random.random() < 0.3:
544 |
545 |                     back = pos * random.uniform(0.2, 0.5)
546 |
547 |                     driver.execute_script(
548 |
549 |                         f"window.scrollTo({ top: {int(back)}, "
```

src\forecasting_service\parsers\common\browser.py (continued)

```
                        f"behavior: 'smooth'} );"

                    )

                    time.sleep(random.uniform(0.5, 1.5))

            driver.execute_script(

                "window.scrollTo({top: 0, behavior: 'smooth'});"

            )

            time.sleep(random.uniform(0.5, 1.0))

        except Exception as e:

            logger.debug(f"Ошибка скролла: {e}")

    @staticmethod

    def _simulate_mouse_movement(driver) -> None:

        try:

            actions = ActionChains(driver)

            body = driver.find_element(By.TAG_NAME, "body")

            for _ in range(random.randint(2, 3)):

                x = random.randint(-200, 200)

                y = random.randint(-100, 100)

                try:

                    actions.move_to_element_with_offset(

                        body, x, y

                    ).perform()

                except Exception:

                    pass

                time.sleep(random.uniform(0.3, 0.8))

        except Exception as e:

            logger.debug(f"Ошибка мыши: {e}")

```

src\forecasting_service\parsers\common\rate_limiter.py

```python
import time

import random

from collections import deque

from loguru import logger

class AdaptiveRateLimiter:

    def __init__(

        self,

        base_page_delay: tuple[float, float] = (5.0, 15.0),

        base_detail_delay: tuple[float, float] = (30.0, 60.0),

        action_delay: tuple[float, float] = (1.0, 3.0),

        long_pause_range: tuple[float, float] = (60.0, 120.0),

        section_pause_range: tuple[float, float] = (20.0, 40.0),

        window_size: int = 15,

        captcha_threshold: int = 2,

        max_multiplier: float = 5.0,

    ):

        self._base_page_delay = base_page_delay

        self._base_detail_delay = base_detail_delay

        self._action_delay = action_delay

        self._long_pause_range = long_pause_range

        self._section_pause_range = section_pause_range

        self._max_multiplier = max_multiplier

        self._multiplier: float = 1.0

        self._request_count: int = 0

        self._captcha_count: int = 0

        self._window_size = window_size

        self._captcha_threshold = captcha_threshold

        self._results: deque[bool] = deque(maxlen=window_size)

        self._next_long_pause_at: int = random.randint(8, 15)

    def record_success(self) -> None:
```

src\forecasting_service\parsers\common\rate_limiter.py (continued)

```
 60 |
 61 |         self._results.append(True)
 62 |
 63 |         self._try_decrease_multiplier()
 64 |
 65 |     def record_captcha(self) -> None:
 66 |
 67 |         self._results.append(False)
 68 |
 69 |         self._captcha_count += 1
 70 |
 71 |         self._try_increase_multiplier()
 72 |
 73 |     def record_error(self) -> None:
 74 |
 75 |         self._results.append(False)
 76 |
 77 |     def wait_between_pages(self) -> None:
 78 |
 79 |         self._request_count += 1
 80 |
 81 |         if self._should_long_pause():
 82 |
 83 |             self._long_pause()
 84 |
 85 |             self._schedule_next_long_pause(min_interval=8, max_interval=15)
 86 |
 87 |             return
 88 |
 89 |         delay = self._apply_multiplier(self._base_page_delay)
 90 |
 91 |         logger.debug(
 92 |
 93 |             f"  Пауза стр.: {delay:.1f}с (×{self._multiplier:.1f})"
 94 |
 95 |         )
 96 |
 97 |         time.sleep(delay)
 98 |
 99 |     def wait_between_details(self) -> None:
100 |
101 |         self._request_count += 1
102 |
103 |         if self._should_long_pause():
104 |
105 |             self._long_pause()
106 |
107 |             self._schedule_next_long_pause(min_interval=5, max_interval=10)
108 |
109 |             return
110 |
111 |         delay = self._apply_multiplier(self._base_detail_delay)
112 |
113 |         logger.debug(
114 |
115 |             f"  Пауза деталь: {delay:.1f}с (×{self._multiplier:.1f})"
116 |
117 |         )
118 |
119 |         time.sleep(delay)
```

src\forecasting_service\parsers\common\rate_limiter.py (continued)

```
120 |
121 |     def wait_between_actions(self) -> None:
122 |
123 |         delay = random.uniform(*self._action_delay)
124 |
125 |         time.sleep(delay)
126 |
127 |     def wait_on_captcha(self) -> None:
128 |
129 |         delay = random.uniform(120, 300) * self._multiplier
130 |
131 |         delay = min(delay, 600)
132 |
133 |         logger.warning(f"  Пауза CAPTCHA: {delay:.0f}с")
134 |
135 |         time.sleep(delay)
136 |
137 |     def wait_between_sections(self) -> None:
138 |
139 |         delay = self._apply_multiplier(self._section_pause_range)
140 |
141 |         logger.info(f"  Пауза раздел: {delay:.1f}с")
142 |
143 |         time.sleep(delay)
144 |
145 |     @property
146 |
147 |     def total_requests(self) -> int:
148 |
149 |         return self._request_count
150 |
151 |     @property
152 |
153 |     def multiplier(self) -> float:
154 |
155 |         return self._multiplier
156 |
157 |     @property
158 |
159 |     def total_captchas(self) -> int:
160 |
161 |         return self._captcha_count
162 |
163 |     def _apply_multiplier(
164 |
165 |         self, delay_range: tuple[float, float]
166 |
167 |     ) -> float:
168 |
169 |         base = random.uniform(*delay_range)
170 |
171 |         return base * self._multiplier
172 |
173 |     def _try_decrease_multiplier(self) -> None:
174 |
175 |         if len(self._results) < self._window_size:
176 |
177 |             return
178 |
179 |         if self._multiplier <= 1.0:
```

src\forecasting_service\parsers\common\rate_limiter.py (continued)

```
180 |
181 |             return
182 |
183 |         recent_failures = sum(1 for r in self._results if not r)
184 |
185 |         if recent_failures == 0:
186 |
187 |             old = self._multiplier
188 |
189 |             self._multiplier = max(1.0, self._multiplier * 0.8)
190 |
191 |             logger.debug(
192 |
193 |                 f"  Множитель: {old:.1f}x → {self._multiplier:.1f}x"
194 |
195 |             )
196 |
197 |     def _try_increase_multiplier(self) -> None:
198 |
199 |         recent_captchas = sum(1 for r in self._results if not r)
200 |
201 |         if recent_captchas >= self._captcha_threshold:
202 |
203 |             old = self._multiplier
204 |
205 |             self._multiplier = min(
206 |
207 |                 self._multiplier * 2.0,
208 |
209 |                 self._max_multiplier,
210 |
211 |             )
212 |
213 |             logger.warning(
214 |
215 |                 f"  Множитель: {old:.1f}x → {self._multiplier:.1f}x "
216 |
217 |                 f"({recent_captchas} CAPTCHA в окне)"
218 |
219 |             )
220 |
221 |     def _should_long_pause(self) -> bool:
222 |
223 |         return self._request_count >= self._next_long_pause_at
224 |
225 |     def _long_pause(self) -> None:
226 |
227 |         delay = self._apply_multiplier(self._long_pause_range)
228 |
229 |         delay = min(delay, 300)
230 |
231 |         logger.info(
232 |
233 |             f"  Длинная пауза: {delay:.0f}c "
234 |
235 |             f"({self._request_count} запросов)"
236 |
237 |         )
238 |
239 |         time.sleep(delay)
```

src\forecasting_service\parsers\common\rate_limiter.py (continued)

```
240 |
241 |     def _schedule_next_long_pause(
242 |
243 |         self, min_interval: int, max_interval: int
244 |
245 |     ) -> None:
246 |
247 |         self._next_long_pause_at = (
248 |
249 |             self._request_count + random.randint(min_interval, max_interval)
250 |
251 |         )
252 |
```

src\forecasting_service\parsers\common\warmup.py

```
  1 | import time
  2 |
  3 | import random
  4 |
  5 | from typing import TYPE_CHECKING
  6 |
  7 | from loguru import logger
  8 |
  9 | from forecasting_service.parsers.cian.constants import REGIONS, SUBDOMAINS
 10 |
 11 | if TYPE_CHECKING:
 12 |
 13 |     from forecasting_service.parsers.common.browser import BrowserManager
 14 |
 15 | def _build_warmup_urls(location: str) -> dict[str, str]:
 16 |
 17 |     region_id = REGIONS.get(location)
 18 |
 19 |     if not region_id:
 20 |
 21 |         raise ValueError(f"Неизвестный город: {location}")
 22 |
 23 |     subdomain = SUBDOMAINS.get(location, "www")
 24 |
 25 |     return {
 26 |
 27 |         "main": f"https://{subdomain}.cian.ru/",
 28 |
 29 |         "listing": (
 30 |
 31 |             f"https://{subdomain}.cian.ru/cat.php?"
 32 |
 33 |             f"deal_type=sale&engine_version=2"
 34 |
 35 |             f"&offer_type=flat&region={region_id}"
 36 |
 37 |         ),
 38 |
 39 |     }
 40 |
 41 | def perform_warmup(
 42 |
 43 |     browser: "BrowserManager",
```

src\forecasting_service\parsers\common\warmup.py (continued)

```
44 |
45 |     location: str = "Владивосток",
46 |
47 |     scroll: bool = True,
48 |
49 | ) -> None:
50 |
51 |     from forecasting_service.parsers.common.browser import CaptchaDetectedError
52 |
53 |     urls = _build_warmup_urls(location)
54 |
55 |     logger.info("  Warm-up: заходим на ЦИАН...")
56 |
57 |     try:
58 |
59 |         browser.get_page(urls["main"], scroll=scroll)
60 |
61 |         time.sleep(random.uniform(3, 7))
62 |
63 |         browser.get_page(urls["listing"], scroll=scroll)
64 |
65 |         time.sleep(random.uniform(5, 10))
66 |
67 |         logger.info("  Warm-up завершён")
68 |
69 |     except CaptchaDetectedError:
70 |
71 |         logger.warning("CAPTCHA на warm-up, продолжаем...")
72 |
73 |     except Exception as e:
74 |
75 |         logger.warning(f"Ошибка warm-up: {e}")
76 |
77 | def perform_mini_warmup(
78 |
79 |     browser: "BrowserManager",
80 |
81 |     location: str = "Владивосток",
82 |
83 | ) -> None:
84 |
85 |     from forecasting_service.parsers.common.browser import CaptchaDetectedError
86 |
87 |     urls = _build_warmup_urls(location)
88 |
89 |     try:
90 |
91 |         browser.get_page(urls["main"], scroll=False)
92 |
93 |         time.sleep(random.uniform(3, 7))
94 |
95 |     except (CaptchaDetectedError, Exception):
96 |
97 |         pass
98 |
```

src\forecasting_service\parsers\domclick__init__.py

```
from forecasting_service.parsers.domclick.parser import DomclickListingParser

from forecasting_service.parsers.domclick.detail_parser import parse_detail_page

from forecasting_service.parsers.domclick.http_client import DomclickClient

__all__ = ["DomclickListingParser", "parse_detail_page", "DomclickClient"]

```

src\forecasting_service\parsers\domclick\constants.py

```
BASE_URL = "https://vladivostok.domclick.ru"

SEARCH_URL = f"{BASE_URL}/search"

CARD_URL_PREFIX = f"{BASE_URL}/card"

VLADIVOSTOK_AID = "57471"

ROOM_PARAMS = {

    "studio": "st",

    0: "st",

    1: "1",

    2: "2",

    3: "3",

    4: "4",

    5: "5%2B",

    6: "5%2B",

}

CARDS_PER_PAGE = 20

DEFAULT_HEADERS = {

    "User-Agent": (

        "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:147.0) "

        "Gecko/20100101 Firefox/147.0"

    ),

    "Accept": (

        "text/html,application/xhtml+xml,"

        "application/xml;q=0.9,*/*;q=0.8"

    ),
```

src\forecasting_service\parsers\domclick\constants.py (continued)

```
    |
    |     "Accept-Language": "ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7",
    |
    |     "Accept-Encoding": "gzip, deflate, br",
    |
    |     "Sec-Fetch-Dest": "document",
    |
    |     "Sec-Fetch-Mode": "navigate",
    |
    |     "Sec-Fetch-Site": "none",
    |
    |     "Sec-Fetch-User": "?1",
    |
    |     "Connection": "keep-alive",
    |
    | }
    |
```

src\forecasting_service\parsers\domclick\detail_parser.py

```
  1 | import re
  2 |
  3 | from typing import Optional
  4 |
  5 | from bs4 import BeautifulSoup
  6 |
  7 | def parse_detail_page(html: str) -> dict:
  8 |
  9 |     soup = BeautifulSoup(html, "lxml")
 10 |
 11 |     details = {}
 12 |
 13 |     _parse_main_items(soup, details)
 14 |
 15 |     _parse_building_items(soup, details)
 16 |
 17 |     _parse_extra_flags(soup, details)
 18 |
 19 |     _parse_novostroyka(soup, details)
 20 |
 21 |     _parse_jk(soup, details)
 22 |
 23 |     _parse_address(soup, details)
 24 |
 25 |     return details
 26 |
 27 | _MAIN_MAPPING = {
 28 |
 29 |     "Комнат":              ("rooms_count", "int"),
 30 |
 31 |     "Площадь":             ("total_meters", "area"),
 32 |
 33 |     "Жилая":               ("living_meters", "area"),
 34 |
 35 |     "Кухня":               ("kitchen_meters", "area"),
 36 |
 37 |     "Этаж":                ("floor", "int"),
 38 |
 39 |     "Ремонт":              ("finish_type", "text"),
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
    |
    |     "Тип жилья":           ("object_type", "text"),
    |
    |     "Вид из окон":         ("window_view", "text"),
    |
    |     "Количество балконов": ("balcony_count", "int"),
    |
    |     "Высота потолков":     ("ceiling_height", "float"),
    |
    |     "Санузел":             ("bathroom_type", "text"),
    |
    |     "Количество лифтов":   ("elevator_passenger", "int"),
    |
    |     "Грузовой лифт":       ("elevator_cargo", "bool"),
    |
    |     "Газ":                 ("has_gas", "bool"),
    |
    |     "Мусоропровод":        ("has_garbage_chute", "bool"),
    |
    |     "Перепланировка":      ("has_redevelopment", "bool"),
    |
    | }
    |
    | def _parse_main_items(soup: BeautifulSoup, details: dict) -> None:
    |
    |     items = soup.select("li.C_L_4[data-e2e-id]")
    |
    |     for item in items:
    |
    |         key = item.get("data-e2e-id", "").strip()
    |
    |         if not key:
    |
    |             continue
    |
    |         val_el = item.select_one('span[data-e2e-id="Значение"]')
    |
    |         if not val_el:
    |
    |             val_el = item.select_one("span.yNtG9")
    |
    |         if not val_el:
    |
    |             continue
    |
    |         val = val_el.get_text(strip=True)
    |
    |         if key in _MAIN_MAPPING:
    |
    |             field, parse_type = _MAIN_MAPPING[key]
    |
    |             parsed = _parse_value(val, parse_type)
    |
    |             if parsed is not None:
    |
    |                 details.setdefault(field, parsed)
    |
    | _BUILDING_MAPPING = {
    |
    |     "Год постройки":       ("year_of_construction", "year"),
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
100 |
101 |     "Материал стен":       ("house_material_type", "text"),
102 |
103 |     "Количество этажей":   ("floors_count", "int"),
104 |
105 |     "Тип перекрытий":      ("floor_type", "text"),
106 |
107 |     "Высота потолков":     ("ceiling_height", "float"),
108 |
109 |     "Мусоропровод":        ("has_garbage_chute", "bool"),
110 |
111 |     "Газ":                 ("has_gas", "bool"),
112 |
113 |     "Лифт":                ("elevator_passenger", "bool_to_int"),
114 |
115 | }
116 |
117 | def _parse_building_items(soup: BeautifulSoup, details: dict) -> None:
118 |
119 |     section = soup.select_one(
120 |
121 |         'section[data-e2e-id="building-info-block"]'
122 |
123 |     )
124 |
125 |     if not section:
126 |
127 |         return
128 |
129 |     items = section.select("li.ByFq7[data-e2e-id]")
130 |
131 |     for item in items:
132 |
133 |         key = item.get("data-e2e-id", "").strip()
134 |
135 |         if not key:
136 |
137 |             continue
138 |
139 |         val_el = (
140 |
141 |             item.select_one('span[data-e2e-id="Значение"]')
142 |
143 |             or item.select_one("span.upbHP")
144 |
145 |         )
146 |
147 |         if not val_el:
148 |
149 |             continue
150 |
151 |         val = val_el.get_text(strip=True)
152 |
153 |         if key in _BUILDING_MAPPING:
154 |
155 |             field, parse_type = _BUILDING_MAPPING[key]
156 |
157 |             parsed = _parse_value(val, parse_type)
158 |
159 |             if parsed is not None:
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
160 |
161 |                 details.setdefault(field, parsed)
162 |
163 | _FLAG_MAPPING = {
164 |
165 |     "Консьерж":            "has_concierge",
166 |
167 |     "Домофон":             "has_intercom",
168 |
169 |     "Закрытая территория": "has_closed_territory",
170 |
171 |     "Кодовая дверь":       "has_code_door",
172 |
173 |     "Во дворе":            "parking_yard",
174 |
175 |     "Подземная":           "parking_underground",
176 |
177 |     "Охраняемая":          "parking_guarded",
178 |
179 |     "Со шлагбаумом":       "parking_barrier",
180 |
181 |     "Есть гараж":          "has_garage",
182 |
183 | }
184 |
185 | def _parse_extra_flags(soup: BeautifulSoup, details: dict) -> None:
186 |
187 |     items = soup.select("li.IY5_T[data-e2e-id]")
188 |
189 |     for item in items:
190 |
191 |         key = item.get("data-e2e-id", "").strip()
192 |
193 |         if key in _FLAG_MAPPING:
194 |
195 |             details[_FLAG_MAPPING[key]] = True
196 |
197 |     parking_parts = []
198 |
199 |     for flag, label in [
200 |
201 |         ("parking_yard", "во дворе"),
202 |
203 |         ("parking_underground", "подземная"),
204 |
205 |         ("parking_guarded", "охраняемая"),
206 |
207 |         ("parking_barrier", "со шлагбаумом"),
208 |
209 |     ]:
210 |
211 |         if details.pop(flag, False):
212 |
213 |             parking_parts.append(label)
214 |
215 |     if parking_parts:
216 |
217 |         details.setdefault("parking_type", ", ".join(parking_parts))
218 |
219 |     if details.pop("has_intercom", None):
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
220 |
221 |         pass
222 |
223 |     if details.pop("has_code_door", None):
224 |
225 |         pass
226 |
227 |     if details.pop("has_closed_territory", None):
228 |
229 |         pass
230 |
231 | def _parse_novostroyka(soup: BeautifulSoup, details: dict) -> None:
232 |
233 |     section = soup.select_one('div[data-id="aboutOffer"]')
234 |
235 |     if not section:
236 |
237 |         return
238 |
239 |     for item in section.select("div.ri4j3"):
240 |
241 |         key_el = item.select_one("div.K_m3c")
242 |
243 |         val_el = item.select_one("div.BCmK9")
244 |
245 |         if not key_el or not val_el:
246 |
247 |             continue
248 |
249 |         key = key_el.get_text(strip=True).lower()
250 |
251 |         val = val_el.get_text(strip=True)
252 |
253 |         if "жилая" in key:
254 |
255 |             details.setdefault("living_meters", _parse_float(val))
256 |
257 |         elif "кухня" in key:
258 |
259 |             details.setdefault("kitchen_meters", _parse_float(val))
260 |
261 |         elif "отделка" in key:
262 |
263 |             details.setdefault("finish_type", val.lower())
264 |
265 |         elif "класс жилья" in key:
266 |
267 |             details.setdefault("jk_class", val.lower())
268 |
269 |         elif "санузел" in key:
270 |
271 |             details.setdefault("bathroom_type", val.lower())
272 |
273 |         elif "окна" in key or "вид" in key:
274 |
275 |             details.setdefault("window_view", val.lower())
276 |
277 |     section2 = soup.select_one('div[data-id="aboutComplex"]')
278 |
279 |     if not section2:
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
280 |
281 |         return
282 |
283 |     for item in section2.select("div.nJDVt"):
284 |
285 |         key_el = item.select_one("div.Qgi72")
286 |
287 |         val_el = item.select_one("div.wib4o")
288 |
289 |         if not key_el or not val_el:
290 |
291 |             continue
292 |
293 |         key = key_el.get_text(strip=True).lower()
294 |
295 |         val = val_el.get_text(strip=True)
296 |
297 |         if "материал" in key:
298 |
299 |             details.setdefault("house_material_type", val.lower())
300 |
301 |         elif "этажей" in key:
302 |
303 |             details.setdefault("floors_count", _parse_int(val))
304 |
305 | def _parse_jk(soup: BeautifulSoup, details: dict) -> None:
306 |
307 |     jk_el = soup.select_one(
308 |
309 |         'div[data-id="aboutComplex"] h2.tebMt span.UD6h3'
310 |
311 |     )
312 |
313 |     if jk_el:
314 |
315 |         name = jk_el.get_text(strip=True)
316 |
317 |         name = re.sub(r'^ЖК\s*[«""]?\s*', '', name)
318 |
319 |         name = re.sub(r'[»""]$', '', name)
320 |
321 |         details["jk_name"] = name.strip()
322 |
323 | def _parse_address(soup: BeautifulSoup, details: dict) -> None:
324 |
325 |     location = soup.select_one('div[data-e2e-id="location"]')
326 |
327 |     if location:
328 |
329 |         addr_el = location.select_one('a[data-e2e-id="building_uri"]')
330 |
331 |         if addr_el:
332 |
333 |             details.setdefault("address_raw", addr_el.get_text(strip=True))
334 |
335 |         district_el = location.select_one("div.ANXSp")
336 |
337 |         if district_el:
338 |
339 |             details.setdefault("district", district_el.get_text(strip=True))
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
340 |
341 | def _parse_value(val: str, parse_type: str):
342 |
343 |     if parse_type == "text":
344 |
345 |         return val.lower().strip() if val else None
346 |
347 |     elif parse_type == "int":
348 |
349 |         return _parse_int(val)
350 |
351 |     elif parse_type == "float":
352 |
353 |         return _parse_float(val)
354 |
355 |     elif parse_type == "area":
356 |
357 |         return _parse_float(val)
358 |
359 |     elif parse_type == "year":
360 |
361 |         m = re.search(r'((?:19|20)\d{2})', val)
362 |
363 |         return int(m.group(1)) if m else None
364 |
365 |     elif parse_type == "bool":
366 |
367 |         lower = val.lower()
368 |
369 |         if any(w in lower for w in ("есть", "да")):
370 |
371 |             return True
372 |
373 |         if any(w in lower for w in ("нет",)):
374 |
375 |             return False
376 |
377 |         return None
378 |
379 |     elif parse_type == "bool_to_int":
380 |
381 |         lower = val.lower()
382 |
383 |         if any(w in lower for w in ("есть", "да")):
384 |
385 |             return 1
386 |
387 |         return None
388 |
389 |     return val
390 |
391 | def _parse_int(val: str) -> Optional[int]:
392 |
393 |     m = re.search(r'\d+', val)
394 |
395 |     return int(m.group()) if m else None
396 |
397 | def _parse_float(val: str) -> Optional[float]:
398 |
399 |     m = re.search(r'([\d]+[,.]?\d*)', val)
```

src\forecasting_service\parsers\domclick\detail_parser.py (continued)

```
400 |
401 |     if m:
402 |
403 |         return float(m.group(1).replace(',', '.'))
404 |
405 |     return None
406 |
```

src\forecasting_service\parsers\domclick\http_client.py

```
  1 | import requests
  2 |
  3 | from loguru import logger
  4 |
  5 | class QratorBlockedError(Exception):
  6 |
  7 |     def __init__(self, url: str):
  8 |
  9 |         self.url = url
 10 |
 11 |         super().__init__(f"Qrator blocked: {url}")
 12 |
 13 | class DomclickClient:
 14 |
 15 |     def __init__(self, qrator_cookie: str):
 16 |
 17 |         from forecasting_service.parsers.domclick.constants import (
 18 |
 19 |             DEFAULT_HEADERS,
 20 |
 21 |         )
 22 |
 23 |         self._session = requests.Session()
 24 |
 25 |         self._session.headers.update(DEFAULT_HEADERS)
 26 |
 27 |         self._session.cookies.set(
 28 |
 29 |             "qrator_jsid2",
 30 |
 31 |             qrator_cookie,
 32 |
 33 |             domain=".domclick.ru",
 34 |
 35 |         )
 36 |
 37 |     def get_page(self, url: str, timeout: int = 30) -> str:
 38 |
 39 |         logger.debug(f"  GET {url[:80]}...")
 40 |
 41 |         resp = self._session.get(url, timeout=timeout)
 42 |
 43 |         if self._is_qrator_block(resp):
 44 |
 45 |             raise QratorBlockedError(url)
 46 |
 47 |         resp.raise_for_status()
 48 |
 49 |         return resp.text
```

src\forecasting_service\parsers\domclick\http_client.py (continued)

```
 50 |
 51 |     def test_connection(self) -> bool:
 52 |
 53 |         from forecasting_service.parsers.domclick.constants import (
 54 |
 55 |             SEARCH_URL,
 56 |
 57 |             VLADIVOSTOK_AID,
 58 |
 59 |         )
 60 |
 61 |         test_url = (
 62 |
 63 |             f"{SEARCH_URL}?deal_type=sale&category=living"
 64 |
 65 |             f"&offer_type=flat&aids={VLADIVOSTOK_AID}"
 66 |
 67 |             f"&rooms=1&offset=0"
 68 |
 69 |         )
 70 |
 71 |         try:
 72 |
 73 |             resp = self._session.get(test_url, timeout=15)
 74 |
 75 |             if self._is_qrator_block(resp):
 76 |
 77 |                 return False
 78 |
 79 |             if resp.status_code == 200 and len(resp.text) > 50000:
 80 |
 81 |                 return True
 82 |
 83 |             return False
 84 |
 85 |         except Exception as e:
 86 |
 87 |             logger.debug(f"Тест соединения: {e}")
 88 |
 89 |             return False
 90 |
 91 |     @staticmethod
 92 |
 93 |     def _is_qrator_block(resp: requests.Response) -> bool:
 94 |
 95 |         text_lower = resp.text[:5000].lower()
 96 |
 97 |         if len(resp.text) < 10000:
 98 |
 99 |             if "qrator" in text_lower:
100 |
101 |                 return True
102 |
103 |             if "необычно" in text_lower:
104 |
105 |                 return True
106 |
107 |         if resp.status_code == 403:
108 |
109 |             return True
```

src\forecasting_service\parsers\domclick\http_client.py (continued)

```
110 |
111 |         return False
112 |
113 |     def close(self) -> None:
114 |
115 |         self._session.close()
116 |
117 |     def __enter__(self) -> "DomclickClient":
118 |
119 |         return self
120 |
121 |     def __exit__(self, *args) -> None:
122 |
123 |         self.close()
124 |
```

src\forecasting_service\parsers\domclick\page_parser.py

```
  1 | import re
  2 |
  3 | from typing import Optional
  4 |
  5 | from bs4 import BeautifulSoup, Tag
  6 |
  7 | from loguru import logger
  8 |
  9 | from forecasting_service.parsers.domclick.constants import BASE_URL
 10 |
 11 | def parse_listing_page(html: str) -> list[dict]:
 12 |
 13 |     soup = BeautifulSoup(html, "lxml")
 14 |
 15 |     cards = soup.select('div[data-e2e-id="offers-list__item"]')
 16 |
 17 |     if not cards:
 18 |
 19 |         logger.debug("DomClick: карточки не найдены")
 20 |
 21 |         return []
 22 |
 23 |     flats = []
 24 |
 25 |     seen_urls = set()
 26 |
 27 |     for card in cards:
 28 |
 29 |         try:
 30 |
 31 |             flat = _parse_card(card)
 32 |
 33 |             if flat and flat["url"] and flat["url"] not in seen_urls:
 34 |
 35 |                 seen_urls.add(flat["url"])
 36 |
 37 |                 flats.append(flat)
 38 |
 39 |         except Exception as e:
 40 |
 41 |             logger.debug(f"Ошибка парсинга карточки DC: {e}")
```

src\forecasting_service\parsers\domclick\page_parser.py (continued)

```
 42 |
 43 |     return flats
 44 |
 45 | def is_empty_listing(html: str) -> bool:
 46 |
 47 |     html_lower = html[:10000].lower()
 48 |
 49 |     indicators = [
 50 |
 51 |         "ничего не найдено",
 52 |
 53 |         "нет объявлений",
 54 |
 55 |         "попробуйте изменить",
 56 |
 57 |         "ничего не нашлось",
 58 |
 59 |     ]
 60 |
 61 |     return any(ind in html_lower for ind in indicators)
 62 |
 63 | def get_total_count(html: str) -> Optional[int]:
 64 |
 65 |     match = re.search(
 66 |
 67 |         r'(\d[\d\s]*\d)\s*(?:объявлени|предложени|квартир)',
 68 |
 69 |         html[:20000],
 70 |
 71 |     )
 72 |
 73 |     if match:
 74 |
 75 |         digits = re.sub(r'\s', '', match.group(1))
 76 |
 77 |         return int(digits) if digits else None
 78 |
 79 |     return None
 80 |
 81 | def _parse_card(card: Tag) -> Optional[dict]:
 82 |
 83 |     link_el = card.select_one(
 84 |
 85 |         'a[data-test="product-snippet-property-offer"]'
 86 |
 87 |     )
 88 |
 89 |     if not link_el:
 90 |
 91 |         return None
 92 |
 93 |     href = link_el.get("href", "")
 94 |
 95 |     url = f"{BASE_URL}{href}" if href.startswith("/") else href
 96 |
 97 |     source_id = _extract_source_id(url)
 98 |
 99 |     rooms, total_meters, floor, floors_count = _parse_title_spans(
100 |
101 |         link_el
```

src\forecasting_service\parsers\domclick\page_parser.py (continued)

```
    )

    price = _extract_price(card)

    address = _extract_address(card)

    return {

        "source": "domclick",

        "source_id": source_id,

        "url": url,

        "price": price,

        "total_meters": total_meters,

        "rooms_count": rooms,

        "floor": floor,

        "floors_count": floors_count,

        "address_raw": address,

    }

def _extract_source_id(url: str) -> Optional[str]:

    match = re.search(r'flat__(\d+)', url)

    return match.group(1) if match else None

def _parse_title_spans(

    link_el: Tag,

) -> tuple[Optional[int], Optional[float], Optional[int], Optional[int]]:

    rooms = None

    total_meters = None

    floor = None

    floors_count = None

    spans = link_el.select("span.s6KKu.siaB6")

    if len(spans) < 3:

        spans = link_el.select("span")

    if len(spans) < 3:

        return rooms, total_meters, floor, floors_count

    title_text = spans[0].get_text(strip=True)
```

src\forecasting_service\parsers\domclick\page_parser.py (continued)

```
162 |
163 |     if "студия" in title_text.lower():
164 |
165 |         rooms = 0
166 |
167 |     else:
168 |
169 |         r_match = re.search(r'(\d+)-комн', title_text)
170 |
171 |         if r_match:
172 |
173 |             rooms = int(r_match.group(1))
174 |
175 |     meters_text = spans[1].get_text(strip=True)
176 |
177 |     m_match = re.search(r'([\d,.]+)', meters_text)
178 |
179 |     if m_match:
180 |
181 |         total_meters = float(m_match.group(1).replace(',', '.'))
182 |
183 |     floor_text = spans[2].get_text(strip=True)
184 |
185 |     f_match = re.search(r'(\d+)\s*/\s*(\d+)', floor_text)
186 |
187 |     if f_match:
188 |
189 |         floor = int(f_match.group(1))
190 |
191 |         floors_count = int(f_match.group(2))
192 |
193 |     return rooms, total_meters, floor, floors_count
194 |
195 | def _extract_price(card: Tag) -> Optional[int]:
196 |
197 |     price_el = card.select_one(
198 |
199 |         'div[data-e2e-id="product-snippet-price-sale"] p'
200 |
201 |     )
202 |
203 |     if not price_el:
204 |
205 |         price_el = card.select_one('[data-e2e-id*="price"] p')
206 |
207 |     if not price_el:
208 |
209 |         return None
210 |
211 |     digits = re.sub(r'[^\d]', '', price_el.get_text(strip=True))
212 |
213 |     return int(digits) if digits else None
214 |
215 | def _extract_address(card: Tag) -> str:
216 |
217 |     addr_el = card.select_one(
218 |
219 |         'span[data-e2e-id="product-snippet-address"]'
220 |
221 |     )
```

src\forecasting_service\parsers\domclick\page_parser.py (continued)

```
222 |
223 |     if not addr_el:
224 |
225 |         addr_el = card.select_one(
226 |
227 |             'span[data-e2-id="product-snippet-address"]'
228 |
229 |         )
230 |
231 |     if not addr_el:
232 |
233 |         addr_el = card.select_one('[class*="address"]')
234 |
235 |     return addr_el.get_text(strip=True) if addr_el else ""
236 |
```

src\forecasting_service\parsers\domclick\parser.py

```
  1 | import time
  2 |
  3 | import random
  4 |
  5 | from typing import Optional
  6 |
  7 | from loguru import logger
  8 |
  9 | from forecasting_service.parsers.domclick.constants import (
 10 |
 11 |     SEARCH_URL,
 12 |
 13 |     VLADIVOSTOK_AID,
 14 |
 15 |     ROOM_PARAMS,
 16 |
 17 |     CARDS_PER_PAGE,
 18 |
 19 | )
 20 |
 21 | from forecasting_service.parsers.domclick.http_client import (
 22 |
 23 |     DomclickClient,
 24 |
 25 |     QratorBlockedError,
 26 |
 27 | )
 28 |
 29 | from forecasting_service.parsers.domclick.page_parser import (
 30 |
 31 |     parse_listing_page,
 32 |
 33 |     is_empty_listing,
 34 |
 35 |     get_total_count,
 36 |
 37 | )
 38 |
 39 | from forecasting_service.parsers.domclick.detail_parser import (
 40 |
 41 |     parse_detail_page,
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
 42 |
 43 | )
 44 |
 45 | from forecasting_service.data.storage import FlatStorage
 46 |
 47 | from forecasting_service.utils.formatting import format_stats_block
 48 |
 49 | from forecasting_service.parsers.domclick.ssr_state import (
 50 |
 51 |     extract_ssr_state_safe,
 52 |
 53 | )
 54 |
 55 | from forecasting_service.parsers.domclick.state_parser import (
 56 |
 57 |     parse_detail_state,
 58 |
 59 | )
 60 |
 61 | class DomclickListingParser:
 62 |
 63 |     def __init__(
 64 |
 65 |         self,
 66 |
 67 |         qrator_cookie: str,
 68 |
 69 |         location_id: str = VLADIVOSTOK_AID,
 70 |
 71 |         storage: Optional[FlatStorage] = None,
 72 |
 73 |         page_delay: tuple[float, float] = (2.0, 5.0),
 74 |
 75 |         detail_delay: tuple[float, float] = (1.5, 4.0),
 76 |
 77 |     ):
 78 |
 79 |         self.client = DomclickClient(qrator_cookie)
 80 |
 81 |         self.location_id = location_id
 82 |
 83 |         self.storage = storage or FlatStorage()
 84 |
 85 |         self._page_delay = page_delay
 86 |
 87 |         self._detail_delay = detail_delay
 88 |
 89 |     @staticmethod
 90 |
 91 |     def _merge_detail_dicts(
 92 |
 93 |         html_details: dict,
 94 |
 95 |         ssr_details: dict,
 96 |
 97 |     ) -> dict:
 98 |
 99 |         merged = dict(html_details or {})
100 |
101 |         for key, value in (ssr_details or {}).items():
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
102 |
103 |             if value is None:
104 |
105 |                 continue
106 |
107 |             if value == "":
108 |
109 |                 continue
110 |
111 |             merged[key] = value
112 |
113 |         return merged
114 |
115 |     def _build_search_url(
116 |
117 |         self, rooms: int | str, offset: int = 0
118 |
119 |     ) -> str:
120 |
121 |         room_param = ROOM_PARAMS.get(rooms, str(rooms))
122 |
123 |         url = (
124 |
125 |             f"{SEARCH_URL}?"
126 |
127 |             f"deal_type=sale&category=living"
128 |
129 |             f"&offer_type=flat&offer_type=layout"
130 |
131 |             f"&aids={self.location_id}"
132 |
133 |             f"&rooms={room_param}"
134 |
135 |             f"&offset={offset}"
136 |
137 |         )
138 |
139 |         return url
140 |
141 |     def collect(
142 |
143 |         self,
144 |
145 |         rooms: tuple = ("studio", 1, 2, 3),
146 |
147 |         start_page: int = 1,
148 |
149 |         end_page: int = 100,
150 |
151 |     ) -> None:
152 |
153 |         if isinstance(rooms, (int, str)):
154 |
155 |             rooms = (rooms,)
156 |
157 |         if not self.client.test_connection():
158 |
159 |             logger.error(
160 |
161 |                 "Cookie невалидна! "
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
162 |
163 |                 "Получите новую из браузера."
164 |
165 |             )
166 |
167 |             return
168 |
169 |         logger.info("✓ Cookie валидна, начинаем сбор")
170 |
171 |         for room_type in rooms:
172 |
173 |             logger.info(f"\n{'=' * 50}")
174 |
175 |             logger.info(f"  ДомКлик: комнатность {room_type}")
176 |
177 |             logger.info(f"{'=' * 50}")
178 |
179 |             self._collect_room_type(
180 |
181 |                 room_type, start_page, end_page
182 |
183 |             )
184 |
185 |         stats = self.storage.get_stats()
186 |
187 |         logger.info(
188 |
189 |             format_stats_block(stats, title="ИТОГО В БД (DomClick)")
190 |
191 |         )
192 |
193 |     def _collect_room_type(
194 |
195 |         self,
196 |
197 |         room_type: int | str,
198 |
199 |         start_page: int,
200 |
201 |         end_page: int,
202 |
203 |     ) -> None:
204 |
205 |         consecutive_empty = 0
206 |
207 |         total_offers = None
208 |
209 |         for page in range(start_page, end_page + 1):
210 |
211 |             offset = (page - 1) * CARDS_PER_PAGE
212 |
213 |             if total_offers and offset >= total_offers:
214 |
215 |                 logger.info(
216 |
217 |                     f"  Достигнут конец: offset={offset} >= {total_offers}"
218 |
219 |                 )
220 |
221 |                 break
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
222 |
223 |             if page > start_page:
224 |
225 |                 delay = random.uniform(*self._page_delay)
226 |
227 |                 time.sleep(delay)
228 |
229 |             url = self._build_search_url(room_type, offset)
230 |
231 |             try:
232 |
233 |                 html = self.client.get_page(url)
234 |
235 |                 if total_offers is None:
236 |
237 |                     total_offers = get_total_count(html)
238 |
239 |                     if total_offers:
240 |
241 |                         max_pages = (total_offers // CARDS_PER_PAGE) + 1
242 |
243 |                         logger.info(
244 |
245 |                             f"  Всего: {total_offers} объявлений "
246 |
247 |                             f"(~{max_pages} стр.)"
248 |
249 |                         )
250 |
251 |                 if is_empty_listing(html):
252 |
253 |                     logger.info(f"  Стр. {page}: пустая выдача")
254 |
255 |                     break
256 |
257 |                 flats = parse_listing_page(html)
258 |
259 |                 if not flats:
260 |
261 |                     consecutive_empty += 1
262 |
263 |                     logger.info(
264 |
265 |                         f"  Стр. {page} (offset={offset}): "
266 |
267 |                         f"0 карточек (empty={consecutive_empty})"
268 |
269 |                     )
270 |
271 |                     if consecutive_empty >= 2:
272 |
273 |                         logger.info(
274 |
275 |                             "  2 пустые подряд → "
276 |
277 |                             "следующая комнатность"
278 |
279 |                         )
280 |
281 |                         break
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
282 |
283 |                     continue
284 |
285 |                 consecutive_empty = 0
286 |
287 |                 new, updated = (
288 |
289 |                     self.storage.bulk_upsert_from_listing(flats)
290 |
291 |                 )
292 |
293 |                 logger.info(
294 |
295 |                     f"  Стр. {page} (offset={offset}): "
296 |
297 |                     f"{len(flats)} карточек → "
298 |
299 |                     f"+{new} новых, ~{updated} обновл."
300 |
301 |                 )
302 |
303 |             except QratorBlockedError:
304 |
305 |                 logger.error(
306 |
307 |                     "  Qrator заблокировал! "
308 |
309 |                     "Cookie протухла, нужна новая."
310 |
311 |                 )
312 |
313 |                 return
314 |
315 |             except Exception as e:
316 |
317 |                 logger.error(f"  Стр. {page}: ошибка {e}")
318 |
319 |                 consecutive_empty += 1
320 |
321 |                 if consecutive_empty >= 3:
322 |
323 |                     logger.error("  3 ошибки подряд, стоп")
324 |
325 |                     break
326 |
327 |     def collect_details(
328 |
329 |         self,
330 |
331 |         batch_size: int = 100,
332 |
333 |     ) -> None:
334 |
335 |         logger.info(f"\n{'=' * 50}")
336 |
337 |         logger.info(f"  ДомКлик: сбор деталей (batch={batch_size})")
338 |
339 |         logger.info(f"{'=' * 50}")
340 |
341 |         processed = 0
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
342 |
343 |         success = 0
344 |
345 |         while processed < batch_size:
346 |
347 |             flat = self.storage.get_next_for_detail(source="domclick")
348 |
349 |             if not flat:
350 |
351 |                 logger.info("  Нет объявлений для обработки")
352 |
353 |                 break
354 |
355 |             flat_id = flat["id"]
356 |
357 |             url = flat["url"]
358 |
359 |             processed += 1
360 |
361 |             logger.info(
362 |
363 |                 f"  [{processed}/{batch_size}] id={flat_id}"
364 |
365 |             )
366 |
367 |             if processed > 1:
368 |
369 |                 time.sleep(random.uniform(*self._detail_delay))
370 |
371 |             try:
372 |
373 |                 html = self.client.get_page(url)
374 |
375 |                 ssr_details = {}
376 |
377 |                 state = extract_ssr_state_safe(html)
378 |
379 |                 if state is not None:
380 |
381 |                     try:
382 |
383 |                         ssr_details = parse_detail_state(state)
384 |
385 |                     except Exception as e:
386 |
387 |                         logger.warning(f" SSR parse failed: {e}")
388 |
389 |                 html_details = {}
390 |
391 |                 try:
392 |
393 |                     html_details = parse_detail_page(html)
394 |
395 |                 except Exception as e:
396 |
397 |                     logger.debug(f" HTML fallback parse failed: {e}")
398 |
399 |                 details = self._merge_detail_dicts(
400 |
401 |                     html_details=html_details,
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
402 |
403 |                     ssr_details=ssr_details,
404 |
405 |                 )
406 |
407 |                 if not details:
408 |
409 |                     raise ValueError("Не удалось извлечь детали ни из SSR, ни из HTML")
410 |
411 |                 self.storage.update_detail(flat_id, details)
412 |
413 |                 success += 1
414 |
415 |                 filled = sum(
416 |
417 |                     1 for v in details.values()
418 |
419 |                     if v is not None and v != ""
420 |
421 |                 )
422 |
423 |                 logger.info(
424 |
425 |                     f" ✓ {filled} полей "
426 |
427 |                     f"(SSR={'yes' if ssr_details else 'no'}, "
428 |
429 |                     f"ssr_keys={len(ssr_details)}, html_keys={len(html_details)})"
430 |
431 |                 )
432 |
433 |             except QratorBlockedError:
434 |
435 |                 self.storage.mark_blocked(flat_id)
436 |
437 |                 logger.error("  Qrator заблокировал! Стоп.")
438 |
439 |                 break
440 |
441 |             except Exception as e:
442 |
443 |                 self.storage.mark_failed(flat_id)
444 |
445 |                 logger.warning(f"    ✗ Ошибка: {e}")
446 |
447 |         logger.info(
448 |
449 |             f"\n  Итого: обработано={processed}, "
450 |
451 |             f"успешно={success}"
452 |
453 |         )
454 |
455 |     def close(self) -> None:
456 |
457 |         self.client.close()
458 |
459 |     def __enter__(self) -> "DomclickListingParser":
460 |
461 |         return self
```

src\forecasting_service\parsers\domclick\parser.py (continued)

```
462 |
463 |     def __exit__(self, *args) -> None:
464 |
465 |         self.close()
466 |
```

src\forecasting_service\parsers\domclick\ssr_state.py

```
  1 | import json
  2 |
  3 | import re
  4 |
  5 | from typing import Any, Optional
  6 |
  7 | from loguru import logger
  8 |
  9 | _SSR_STATE_PATTERN = re.compile(
 10 |
 11 |     r"window\.__SSR_STATE__\s*=\s*(\{.*?\})\s*;\s*window\.__SSR_CONTEXT__",
 12 |
 13 |     re.S,
 14 |
 15 | )
 16 |
 17 | def extract_ssr_state(html: str) -> dict[str, Any]:
 18 |
 19 |     match = _SSR_STATE_PATTERN.search(html)
 20 |
 21 |     if not match:
 22 |
 23 |         raise ValueError("window.__SSR_STATE__ не найден в HTML")
 24 |
 25 |     raw_json = match.group(1)
 26 |
 27 |     valid_json_data = raw_json.replace("undefined", "null")
 28 |
 29 |     try:
 30 |
 31 |         return json.loads(valid_json_data)
 32 |
 33 |     except json.JSONDecodeError as e:
 34 |
 35 |         logger.error(f"Ошибка парсинга SSR_STATE JSON: {e}")
 36 |
 37 |         raise
 38 |
 39 | def extract_ssr_state_safe(html: str) -> Optional[dict[str, Any]]:
 40 |
 41 |     try:
 42 |
 43 |         return extract_ssr_state(html)
 44 |
 45 |     except Exception as e:
 46 |
 47 |         logger.debug(f"Не удалось извлечь SSR_STATE: {e}")
 48 |
 49 |         return None
 50 |
 51 | def deep_get(data: dict[str, Any], path: list[str], default: Any = None) -> Any:
```

src\forecasting_service\parsers\domclick\ssr_state.py (continued)

```
 52 |
 53 |     current: Any = data
 54 |
 55 |     for key in path:
 56 |
 57 |         if not isinstance(current, dict):
 58 |
 59 |             return default
 60 |
 61 |         current = current.get(key)
 62 |
 63 |         if current is None:
 64 |
 65 |             return default
 66 |
 67 |     return current
 68 |
 69 | def extract_detail_coordinates(state: dict[str, Any]) -> dict[str, Optional[float]]:
 70 |
 71 |     position = deep_get(
 72 |
 73 |         state,
 74 |
 75 |         ["productCard", "originalProduct", "address", "position"],
 76 |
 77 |         default=None,
 78 |
 79 |     )
 80 |
 81 |     if not isinstance(position, dict):
 82 |
 83 |         position = deep_get(
 84 |
 85 |             state,
 86 |
 87 |             ["productCard", "address", "position"],
 88 |
 89 |             default=None,
 90 |
 91 |         )
 92 |
 93 |     if not isinstance(position, dict):
 94 |
 95 |         return {"latitude": None, "longitude": None}
 96 |
 97 |     lat = position.get("lat")
 98 |
 99 |     lon = position.get("lon")
100 |
101 |     return {
102 |
103 |         "latitude": _to_float(lat),
104 |
105 |         "longitude": _to_float(lon),
106 |
107 |     }
108 |
109 | def _to_float(value: Any) -> Optional[float]:
110 |
111 |     try:
```

src\forecasting_service\parsers\domclick\ssr_state.py (continued)

```
112 |
113 |         if value is None:
114 |
115 |             return None
116 |
117 |         return float(value)
118 |
119 |     except (TypeError, ValueError):
120 |
121 |         return None
122 |
123 | def extract_listing_coordinates(entity: dict[str, Any]) -> dict[str, Optional[float]]:
124 |
125 |     location = entity.get("location")
126 |
127 |     if not isinstance(location, dict):
128 |
129 |         return {"latitude": None, "longitude": None}
130 |
131 |     return {
132 |
133 |         "latitude": _to_float(location.get("lat")),
134 |
135 |         "longitude": _to_float(location.get("lon")),
136 |
137 |     }
138 |
```

src\forecasting_service\parsers\domclick\state_parser.py

```
  1 | from __future__ import annotations
  2 |
  3 | import json
  4 |
  5 | from typing import Any, Optional
  6 |
  7 | from forecasting_service.parsers.domclick.ssr_state import deep_get
  8 |
  9 | def _count_house_photo_types(items: list[Any]) -> dict[str, int]:
 10 |
 11 |     counts = {
 12 |
 13 |         "facade": 0,
 14 |
 15 |         "lift": 0,
 16 |
 17 |         "entrance_inside": 0,
 18 |
 19 |         "entrance_outside": 0,
 20 |
 21 |         "territory": 0,
 22 |
 23 |         "other": 0,
 24 |
 25 |     }
 26 |
 27 |     if not isinstance(items, list):
 28 |
 29 |         return counts
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
    |
    |     for item in items:
    |
    |         if not isinstance(item, dict):
    |
    |             continue
    |
    |         photo_type = item.get("type")
    |
    |         if photo_type == "Facade":
    |
    |             counts["facade"] += 1
    |
    |         elif photo_type == "Lift":
    |
    |             counts["lift"] += 1
    |
    |         elif photo_type == "EntranceInside":
    |
    |             counts["entrance_inside"] += 1
    |
    |         elif photo_type == "EntranceOutside":
    |
    |             counts["entrance_outside"] += 1
    |
    |         elif photo_type == "Territory":
    |
    |             counts["territory"] += 1
    |
    |         else:
    |
    |             counts["other"] += 1
    |
    |     return counts
    |
    | def parse_detail_state(raw_state: dict[str, Any]) -> dict[str, Any]:
    |
    |     state = raw_state
    |
    |     product = _get_product_card(state)
    |
    |     original = product.get("originalProduct", {}) if isinstance(product, dict) else {}
    |
    |     address_orig = original.get("address", {}) if isinstance(original, dict) else {}
    |
    |     address_norm = product.get("address", {}) if isinstance(product, dict) else {}
    |
    |     object_orig = original.get("object_info", {}) if isinstance(original, dict) else {}
    |
    |     object_norm = product.get("objectInfo", {}) if isinstance(product, dict) else {}
    |
    |     house_orig = original.get("house", {}) if isinstance(original, dict) else {}
    |
    |     house_norm = deep_get(product, ["house", "info"], default=None)
    |
    |     if not isinstance(house_norm, dict):
    |
    |         raw_house = product.get("house", {})
    |
    |         house_norm = raw_house if isinstance(raw_house, dict) else {}
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
 90 |
 91 |     if not isinstance(house_norm, dict):
 92 |
 93 |         house_norm = {}
 94 |
 95 |     house_state = deep_get(state, ["houseInfo", "info"], default={}) or {}
 96 |
 97 |     if not isinstance(house_state, dict):
 98 |
 99 |         house_state = {}
100 |
101 |     offer_photos = product.get("photos") or original.get("photos") or []
102 |
103 |     if not isinstance(offer_photos, list):
104 |
105 |         offer_photos = []
106 |
107 |     house_photos = deep_get(state, ["houseInfo", "photos"], default=[]) or []
108 |
109 |     if not isinstance(house_photos, list):
110 |
111 |         house_photos = []
112 |
113 |     has_plan_photo = any(
114 |
115 |         isinstance(photo, dict) and photo.get("isPlan") is True
116 |
117 |         for photo in offer_photos
118 |
119 |     )
120 |
121 |     raw_original_photos = original.get("photos", [])
122 |
123 |     if not isinstance(raw_original_photos, list):
124 |
125 |         raw_original_photos = []
126 |
127 |     has_window_photo = any(
128 |
129 |         isinstance(photo, dict) and photo.get("class_name") == "FromWindow"
130 |
131 |         for photo in raw_original_photos
132 |
133 |     )
134 |
135 |     house_photo_counts = _count_house_photo_types(house_photos)
136 |
137 |     legal_orig = original.get("legal_options", {}) if isinstance(original, dict) else {}
138 |
139 |     legal_norm = product.get("legalOptions", {}) if isinstance(product, dict) else {}
140 |
141 |     seller_orig = original.get("seller", {}) if isinstance(original, dict) else {}
142 |
143 |     seller_norm = product.get("seller", {}) if isinstance(product, dict) else {}
144 |
145 |     stats_orig = original.get("offer_stat", {}) if isinstance(original, dict) else {}
146 |
147 |     egrn_orig = original.get("egrn_data", {}) if isinstance(original, dict) else {}
148 |
149 |     egrn_norm = product.get("egrnData", {}) if isinstance(product, dict) else {}
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
150 |
151 |     flat_complex_orig = original.get("flat_complex", {}) if isinstance(original, dict) else {}
152 |
153 |     flat_complex_norm = product.get("flatComplex", {}) if isinstance(product, dict) else {}
154 |
155 |     price_orig = original.get("price_info", {}) if isinstance(original, dict) else {}
156 |
157 |     price_norm = product.get("priceInfo", {}) if isinstance(product, dict) else {}
158 |
159 |     price_prediction = state.get("pricePrediction", {}) if isinstance(state, dict) else {}
160 |
161 |     source_id = _first_not_none(
162 |
163 |         original.get("id"),
164 |
165 |         product.get("id"),
166 |
167 |         product.get("_id"),
168 |
169 |     )
170 |
171 |     lat = _first_not_none(
172 |
173 |         deep_get(address_orig, ["position", "lat"]),
174 |
175 |         deep_get(address_norm, ["position", "lat"]),
176 |
177 |     )
178 |
179 |     lon = _first_not_none(
180 |
181 |         deep_get(address_orig, ["position", "lon"]),
182 |
183 |         deep_get(address_norm, ["position", "lon"]),
184 |
185 |     )
186 |
187 |     district_name, district_guid, district_slug = _extract_district(address_orig, address_norm)
188 |
189 |     street_name, street_guid = _extract_street(address_orig, address_norm)
190 |
191 |     house_number = _extract_house_number(address_orig, address_norm)
192 |
193 |     province_guid = _extract_parent_guid(address_orig, address_norm, "province")
194 |
195 |     locality_guid = _first_not_none(
196 |
197 |         deep_get(address_orig, ["locality", "guid"]),
198 |
199 |         address_norm.get("localityGuid"),
200 |
201 |         _extract_parent_guid(address_orig, address_norm, "locality"),
202 |
203 |     )
204 |
205 |     parking_keys, parking_labels = _extract_named_items(
206 |
207 |         _first_non_empty_list(house_orig.get("parking"), house_norm.get("parkingWithKey"))
208 |
209 |     )
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
210 |
211 |     security_keys, security_labels = _extract_named_items(
212 |
213 |         _first_non_empty_list(house_orig.get("security"), house_norm.get("securityWithKey"))
214 |
215 |     )
216 |
217 |     yard_keys, yard_labels = _extract_named_items(
218 |
219 |         _first_non_empty_list(house_orig.get("yard"), house_norm.get("yardWithKey"))
220 |
221 |     )
222 |
223 |     infra_keys, infra_labels = _extract_named_items(
224 |
225 |         _first_non_empty_list(house_orig.get("infrastructure"), house_norm.get("infrastructureWithKe ...
226 |
227 |     )
228 |
229 |     window_view_labels = _extract_window_view_labels(
230 |
231 |         _first_not_none(object_orig.get("window_view"), object_norm.get("windowView"))
232 |
233 |     )
234 |
235 |     seller_agent_orig = seller_orig.get("agent", {}) if isinstance(seller_orig, dict) else {}
236 |
237 |     seller_agent_norm = seller_norm.get("agent", {}) if isinstance(seller_norm, dict) else {}
238 |
239 |     seller_company_orig = seller_orig.get("company", {}) if isinstance(seller_orig, dict) else {}
240 |
241 |     seller_company_norm = seller_norm.get("company", {}) if isinstance(seller_norm, dict) else {}
242 |
243 |     is_owner = _first_not_none(
244 |
245 |         legal_orig.get("is_owner"),
246 |
247 |         legal_norm.get("isOwner"),
248 |
249 |         product.get("isOwner"),
250 |
251 |     )
252 |
253 |     is_agent = _first_not_none(
254 |
255 |         seller_agent_orig.get("is_agent"),
256 |
257 |         seller_agent_norm.get("isAgent"),
258 |
259 |     )
260 |
261 |     author_type = _detect_author_type(
262 |
263 |         source=original.get("source"),
264 |
265 |         is_owner=is_owner,
266 |
267 |         is_agent=is_agent,
268 |
269 |         company=seller_company_orig or seller_company_norm,
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
    )

    tariff_orig = original.get("tariff_options", {}) if isinstance(original, dict) else {}

    tariff_norm = product.get("tariffOptions", {}) if isinstance(product, dict) else {}

    parsed = {

        "source": "domclick",

        "source_id": str(source_id) if source_id is not None else None,

        "url": _first_not_none(

            product.get("href"),

            original.get("href"),

        ),

        "price": _to_int(_first_not_none(

            price_orig.get("price"),

            price_norm.get("price"),

        )),

        "price_per_sqm": _to_float(_first_not_none(

            price_orig.get("square_price"),

            price_norm.get("squarePrice"),

        )),

        "total_meters": _to_float(_first_not_none(

            object_orig.get("area"),

            object_norm.get("area"),

        )),

        "living_meters": _to_float(_first_not_none(

            object_orig.get("living_area"),

            object_norm.get("livingArea"),

        )),

        "kitchen_meters": _to_float(_first_not_none(

            object_orig.get("kitchen_area"),

            object_norm.get("kitchenArea"),

        )),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
330 |
331 |         "rooms_count": _to_int(_first_not_none(
332 |
333 |             object_orig.get("rooms"),
334 |
335 |             object_norm.get("rooms"),
336 |
337 |         )),
338 |
339 |         "floor": _to_int(_first_not_none(
340 |
341 |             object_orig.get("floor"),
342 |
343 |             object_norm.get("floor"),
344 |
345 |         )),
346 |
347 |         "floors_count": _to_int(_first_not_none(
348 |
349 |             house_orig.get("floors"),
350 |
351 |             house_norm.get("floors"),
352 |
353 |         )),
354 |
355 |         "latitude": _to_float(lat),
356 |
357 |         "longitude": _to_float(lon),
358 |
359 |         "address_raw": _first_not_none(
360 |
361 |             address_orig.get("short_display_name"),
362 |
363 |             address_norm.get("displayNameShort"),
364 |
365 |             address_orig.get("display_name"),
366 |
367 |             address_norm.get("displayName"),
368 |
369 |         ),
370 |
371 |         "address_guid": _first_not_none(
372 |
373 |             address_orig.get("guid"),
374 |
375 |             address_norm.get("guid"),
376 |
377 |         ),
378 |
379 |         "district": district_name,
380 |
381 |         "district_guid": district_guid,
382 |
383 |         "district_slug": district_slug,
384 |
385 |         "street": street_name,
386 |
387 |         "street_guid": street_guid,
388 |
389 |         "house_number": house_number,
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
390 |
391 |         "locality": _first_not_none(
392 |
393 |             deep_get(address_orig, ["locality", "name"]),
394 |
395 |             address_norm.get("locality"),
396 |
397 |         ),
398 |
399 |         "locality_guid": locality_guid,
400 |
401 |         "province_guid": province_guid,
402 |
403 |         "timezone": _first_not_none(
404 |
405 |             deep_get(address_orig, ["info", "timezone"]),
406 |
407 |             address_norm.get("timezone"),
408 |
409 |         ),
410 |
411 |         "timezone_offset": _to_int(_first_not_none(
412 |
413 |             deep_get(address_orig, ["info", "timezone_offset"]),
414 |
415 |             address_norm.get("timezoneOffset"),
416 |
417 |         )),
418 |
419 |         "is_apartment": _to_bool(_first_not_none(
420 |
421 |             object_orig.get("is_apartment"),
422 |
423 |             object_norm.get("isApartment"),
424 |
425 |         )),
426 |
427 |         "finish_type": _extract_display_name_or_value(
428 |
429 |             object_orig.get("renovation"),
430 |
431 |             object_norm.get("renovation"),
432 |
433 |         ),
434 |
435 |         "bathroom_type": _extract_display_name_or_value(
436 |
437 |             object_orig.get("bathroom"),
438 |
439 |             object_norm.get("bathroom"),
440 |
441 |         ),
442 |
443 |         "bathroom_count": _calc_bathroom_count(
444 |
445 |             object_orig,
446 |
447 |             object_norm,
448 |
449 |         ),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
450 |
451 |         "balcony_count": _to_int(_first_not_none(
452 |
453 |             object_orig.get("balconies"),
454 |
455 |             object_norm.get("balconies"),
456 |
457 |         )),
458 |
459 |         "loggia_count": _to_int(_first_not_none(
460 |
461 |             object_orig.get("loggias"),
462 |
463 |             object_norm.get("loggias"),
464 |
465 |         )),
466 |
467 |         "window_view": ", ".join(window_view_labels) if window_view_labels else None,
468 |
469 |         "window_view_json": window_view_labels,
470 |
471 |         "has_gas": _to_bool(_first_not_none(
472 |
473 |             object_orig.get("has_gas"),
474 |
475 |             object_norm.get("hasGas"),
476 |
477 |         )),
478 |
479 |         "has_redevelopment": _to_bool(_first_not_none(
480 |
481 |             object_orig.get("redevelopment"),
482 |
483 |             object_norm.get("redevelopment"),
484 |
485 |         )),
486 |
487 |         "year_of_construction": _to_int(_first_not_none(
488 |
489 |             house_orig.get("build_year"),
490 |
491 |             house_norm.get("buildYear"),
492 |
493 |         )),
494 |
495 |         "house_material_type": _extract_display_name_or_value(
496 |
497 |             house_orig.get("wall_type"),
498 |
499 |             house_norm.get("wallType"),
500 |
501 |         ),
502 |
503 |         "floor_type": _first_not_none(
504 |
505 |             house_norm.get("floorType"),
506 |
507 |             house_state.get("floorType"),
508 |
509 |         ),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
510 |
511 |         "elevator_passenger": _to_int(_first_not_none(
512 |
513 |             house_orig.get("lifts_passenger"),
514 |
515 |             house_norm.get("liftsPassenger"),
516 |
517 |         )),
518 |
519 |         "elevator_cargo": _to_int(_first_not_none(
520 |
521 |             house_orig.get("lifts_freight"),
522 |
523 |             house_norm.get("liftsFreight"),
524 |
525 |         )),
526 |
527 |         "has_garbage_chute": _to_bool(_first_not_none(
528 |
529 |             house_orig.get("has_garbage_disposer"),
530 |
531 |             house_norm.get("hasGarbageDisposer"),
532 |
533 |         )),
534 |
535 |         "entrances_count": _to_int(_first_not_none(
536 |
537 |             house_norm.get("entranceCount"),
538 |
539 |             house_state.get("entranceCount"),
540 |
541 |         )),
542 |
543 |         "quarters_count": _to_int(_first_not_none(
544 |
545 |             house_norm.get("quartersCount"),
546 |
547 |             house_state.get("quartersCount"),
548 |
549 |         )),
550 |
551 |         "living_quarters_count": _to_int(_first_not_none(
552 |
553 |             house_norm.get("livingQuartersCount"),
554 |
555 |             house_state.get("livingQuartersCount"),
556 |
557 |         )),
558 |
559 |         "heating_type": _first_not_none(
560 |
561 |             house_norm.get("heatingType"),
562 |
563 |             house_state.get("heatingType"),
564 |
565 |         ),
566 |
567 |         "hot_water_type": _first_not_none(
568 |
569 |             house_norm.get("hotWaterType"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
570 |
571 |             house_state.get("hotWaterType"),
572 |
573 |         ),
574 |
575 |         "cold_water_type": _first_not_none(
576 |
577 |             house_norm.get("coldWaterType"),
578 |
579 |             house_state.get("coldWaterType"),
580 |
581 |         ),
582 |
583 |         "foundation_type": _first_not_none(
584 |
585 |             house_norm.get("foundationType"),
586 |
587 |             house_state.get("foundationType"),
588 |
589 |         ),
590 |
591 |         "ventilation_type": _first_not_none(
592 |
593 |             house_norm.get("ventilationType"),
594 |
595 |             house_state.get("ventilationType"),
596 |
597 |         ),
598 |
599 |         "energy_efficiency": _first_not_none(
600 |
601 |             house_norm.get("energyEfficiency"),
602 |
603 |             house_state.get("energyEfficiency"),
604 |
605 |         ),
606 |
607 |         "has_intercom": "intercom" in parking_or_security_keys(security_keys),
608 |
609 |         "has_concierge": "concierge" in parking_or_security_keys(security_keys),
610 |
611 |         "has_closed_territory": "closed_area" in parking_or_security_keys(security_keys),
612 |
613 |         "has_code_door": "code_door" in parking_or_security_keys(security_keys),
614 |
615 |         "has_garage": "garage" in parking_or_security_keys(parking_keys),
616 |
617 |         "parking_type": ", ".join(parking_labels) if parking_labels else None,
618 |
619 |         "parking_keys_json": parking_keys,
620 |
621 |         "security_features": ", ".join(security_labels) if security_labels else None,
622 |
623 |         "security_keys_json": security_keys,
624 |
625 |         "yard_features": ", ".join(yard_labels) if yard_labels else None,
626 |
627 |         "yard_keys_json": yard_keys,
628 |
629 |         "infrastructure_features": ", ".join(infra_labels) if infra_labels else None,
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
630 |
631 |         "infrastructure_keys_json": infra_keys,
632 |
633 |         "jk_id": _to_int(_first_not_none(
634 |
635 |             flat_complex_orig.get("id"),
636 |
637 |             flat_complex_norm.get("id"),
638 |
639 |         )),
640 |
641 |         "jk_name": _first_not_none(
642 |
643 |             flat_complex_orig.get("name"),
644 |
645 |             flat_complex_norm.get("name"),
646 |
647 |         ),
648 |
649 |         "jk_slug": _first_not_none(
650 |
651 |             flat_complex_orig.get("slug"),
652 |
653 |             flat_complex_norm.get("slug"),
654 |
655 |         ),
656 |
657 |         "is_owner": _to_bool(is_owner),
658 |
659 |         "author_type": author_type,
660 |
661 |         "owner_count": _to_int(_first_not_none(
662 |
663 |             legal_orig.get("owner_count"),
664 |
665 |             legal_norm.get("ownerCount"),
666 |
667 |         )),
668 |
669 |         "owner_minors": _to_bool(_first_not_none(
670 |
671 |             legal_orig.get("owner_minors"),
672 |
673 |             legal_norm.get("ownerMinors"),
674 |
675 |         )),
676 |
677 |         "residence_minors": _to_bool(_first_not_none(
678 |
679 |             legal_orig.get("residence_minors"),
680 |
681 |             legal_norm.get("residenceMinors"),
682 |
683 |         )),
684 |
685 |         "years_ownership": _extract_display_name_or_value(
686 |
687 |             legal_orig.get("years_ownership"),
688 |
689 |             legal_norm.get("yearsOwnership"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
690 |
691 |         ),
692 |
693 |         "sale_type": _extract_display_name_or_value(
694 |
695 |             legal_orig.get("sale_type"),
696 |
697 |             legal_norm.get("saleType"),
698 |
699 |         ),
700 |
701 |         "seller_name": _first_not_none(
702 |
703 |             seller_agent_orig.get("full_name"),
704 |
705 |             seller_agent_norm.get("fullName"),
706 |
707 |         ),
708 |
709 |         "seller_phone_masked": _first_not_none(
710 |
711 |             seller_agent_orig.get("phone"),
712 |
713 |             seller_agent_norm.get("phone"),
714 |
715 |         ),
716 |
717 |         "seller_is_agent": _to_bool(is_agent),
718 |
719 |         "seller_is_sbol_verified": _to_bool(_first_not_none(
720 |
721 |             seller_agent_orig.get("is_sbol_verified"),
722 |
723 |             seller_agent_norm.get("isSbolVerified"),
724 |
725 |         )),
726 |
727 |         "seller_is_esia_verified": _to_bool(_first_not_none(
728 |
729 |             seller_agent_orig.get("is_esia_verified"),
730 |
731 |             seller_agent_norm.get("isEsiaVerified"),
732 |
733 |         )),
734 |
735 |         "seller_company_name": _first_not_none(
736 |
737 |             seller_company_orig.get("trade_name"),
738 |
739 |             seller_company_orig.get("displayName"),
740 |
741 |             seller_company_norm.get("tradeName"),
742 |
743 |             seller_company_norm.get("displayName"),
744 |
745 |         ),
746 |
747 |         "seller_company_id": _to_int(_first_not_none(
748 |
749 |             seller_company_orig.get("id"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
            seller_company_norm.get("id"),

        )),

        "source_type": original.get("source"),

        "views_count": _to_int(_first_not_none(

            stats_orig.get("views_count"),

            product.get("viewsCount"),

        )),

        "calls_count": _to_int(_first_not_none(

            stats_orig.get("calls_count"),

            product.get("callsCount"),

        )),

        "favorites_count": _to_int(_first_not_none(

            stats_orig.get("favorite_offer_users_count"),

            product.get("favoriteOfferUsersCount"),

        )),

        "online_show": _to_bool(_first_not_none(

            original.get("online_show"),

            product.get("onlineShow"),

        )),

        "chat_available": _to_bool(_first_not_none(

            original.get("chat_available"),

            product.get("chatAvailable"),

        )),

        "is_auction": _to_bool(_first_not_none(

            original.get("is_auction"),

            product.get("isAuction"),

        )),

        "is_exclusive": _to_bool(original.get("is_exclusive")),

        "is_placement_paid": _to_bool(_first_not_none(

            original.get("is_placement_paid"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
            product.get("isPlacementPaid"),

        )),

        "duplicates_offer_count": _to_int(_first_not_none(

            product.get("duplicatesOfferCount"),

            original.get("duplicatesOfferCount"),

        )),

        "published_at_source": _first_not_none(

            original.get("published_dt"),

            product.get("publishedDate"),

        ),

        "updated_at_source": _first_not_none(

            original.get("updated_dt"),

            product.get("updatedDate"),

        ),

        "description": _normalize_text(_first_not_none(

            original.get("description"),

            object_orig.get("description"),

            object_norm.get("description"),

        )),

        "egrn_area_status": _first_not_none(

            deep_get(egrn_orig, ["area", "status"]),

            deep_get(egrn_norm, ["area", "status"]),

        ),

        "egrn_floor_status": _first_not_none(

            deep_get(egrn_orig, ["floor", "status"]),

            deep_get(egrn_norm, ["floor", "status"]),

        ),

        "egrn_owners_status": _first_not_none(

            deep_get(egrn_orig, ["owners_count", "status"]),

            deep_get(egrn_norm, ["owners_count", "status"]),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
870 |
871 |         ),
872 |
873 |         "collateral": _to_bool(_first_not_none(
874 |
875 |             egrn_orig.get("collateral"),
876 |
877 |             egrn_norm.get("collateral"),
878 |
879 |         )),
880 |
881 |         "collateral_sber": _to_bool(_first_not_none(
882 |
883 |             egrn_orig.get("collateral_sber"),
884 |
885 |             egrn_norm.get("collateral_sber"),
886 |
887 |         )),
888 |
889 |         "market_price_min": _to_int(price_prediction.get("minMarketPrice")),
890 |
891 |         "market_price_max": _to_int(price_prediction.get("maxMarketPrice")),
892 |
893 |         "market_price": _to_int(price_prediction.get("marketPrice")),
894 |
895 |         "rent_long_predicted": _to_int(price_prediction.get("rentLongPricePredicted")),
896 |
897 |         "rent_short_predicted": _to_int(price_prediction.get("rentShortPricePredicted")),
898 |
899 |         "repair_quality_predicted": _to_int(price_prediction.get("repairQuality")),
900 |
901 |         "price_history_json": _first_not_none(
902 |
903 |             price_orig.get("price_history"),
904 |
905 |             price_norm.get("priceHistory"),
906 |
907 |         ),
908 |
909 |         "offer_photos_json": product.get("photos") or original.get("photos"),
910 |
911 |         "house_photos_json": deep_get(state, ["houseInfo", "photos"], default=[]),
912 |
913 |         "address_parents_json": _first_not_none(
914 |
915 |             address_orig.get("parents"),
916 |
917 |             address_norm.get("parents"),
918 |
919 |         ),
920 |
921 |         "discounts_json": _first_not_none(
922 |
923 |             original.get("discount_status"),
924 |
925 |             product.get("discountStatus"),
926 |
927 |         ),
928 |
929 |         "offer_photos_count": len(offer_photos),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
930 |
931 |         "house_photos_count": len(house_photos),
932 |
933 |         "has_plan_photo": has_plan_photo,
934 |
935 |         "has_window_photo": has_window_photo,
936 |
937 |         "house_photo_facade_count": house_photo_counts["facade"],
938 |
939 |         "house_photo_lift_count": house_photo_counts["lift"],
940 |
941 |         "house_photo_entrance_inside_count": house_photo_counts["entrance_inside"],
942 |
943 |         "house_photo_entrance_outside_count": house_photo_counts["entrance_outside"],
944 |
945 |         "house_photo_territory_count": house_photo_counts["territory"],
946 |
947 |         "approve_for_mortgage": _to_bool(_first_not_none(
948 |
949 |             legal_orig.get("approve"),
950 |
951 |             legal_norm.get("approve"),
952 |
953 |         )),
954 |
955 |         "without_evaluation": _to_bool(_first_not_none(
956 |
957 |             legal_orig.get("without_evaluation"),
958 |
959 |             legal_norm.get("withoutEvaluation"),
960 |
961 |         )),
962 |
963 |         "is_individual_seller": _to_bool(_first_not_none(
964 |
965 |             legal_orig.get("is_individual"),
966 |
967 |             legal_norm.get("isIndividual"),
968 |
969 |         )),
970 |
971 |         "seller_cas_id": _to_int(_first_not_none(
972 |
973 |             seller_agent_orig.get("cas_id"),
974 |
975 |             seller_agent_norm.get("casId"),
976 |
977 |         )),
978 |
979 |         "seller_phone_visible": _to_bool(_first_not_none(
980 |
981 |             seller_agent_orig.get("show_original_phone"),
982 |
983 |             seller_agent_norm.get("showOriginalPhone"),
984 |
985 |         )),
986 |
987 |         "tariff_name": _first_not_none(
988 |
989 |             tariff_orig.get("name"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
 990 |
 991 |             tariff_norm.get("name"),
 992 |
 993 |         ),
 994 |
 995 |         "tariff_display_name": _first_not_none(
 996 |
 997 |             tariff_orig.get("display_name"),
 998 |
 999 |             tariff_norm.get("displayName"),
1000 |
1001 |         ),
1002 |
1003 |         "area_common_property": _to_float(_first_not_none(
1004 |
1005 |             house_state.get("areaCommonProperty"),
1006 |
1007 |         )),
1008 |
1009 |         "area_residential_total": _to_float(_first_not_none(
1010 |
1011 |             house_state.get("areaResidential"),
1012 |
1013 |         )),
1014 |
1015 |         "area_non_residential": _to_float(_first_not_none(
1016 |
1017 |             house_state.get("areaNonResidential"),
1018 |
1019 |         )),
1020 |
1021 |         "parking_square": _to_float(_first_not_none(
1022 |
1023 |             house_state.get("parkingSquare"),
1024 |
1025 |         )),
1026 |
1027 |         "gas_type": _to_bool(_first_not_none(
1028 |
1029 |             house_state.get("gasType"),
1030 |
1031 |         )),
1032 |
1033 |         "sewerage_type": _first_not_none(
1034 |
1035 |             house_state.get("sewerageType"),
1036 |
1037 |         ),
1038 |
1039 |         "electrical_type": _first_not_none(
1040 |
1041 |             house_state.get("electricalType"),
1042 |
1043 |         ),
1044 |
1045 |         "has_family_mortgage": _to_bool(_first_not_none(
1046 |
1047 |             original.get("has_family_mortgage"),
1048 |
1049 |             product.get("hasFamilyMortgage"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
1050 |
1051 |         )),
1052 |
1053 |         "has_domclick_plan": bool(_first_not_none(
1054 |
1055 |             deep_get(original, ["domclick_plan", "url"]),
1056 |
1057 |             deep_get(product, ["domclickPlan", "url"]),
1058 |
1059 |         )),
1060 |
1061 |         "mortgage_badge": _to_bool(
1062 |
1063 |             deep_get(product, ["mortgageBadge", "has_badge_info"])
1064 |
1065 |         ),
1066 |
1067 |         "base_credit_rate": _to_float(
1068 |
1069 |             deep_get(product, ["mortgageBadge", "base_credit_rate"])
1070 |
1071 |         ),
1072 |
1073 |     }
1074 |
1075 |     return parsed
1076 |
1077 | def _get_product_card(state: dict[str, Any]) -> dict[str, Any]:
1078 |
1079 |     product = state.get("productCard")
1080 |
1081 |     return product if isinstance(product, dict) else {}
1082 |
1083 | def _first_not_none(*values: Any) -> Any:
1084 |
1085 |     for value in values:
1086 |
1087 |         if value is not None:
1088 |
1089 |             return value
1090 |
1091 |     return None
1092 |
1093 | def _first_non_empty_list(*values: Any) -> list[Any]:
1094 |
1095 |     for value in values:
1096 |
1097 |         if isinstance(value, list) and value:
1098 |
1099 |             return value
1100 |
1101 |     return []
1102 |
1103 | def _to_int(value: Any) -> Optional[int]:
1104 |
1105 |     try:
1106 |
1107 |         if value is None or value == "":
1108 |
1109 |             return None
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
1110 |
1111 |         return int(float(value))
1112 |
1113 |     except (TypeError, ValueError):
1114 |
1115 |         return None
1116 |
1117 | def _to_float(value: Any) -> Optional[float]:
1118 |
1119 |     try:
1120 |
1121 |         if value is None or value == "":
1122 |
1123 |             return None
1124 |
1125 |         return float(value)
1126 |
1127 |     except (TypeError, ValueError):
1128 |
1129 |         return None
1130 |
1131 | def _to_bool(value: Any) -> Optional[bool]:
1132 |
1133 |     if value is None:
1134 |
1135 |         return None
1136 |
1137 |     if isinstance(value, bool):
1138 |
1139 |         return value
1140 |
1141 |     if isinstance(value, (int, float)):
1142 |
1143 |         return bool(value)
1144 |
1145 |     if isinstance(value, str):
1146 |
1147 |         low = value.strip().lower()
1148 |
1149 |         if low in {"true", "1", "yes", "да", "есть"}:
1150 |
1151 |             return True
1152 |
1153 |         if low in {"false", "0", "no", "нет"}:
1154 |
1155 |             return False
1156 |
1157 |     return None
1158 |
1159 | def _extract_display_name_or_value(*values: Any) -> Optional[str]:
1160 |
1161 |     for value in values:
1162 |
1163 |         if value is None:
1164 |
1165 |             continue
1166 |
1167 |         if isinstance(value, dict):
1168 |
1169 |             display_name = value.get("display_name") or value.get("displayName")
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
1170 |
1171 |             if display_name:
1172 |
1173 |                 return str(display_name)
1174 |
1175 |         elif isinstance(value, str) and value.strip():
1176 |
1177 |             return value.strip()
1178 |
1179 |     return None
1180 |
1181 | def _extract_named_items(items: list[Any]) -> tuple[list[str], list[str]]:
1182 |
1183 |     keys: list[str] = []
1184 |
1185 |     labels: list[str] = []
1186 |
1187 |     for item in items:
1188 |
1189 |         if isinstance(item, dict):
1190 |
1191 |             key = item.get("key")
1192 |
1193 |             label = item.get("display_name") or item.get("displayName")
1194 |
1195 |             if key:
1196 |
1197 |                 keys.append(str(key))
1198 |
1199 |             if label:
1200 |
1201 |                 labels.append(str(label))
1202 |
1203 |         elif isinstance(item, str):
1204 |
1205 |             labels.append(item)
1206 |
1207 |     return keys, labels
1208 |
1209 | def _extract_window_view_labels(value: Any) -> list[str]:
1210 |
1211 |     result: list[str] = []
1212 |
1213 |     if isinstance(value, list):
1214 |
1215 |         for item in value:
1216 |
1217 |             if isinstance(item, dict):
1218 |
1219 |                 label = item.get("display_name") or item.get("displayName")
1220 |
1221 |                 if label:
1222 |
1223 |                     result.append(str(label))
1224 |
1225 |             elif isinstance(item, str):
1226 |
1227 |                 result.append(item)
1228 |
1229 |     return result
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
1230 |
1231 | def _extract_parent_guid(address_orig: dict[str, Any], address_norm: dict[str, Any], kind: str) -> O ...
1232 |
1233 |     parents = _first_not_none(address_orig.get("parents"), address_norm.get("parents"))
1234 |
1235 |     if not isinstance(parents, list):
1236 |
1237 |         return None
1238 |
1239 |     for parent in parents:
1240 |
1241 |         if isinstance(parent, dict) and parent.get("kind") == kind:
1242 |
1243 |             guid = parent.get("guid")
1244 |
1245 |             if guid:
1246 |
1247 |                 return str(guid)
1248 |
1249 |     return None
1250 |
1251 | def _extract_district(
1252 |
1253 |     address_orig: dict[str, Any],
1254 |
1255 |     address_norm: dict[str, Any],
1256 |
1257 | ) -> tuple[Optional[str], Optional[str], Optional[str]]:
1258 |
1259 |     district_name = None
1260 |
1261 |     district_guid = None
1262 |
1263 |     district_slug = None
1264 |
1265 |     parents = _first_not_none(
1266 |
1267 |         address_orig.get("parents"),
1268 |
1269 |         address_norm.get("parents"),
1270 |
1271 |     )
1272 |
1273 |     if isinstance(parents, list):
1274 |
1275 |         for parent in parents:
1276 |
1277 |             if isinstance(parent, dict) and parent.get("kind") == "district":
1278 |
1279 |                 district_name = parent.get("name")
1280 |
1281 |                 district_guid = parent.get("guid")
1282 |
1283 |                 break
1284 |
1285 |     districts = _first_not_none(
1286 |
1287 |         address_orig.get("districts"),
1288 |
1289 |         address_norm.get("districts"),
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
1290 |
1291 |     )
1292 |
1293 |     if isinstance(districts, list) and districts:
1294 |
1295 |         item = districts[0]
1296 |
1297 |         if isinstance(item, dict):
1298 |
1299 |             district_slug = item.get("slug")
1300 |
1301 |             if district_name is None:
1302 |
1303 |                 district_name = item.get("short_name")
1304 |
1305 |     return district_name, district_guid, district_slug
1306 |
1307 | def _normalize_text(value: Any) -> Optional[str]:
1308 |
1309 |     if not isinstance(value, str):
1310 |
1311 |         return None
1312 |
1313 |     return value.replace("\\n", "\n").strip()
1314 |
1315 | def _extract_street(address_orig: dict[str, Any], address_norm: dict[str, Any]) -> tuple[Optional[st ...
1316 |
1317 |     parents = _first_not_none(address_orig.get("parents"), address_norm.get("parents"))
1318 |
1319 |     if not isinstance(parents, list):
1320 |
1321 |         return None, None
1322 |
1323 |     for parent in parents:
1324 |
1325 |         if isinstance(parent, dict) and parent.get("kind") == "street":
1326 |
1327 |             return parent.get("name"), parent.get("guid")
1328 |
1329 |     return None, None
1330 |
1331 | def _extract_house_number(address_orig: dict[str, Any], address_norm: dict[str, Any]) -> Optional[st ...
1332 |
1333 |     name = _first_not_none(address_orig.get("name"), address_norm.get("name"))
1334 |
1335 |     street = _extract_street(address_orig, address_norm)[0]
1336 |
1337 |     if not isinstance(name, str):
1338 |
1339 |         return None
1340 |
1341 |     if street and isinstance(street, str) and name.startswith(street):
1342 |
1343 |         raw = name[len(street):].strip(", ").strip()
1344 |
1345 |         return raw or None
1346 |
1347 |     parts = [p.strip() for p in name.split(",")]
1348 |
1349 |     return parts[-1] if parts else None
```

src\forecasting_service\parsers\domclick\state_parser.py (continued)

```
1350 |
1351 | def _calc_bathroom_count(object_orig: dict[str, Any], object_norm: dict[str, Any]) -> Optional[int]:
1352 |
1353 |     connected = _to_int(_first_not_none(
1354 |
1355 |         object_orig.get("connected_bathrooms"),
1356 |
1357 |         object_norm.get("connectedBathrooms"),
1358 |
1359 |     )) or 0
1360 |
1361 |     separated = _to_int(_first_not_none(
1362 |
1363 |         object_orig.get("separated_bathrooms"),
1364 |
1365 |         object_norm.get("separatedBathrooms"),
1366 |
1367 |     )) or 0
1368 |
1369 |     total = connected + separated
1370 |
1371 |     return total if total > 0 else None
1372 |
1373 | def _detect_author_type(
1374 |
1375 |     source: Any,
1376 |
1377 |     is_owner: Any,
1378 |
1379 |     is_agent: Any,
1380 |
1381 |     company: Any,
1382 |
1383 | ) -> Optional[str]:
1384 |
1385 |     if _to_bool(is_owner) is True and _to_bool(is_agent) is not True:
1386 |
1387 |         return "owner"
1388 |
1389 |     if _to_bool(is_agent) is True:
1390 |
1391 |         return "agent"
1392 |
1393 |     if source == "individual":
1394 |
1395 |         return "owner"
1396 |
1397 |     if isinstance(company, dict) and company:
1398 |
1399 |         return "agency"
1400 |
1401 |     return None
1402 |
1403 | def parking_or_security_keys(keys: list[str]) -> set[str]:
1404 |
1405 |     return set(keys or [])
1406 |
```

src\forecasting_service\scripts\collect_details.py

```
import argparse

from forecasting_service.utils.logging_setup import setup_logging

from forecasting_service.data.collector import DataCollector

def main():

    parser = argparse.ArgumentParser(

        description="Фаза 2: Сбор деталей объявлений"

    )

    parser.add_argument(

        "--db", default="flats.db",

        help="Имя файла БД",

    )

    parser.add_argument(

        "--batch", type=int, default=5,

        help="Макс. объявлений за сессию",

    )

    parser.add_argument(

        "--min-delay", type=float, default=10.0,

        help="Минимальная задержка между запросами (сек)",

    )

    parser.add_argument(

        "--max-delay", type=float, default=15.0,

        help="Максимальная задержка между запросами (сек)",

    )

    parser.add_argument(

        "--restart-every", type=int, default=5,

        help="Рестарт браузера каждые N объявлений",

    )

    parser.add_argument(

        "--headless", action="store_true",

        help="Headless режим (НЕ рекомендуется для деталей)",
```

src\forecasting_service\scripts\collect_details.py (continued)

```
 60 |
 61 |     )
 62 |
 63 |     parser.add_argument(
 64 |
 65 |         "--reset-blocked", action="store_true",
 66 |
 67 |         help="Сбросить blocked → pending перед запуском",
 68 |
 69 |     )
 70 |
 71 |     parser.add_argument(
 72 |
 73 |         "--max-captcha", type=int, default=10,
 74 |
 75 |         help="Макс. CAPTCHA до остановки",
 76 |
 77 |     )
 78 |
 79 |     args = parser.parse_args()
 80 |
 81 |     setup_logging(log_prefix="details")
 82 |
 83 |     if args.headless:
 84 |
 85 |         from loguru import logger
 86 |
 87 |         logger.warning(
 88 |
 89 |             "  Headless режим! CAPTCHA нельзя пройти вручную. "
 90 |
 91 |             "Рекомендуется запуск БЕЗ --headless"
 92 |
 93 |         )
 94 |
 95 |     with DataCollector(db_name=args.db) as collector:
 96 |
 97 |         collector.collect_details(
 98 |
 99 |             batch_size=args.batch,
100 |
101 |             detail_delay=(args.min_delay, args.max_delay),
102 |
103 |             restart_every=args.restart_every,
104 |
105 |             headless=args.headless,
106 |
107 |             reset_blocked=args.reset_blocked,
108 |
109 |             max_captcha=args.max_captcha,
110 |
111 |         )
112 |
113 | if __name__ == "__main__":
114 |
115 |     main()
116 |
```

src\forecasting_service\scripts\collect_listings.py

```
import argparse

from forecasting_service.utils.logging_setup import setup_logging

from forecasting_service.data.collector import DataCollector

from loguru import logger

def _parse_rooms(raw: list[str]) -> tuple:

    result = []

    for r in raw:

        result.append("studio" if r == "studio" else int(r))

    return tuple(result)

def main():

    parser = argparse.ArgumentParser(

        description="Фаза 1: Сбор листинга ЦИАН"

    )

    parser.add_argument(

        "--location", default="Владивосток",

        help="Город (по умолчанию: Владивосток)",

    )

    parser.add_argument(

        "--pages", type=int, nargs=2, default=[1, 54],

        metavar=("START", "END"),

        help="Диапазон страниц",

    )

    parser.add_argument(

        "--rooms", nargs="+", default=["1", "2", "3"],

        help="Комнатность (1 2 3 studio)",

    )

    parser.add_argument(

        "--headless", action="store_true",

        help="Headless режим",

    )
```

src\forecasting_service\scripts\collect_listings.py (continued)

```
 60 |
 61 |     parser.add_argument(
 62 |
 63 |         "--db", default="flats.db",
 64 |
 65 |         help="Имя файла БД",
 66 |
 67 |     )
 68 |
 69 |     parser.add_argument(
 70 |
 71 |         "--source",
 72 |
 73 |         choices=["cian", "domclick"],
 74 |
 75 |         default="cian"
 76 |
 77 |     )
 78 |
 79 |     parser.add_argument(
 80 |
 81 |         "--dc-cookie",
 82 |
 83 |         type=str,
 84 |
 85 |         help="qrator_jsid2 cookie для ДомКлика"
 86 |
 87 |     )
 88 |
 89 |     args = parser.parse_args()
 90 |
 91 |     setup_logging(log_prefix="listings")
 92 |
 93 |     rooms = _parse_rooms(args.rooms)
 94 |
 95 |     with DataCollector(
 96 |
 97 |         location=args.location,
 98 |
 99 |         db_name=args.db,
100 |
101 |     ) as collector:
102 |
103 |         if args.source == "domclick":
104 |
105 |             if not args.dc_cookie:
106 |
107 |                 logger.error("Для ДомКлика обязателен аргумент --dc-cookie")
108 |
109 |                 return
110 |
111 |             from forecasting_service.parsers.domclick.parser import DomclickListingParser
112 |
113 |             from forecasting_service.data.storage import FlatStorage
114 |
115 |             storage = FlatStorage(args.db)
116 |
117 |             parser = DomclickListingParser(qrator_cookie=args.dc_cookie, storage=storage)
118 |
119 |             parser.collect(rooms=rooms, start_page=args.pages[0], end_page=args.pages[1])
```

src\forecasting_service\scripts\collect_listings.py (continued)

```
120 |
121 |             logger.info(f"Итого в БД: {storage.get_stats()}")
122 |
123 |         else:
124 |
125 |             collector.collect_listings(
126 |
127 |                 rooms=rooms,
128 |
129 |                 start_page=args.pages[0],
130 |
131 |                 end_page=args.pages[1],
132 |
133 |                 headless=args.headless,
134 |
135 |             )
136 |
137 | if __name__ == "__main__":
138 |
139 |     main()
140 |
```

src\forecasting_service\scripts\coverage_report.py

```
  1 | import argparse
  2 |
  3 | from forecasting_service.utils.logging_setup import setup_logging
  4 |
  5 | from forecasting_service.data.collector import DataCollector
  6 |
  7 | def main():
  8 |
  9 |     parser = argparse.ArgumentParser(
 10 |
 11 |         description="Отчёт о покрытии датасета"
 12 |
 13 |     )
 14 |
 15 |     parser.add_argument(
 16 |
 17 |         "--db", default="flats.db",
 18 |
 19 |         help="Имя файла БД",
 20 |
 21 |     )
 22 |
 23 |     args = parser.parse_args()
 24 |
 25 |     setup_logging(log_prefix="coverage", console_level="WARNING")
 26 |
 27 |     with DataCollector(db_name=args.db) as collector:
 28 |
 29 |         collector.print_coverage()
 30 |
 31 | if __name__ == "__main__":
 32 |
 33 |     main()
 34 |
```

src\forecasting_service\scripts\debug_domclick.py

```
import requests

QRATOR_COOKIE = "v2.0.1775797162.167.57e14b78fcCOW2mA|lM7O4rTHxVK9cCDB|7BCJdHavBGYsyrNYLYo6PkPtSCc4+ ...

cookies = {

    "qrator_jsid2": QRATOR_COOKIE,

}

headers = {

    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:147.0) Gecko/20100101 Firefox/147.0" ...

    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",

    "Accept-Language": "ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7",

    "Accept-Encoding": "gzip, deflate, br",

    "Sec-Fetch-Dest": "document",

    "Sec-Fetch-Mode": "navigate",

    "Sec-Fetch-Site": "none",

    "Sec-Fetch-User": "?1",

}

url = (

    "https://vladivostok.domclick.ru/search?"

    "deal_type=sale&category=living"

    "&offer_type=flat&offer_type=layout"

    "&aids=57471&rooms=1&offset=0"

)

detail_url = "https://vladivostok.domclick.ru/card/sale__new_flat__2074994630"

resp2 = requests.get(detail_url, headers=headers, cookies=cookies)

print(f"\nDetail Status: {resp2.status_code}")

print(f"Detail Length: {len(resp2.text)}")

print(f"Has building-info: {'building-info-block' in resp2.text}")

with open("domclick_detail_test.html", "w", encoding="utf-8") as f:

    f.write(resp2.text)

```

src\forecasting_service\scripts\debug_html.py

```
  1 | import time
  2 |
  3 | import random
  4 |
  5 | import argparse
  6 |
  7 | from loguru import logger
  8 |
  9 | from forecasting_service.config import DEBUG_DIR
 10 |
 11 | from forecasting_service.utils.logging_setup import setup_logging
 12 |
 13 | from forecasting_service.parsers.common.browser import BrowserManager
 14 |
 15 | DETAIL_URL = "https://vladivostok.cian.ru/sale/flat/327246079/"
 16 |
 17 | DEFAULT_LISTING_URL = (
 18 |
 19 |     "https://cian.ru/cat.php?"
 20 |
 21 |     "engine_version=2&p=1&with_neighbors=0"
 22 |
 23 |     "&region=4701&deal_type=sale"
 24 |
 25 |     "&offer_type=flat&room1=1&only_flat=1"
 26 |
 27 | )
 28 |
 29 | _CAPTCHA_INDICATORS = (
 30 |
 31 |     "captcha", "я не робот", "i'm not a robot",
 32 |
 33 |     "проверка безопасности", "access denied", "заблокирован",
 34 |
 35 |     "подозрительная активность", "security check",
 36 |
 37 |     "чтобы продолжить", "подтвердите", "cloudflare",
 38 |
 39 | )
 40 |
 41 | _CONTENT_INDICATORS = (
 42 |
 43 |     "OfferSummaryInfoGroup", "OfferSummaryInfoItem",
 44 |
 45 |     "О квартире", "О доме", "Общая площадь",
 46 |
 47 |     "Жилая площадь", "Год постройки", "Тип дома",
 48 |
 49 |     "Санузел", "Балкон", "Ремонт", "Отделка",
 50 |
 51 |     "Высота потолков", "Лифт", "Парковка",
 52 |
 53 | )
 54 |
 55 | def _check_captcha_indicators(html: str) -> None:
 56 |
 57 |     html_lower = html.lower()
 58 |
 59 |     logger.info("\n  ПРОВЕРКА ИНДИКАТОРОВ CAPTCHA:")
```

src\forecasting_service\scripts\debug_html.py (continued)

```
 60 |
 61 |     for ind in _CAPTCHA_INDICATORS:
 62 |
 63 |         count = html_lower.count(ind)
 64 |
 65 |         if count > 0:
 66 |
 67 |             _log_indicator_occurrences(html, html_lower, ind, count)
 68 |
 69 |         else:
 70 |
 71 |             logger.info(f"    '{ind}': 0")
 72 |
 73 | def _check_content_indicators(html: str) -> None:
 74 |
 75 |     logger.info("\n  КОНТЕНТ СТРАНИЦЫ:")
 76 |
 77 |     for ind in _CONTENT_INDICATORS:
 78 |
 79 |         count = html.count(ind)
 80 |
 81 |         status = "Good" if count > 0 else "Bad"
 82 |
 83 |         logger.info(f"    {status} '{ind}': {count}")
 84 |
 85 | def _log_indicator_occurrences(
 86 |
 87 |     html: str, html_lower: str, ind: str, count: int,
 88 |
 89 | ) -> None:
 90 |
 91 |     start_pos = 0
 92 |
 93 |     for found_idx in range(min(count, 3)):
 94 |
 95 |         idx = html_lower.find(ind, start_pos)
 96 |
 97 |         if idx == -1:
 98 |
 99 |             break
100 |
101 |         s = max(0, idx - 100)
102 |
103 |         e = min(len(html), idx + len(ind) + 100)
104 |
105 |         context = html[s:e].replace("\n", " ").replace("\r", "").strip()
106 |
107 |         logger.warning(f"    '{ind}' [#{found_idx + 1}]: позиция {idx}")
108 |
109 |         logger.warning(f"      ...{context}...")
110 |
111 |         start_pos = idx + len(ind)
112 |
113 |     if count > 3:
114 |
115 |         logger.warning(f"    ...и ещё {count - 3} вхождений")
116 |
117 | def main():
118 |
119 |     setup_logging(log_prefix="debug", console_level="DEBUG")
```

src\forecasting_service\scripts\debug_html.py (continued)

```
120 |
121 |     DEBUG_DIR.mkdir(parents=True, exist_ok=True)
122 |
123 |     parser = argparse.ArgumentParser(
124 |
125 |         description="Отладка: сохранение и анализ HTML ЦИАН"
126 |
127 |     )
128 |
129 |     parser.add_argument("--detail", action="store_true")
130 |
131 |     parser.add_argument("--url", type=str, default=None)
132 |
133 |     args = parser.parse_args()
134 |
135 |     browser = BrowserManager(headless=False, manual_captcha=True)
136 |
137 |     try:
138 |
139 |         browser.start()
140 |
141 |         if args.detail:
142 |
143 |             url = args.url or DETAIL_URL
144 |
145 |             logger.info("Warm-up: главная ЦИАН...")
146 |
147 |             browser.get_page("https://vladivostok.cian.ru/", scroll=False)
148 |
149 |             time.sleep(random.uniform(4, 7))
150 |
151 |             logger.info(f"Переход: {url}")
152 |
153 |             html = browser.get_page(url, scroll=True)
154 |
155 |             filepath = DEBUG_DIR / "cian_detail_page.html"
156 |
157 |         else:
158 |
159 |             url = args.url or DEFAULT_LISTING_URL
160 |
161 |             logger.info(f"Загрузка листинга: {url}")
162 |
163 |             html = browser.get_page(url, scroll=True)
164 |
165 |             filepath = DEBUG_DIR / "cian_listing_page.html"
166 |
167 |         filepath.write_text(html, encoding="utf-8")
168 |
169 |         logger.info(f"HTML сохранён: {filepath}")
170 |
171 |         logger.info(f"Размер: {len(html)} символов")
172 |
173 |         _check_captcha_indicators(html)
174 |
175 |         if args.detail:
176 |
177 |             _check_content_indicators(html)
178 |
179 |         input("\n  Нажми Enter чтобы закрыть браузер...")
```

src\forecasting_service\scripts\debug_html.py (continued)

```
180 |
181 |     finally:
182 |
183 |         browser.stop()
184 |
185 | if __name__ == "__main__":
186 |
187 |     main()
188 |
```

src\forecasting_service\scripts\domclick\collect_details.py

```
  1 | import argparse
  2 |
  3 | from loguru import logger
  4 |
  5 | from forecasting_service.utils.logging_setup import setup_logging
  6 |
  7 | from forecasting_service.data.storage import FlatStorage
  8 |
  9 | from forecasting_service.parsers.domclick.parser import (
 10 |
 11 |     DomclickListingParser,
 12 |
 13 | )
 14 |
 15 | def main():
 16 |
 17 |     parser = argparse.ArgumentParser(
 18 |
 19 |         description="Сбор деталей DomClick"
 20 |
 21 |     )
 22 |
 23 |     parser.add_argument(
 24 |
 25 |         "--dc-cookie",
 26 |
 27 |         type=str,
 28 |
 29 |         required=True,
 30 |
 31 |         help="qrator_jsid2 cookie из браузера",
 32 |
 33 |     )
 34 |
 35 |     parser.add_argument(
 36 |
 37 |         "--batch",
 38 |
 39 |         type=int,
 40 |
 41 |         default=100,
 42 |
 43 |         help="Макс. объявлений за сессию",
 44 |
 45 |     )
 46 |
 47 |     parser.add_argument(
```

src\forecasting_service\scripts\domclick\collect_details.py (continued)

```
48 |
49 |         "--db",
50 |
51 |         default="flats.db",
52 |
53 |         help="Имя файла БД",
54 |
55 |     )
56 |
57 |     args = parser.parse_args()
58 |
59 |     setup_logging(log_prefix="dc_details")
60 |
61 |     storage = FlatStorage(db_name=args.db)
62 |
63 |     dc_parser = DomclickListingParser(
64 |
65 |         qrator_cookie=args.dc_cookie,
66 |
67 |         storage=storage,
68 |
69 |     )
70 |
71 |     try:
72 |
73 |         dc_parser.collect_details(batch_size=args.batch)
74 |
75 |     finally:
76 |
77 |         dc_parser.close()
78 |
79 |         storage.close()
80 |
81 | if __name__ == "__main__":
82 |
83 |     main()
84 |
```

src\forecasting_service\utils__init__.py

```
1 | from forecasting_service.utils.formatting import progress_bar, format_stats_block
2 |
3 | from forecasting_service.utils.logging_setup import setup_logging
4 |
5 | __all__ = ["progress_bar", "format_stats_block", "setup_logging"]
6 |
```

src\forecasting_service\utils\formatting.py

```
1 | def progress_bar(pct: float, width: int = 20) -> str:
2 |
3 |     filled = int(pct / (100 / width))
4 |
5 |     filled = max(0, min(filled, width))
6 |
7 |     return "█" * filled + "░" * (width - filled)
8 |
9 | def format_stats_block(stats: dict, title: str = "СОСТОЯНИЕ БД") -> str:
```

src\forecasting_service\utils\formatting.py (continued)

```
    |
    |     lines = [
    |
    |         f"\n{'=' * 50}",
    |
    |         f"  {title}:",
    |
    |         f"  Всего:   {stats.get('total', 0)}",
    |
    |         f"  Pending: {stats.get('pending', 0)}",
    |
    |         f"  Done:    {stats.get('done', 0)}",
    |
    |         f"  Failed:  {stats.get('failed', 0)}",
    |
    |         f"  Blocked: {stats.get('blocked', 0)}",
    |
    |         f"{'=' * 50}",
    |
    |     ]
    |
    |     return "\n".join(lines)
    |
    | def format_coverage_block(coverage: dict, total: int) -> str:
    |
    |     if not coverage:
    |
    |         return "  Нет данных о покрытии"
    |
    |     lines = [f"\n  {'Поле':<25} {'Покрытие':>10}",  "—" * 50]
    |
    |     for field, pct in sorted(coverage.items(), key=lambda x: -x[1]):
    |
    |         filled = int(pct / 100 * total)
    |
    |         bar = progress_bar(pct)
    |
    |         lines.append(f"  {field:<22} {bar} {filled:>4}/{total} {pct:>5.1f}%")
    |
    |     return "\n".join(lines)
    |
    | def format_critical_fields(
    |
    |     coverage: dict,
    |
    |     critical_fields: tuple[str, ...],
    |
    | ) -> str:
    |
    |     lines = ["\n  Критичные поля для ML:"]
    |
    |     for field in critical_fields:
    |
    |         pct = coverage.get(field, 0)
    |
    |         status = "Good" if pct >= 75 else "Warning" if pct >= 50 else "Bad"
    |
    |         lines.append(f"  {status} {field:<25} {pct:.1f}%")
    |
    |     return "\n".join(lines)
```

src\forecasting_service\utils\formatting.py (continued)

```
 70 |
 71 | def format_session_report(
 72 |
 73 |     processed: int,
 74 |
 75 |     success: int,
 76 |
 77 |     captcha_count: int,
 78 |
 79 |     multiplier: float,
 80 |
 81 |     stats: dict,
 82 |
 83 |     coverage: dict | None = None,
 84 |
 85 | ) -> str:
 86 |
 87 |     lines = [
 88 |
 89 |         f"\n{'=' * 60}",
 90 |
 91 |         "  РЕЗУЛЬТАТ СЕССИИ",
 92 |
 93 |         f"  Обработано: {processed}",
 94 |
 95 |         f"  Успешно:    {success}",
 96 |
 97 |         f"  CAPTCHA:    {captcha_count}",
 98 |
 99 |         f"  Множитель:  ×{multiplier:.1f}",
100 |
101 |         f"\n  СОСТОЯНИЕ БД",
102 |
103 |         f"  Всего:   {stats.get('total', 0)}",
104 |
105 |         f"  Done:    {stats.get('done', 0)}",
106 |
107 |         f"  Pending: {stats.get('pending', 0)}",
108 |
109 |         f"  Failed:  {stats.get('failed', 0)}",
110 |
111 |         f"  Blocked: {stats.get('blocked', 0)}",
112 |
113 |     ]
114 |
115 |     if coverage:
116 |
117 |         lines.append(f"\n  ПОКРЫТИЕ ПОЛЕЙ")
118 |
119 |         for field, pct in sorted(coverage.items(), key=lambda x: -x[1]):
120 |
121 |             bar = progress_bar(pct)
122 |
123 |             lines.append(f"  {field:<25} {bar} {pct}%")
124 |
125 |     lines.append(f"{'=' * 60}")
126 |
127 |     return "\n".join(lines)
128 |
```

src\forecasting_service\utils\logging_setup.py

```
 1 | import sys
 2 |
 3 | from loguru import logger
 4 |
 5 | from forecasting_service.config import LOGS_DIR
 6 |
 7 | def setup_logging(
 8 |
 9 |     log_prefix: str = "app",
10 |
11 |     console_level: str = "INFO",
12 |
13 |     file_level: str = "DEBUG",
14 |
15 | ) -> None:
16 |
17 |     LOGS_DIR.mkdir(parents=True, exist_ok=True)
18 |
19 |     logger.remove()
20 |
21 |     logger.add(
22 |
23 |         sys.stderr,
24 |
25 |         format=(
26 |
27 |             "<green>{time:HH:mm:ss}</green> | "
28 |
29 |             "<level>{level:<8}</level> | {message}"
30 |
31 |         ),
32 |
33 |         level=console_level,
34 |
35 |     )
36 |
37 |     logger.add(
38 |
39 |         str(LOGS_DIR / f"{log_prefix}_{ time:YYYY-MM-DD} .log"),
40 |
41 |         level=file_level,
42 |
43 |         rotation="10 MB",
44 |
45 |     )
46 |
```